# Automatic Vehicle Gate Access System

Final Year Project Report

Presented By

**Abdul Hadi**
CIIT/FA22-BCE-018/LHR

**Ali Raza Chaudhry**
CIIT/FA22-BCE-102/LHR

**Abdullah Yaseen**
CIIT/FA22-BCE-065/LHR

In Partial Fulfillment of the Requirement for the Degree of
*Bachelor of Science in Computer Engineering*
DEPARTMENT OF COMPUTER ENGINEERING

**COMSATS UNIVERSITY ISLAMABAD,**
**Lahore Campus,**
**July 2026**

# *Declaration*

We, hereby declare that this project has been copied out from any source either as a whole or as a part thereof. It is further declared that we have developed this project and the accompanying report entirely based on our personal efforts made under the sincere guidance of our supervisor. No portion of the work presented in this report has been submitted in the support of any other degree or qualification of this or any other University or Institute of learning, if found we shall stand responsible.

Signature: _____
Abdul Hadi
Date: _____

Signature: _____
Ali Raza Chaudhry
Date: _____

Signature: _____
Abdullah Yaseen
Date: _____

**COMSATS UNIVERSITY ISLAMABAD,**

**Lahore Campus,**

**July 2026**

# Automatic Vehicle Gate Access System

An Undergraduate Final Year Project Report submitted to the Department of COMPUTER ENGINEERING As a Partial Fulfillment for the award of Degree *Bachelor of Science in Computer Engineering*

*by*


| Name | Registration Number |
|---|---|
| Abdul Hadi | CIIT/FA22-BCE-018/LHR |
| Ali Raza Chaudhry | CIIT/FA22-BCE-102/LHR |
| Abdullah Yaseen | CIIT/FA22-BCE-065/LHR |

**Supervised by**

Dr. Taimoor Naeem
Assistant Professor,
Department of Computer Engineering


**COMSATS UNIVERSITY ISLAMABAD,**

**Lahore Campus,**

**July 2026**

# *Final Approval*

***This Project Titled " Automatic Vehicle Gate Access System" Submitted for the Degree of***

*Bachelor of Science in Computer Engineering*

*by*

| **Name** | **Registration Number** |
|---|---|
| Abdul Hadi | CIIT/FA22-BCE-018/LHR |
| Ali Raza Chaudhry | CIIT/FA22-BCE-102/LHR |
| Abdullah Yaseen | CIIT/FA22-BCE-065/LHR |

*has been approved as partial fulfillment for the award of bachelor's degree.*

| | |
|---|---|
| Supervisor | Internal Examiner-1 |
| Dr. Taimoor Naeem, | Mr. Moazzam Ali Sahi, |
| Assistant Professor | Lecturer |
| *Internal Examiner-2* | *External Examiner* |
| *Dr. Ikram Ullah Khosa,* | *Name,* |
| *Assistant Professor* | *Designation* |

*Head*

*Department of Computer Engineering*

**COMSATS UNIVERSITY ISLAMABAD,**

**Lahore Campus,**

**July 2026**

# *Dedication*

To our parents, teachers, and mentors, whose constant support, patience, and encouragement made this work possible; and to the engineering community striving to make everyday infrastructure safer, smarter, and more accessible through technology.

# *Acknowledgements*

In the name of God, the Most Kind and Most Merciful. We would like to thank our parents, who always backed us up, both financially and morally, throughout our studies.

We would also like to thank Dr. Taimoor Naeem for his guidance and for encouraging us to work hard and smart. We found him very helpful while discussing the optimization issues in this dissertation work. His critical comments on our work have certainly made us think of new ideas and techniques in the fields of optimization, software, and hardware simulation.

We are grateful to God Almighty, who provides all kinds of resources to us so that we may use them properly for the benefit of mankind. May He continue to provide us with all the resources and the guidance to keep helping humanity.

Abdul Hadi
Ali Raza Chaudhry
Abdullah Yaseen

# *Table of Contents*

**1. INTRODUCTION ..... 1**

1.1 Historical Background ..... 1
1.2 Objectives and Limitations ..... 2
1.3 Project Scope ..... 3
1.4 Assumptions and Dependencies ..... 4
1.5 Broader Impact (UN SDGs) ..... 4

1.6 Thesis Key Contributions ..... 6
*1.6.1 Thesis Outline ..... 6*

**2. LITERATURE REVIEW ..... 7**
2.1 Related Technologies ..... 7
*2.1.1 Object Detection Networks ..... 7*
*2.1.2 Character Recognition Approaches ..... 7*
*2.1.3 Image Enhancement and Preprocessing ..... 8*
*2.1.4 Vehicle Make and Model Recognition (VMMR) ..... 8*

2.2 Overview of Existing Systems ..... 9
*2.2.1 Commercial ALPR Systems ..... 9*
*2.2.2 Academic ALPR Research (2023–2025) ..... 9*

2.3 Limitations of Existing Work ..... 10
**3. MODELING AND SIMULATION ..... 12**
3.1 Conceptual Framework ..... 12
3.2 High Level Design ..... 12
*3.2.1 Block Diagram ..... 12*
*3.2.2 Flow Diagram ..... 15*
*3.2.3 Performance ..... 16*

3.3 Subsystem Design ..... 16
*3.3.1 List of Subsystems ..... 16*
*3.3.2 Flowchart of Subsystems ..... 17*

3.4 Low Level Design and Component Selection ..... 18
*3.4.1 Image Acquisition — USB CAMERA ..... 18*
*3.4.2 License Plate Detection — YOLOv8 Detection Model ..... 18*
*3.4.3 Image Preprocessing — OpenCV Image Processing Library ..... 19*
*3.4.4 YOLO — Character and Digit Detection ..... 19*
*3.4.5 Vehicle Classification — EfficientNet-B4 ..... 19*
*3.4.6 Authorization — JSON-based Sypabase ..... 19*
*3.4.7 Embedded Processing — Orange Pi 5 ..... 19*
*3.4.8 User Interface — Web-Based Dashboard ..... 19*
3.5 System Architecture and Module Interaction ..... 20

**4. IMPLEMENTATION AND TESTING ..... 24**

4.1 Software Implementation ..... 24
*4.1.1 License Plate Detection Using YOLOv8 ..... 24*
*4.1.2 Adaptive Image Preprocessing ..... 24*
*4.1.3 License Plate Text Recognition ..... 24*
*4.1.4 Vehicle Make and Model Classification ..... 25*
*4.1.5 Authorization Logic ..... 25*
*4.1.6 Visualization and Logging ..... 26*

4.2 Testing Procedures ..... 26
*4.2.1 Dataset Description ..... 26*
*4.2.2 Testing Methodology ..... 27*

4.3 Edge Deployment and Hardware Setup ..... 28
*4.3.1 Hardware Components and Edge Deployment Setup ..... 28*
*4.3.2 Pipeline Time: ..... 33*
4.4 Web Dashboard Implementation ..... 34
**5. RESULTS AND DISCUSSION ..... 42**
5.1 License Plate Detection Results ..... 42
*5.1.1 Dataset: ..... 42*
*5.1.2 Training Configuration ..... 43*
*5.1.3 Detection Performance ..... 44*

5.2 Character Recognition Results ..... 48
*5.2.1 Dataset ..... 48*
*5.2.2 Training Configuration ..... 51*
*5.2.3 Recognition Performance ..... 51*
*5.2.4 Post Processing: ..... 54*

5.3 Vehicle Make and Model Recognition (VMMR) Results ..... 58
*5.3.1 Dataset Details ..... 59*
*5.3.2 Training Configuration ..... 59*
*5.3.3 Classification Performance ..... 60*
5.4 Preprocessing Module ..... 75
*5.4.1 Fog Removal Pipeline ..... 75*
*5.4.2 Rain Removal Pipeline ..... 78*
*5.4.3 Night / Low-Light Enhancement Pipeline ..... 79*

5.5 Integrated Pipeline Results ..... 80
*5.5.1 Pipeline Results on Foggy Images ..... 81*
*5.5.2 Pipeline Results on Rainy Imamges ..... 81*
*5.5.3 Pipeline Results on Night-Enhanced Images ..... 82*
*5.5.4 End-to-End Performance Summary ..... 83*
5.6 Model Comparison and Benchmarking ..... 84
6.1 Summary ..... 84
6.2 Key Achievements ..... 85
6.3 System Performance Summary ..... 86
6.4 Limitations ..... 86
6.5 Future Work ..... 87
6.6 Conclusion ..... 87
**APPENDICES ..... 88**
Appendix A — Sustainable Development Goals Achievement ..... 88
Appendix B — HDL / C Source Code ..... 88
Appendix C — List of Components ..... 89
Appendix D — Project Timeline ..... 91
**REFERENCES ..... 93**

# *List of Figures*

Figure 3.2.1: High-Level Block Diagram ..... 15
Figure 3.2.2: Flowchart of the Proposed System ..... 16
Figure 3.3.1: Subsystem Diagram ..... 18
Figure 3.4 — Prototype Application Diagram ..... 20
Figure 3.5 — System Architecture Diagram ..... 21
Figure 3.5.1 — Class Diagram ..... 22
Figure 3.5.2 — ER Diagram ..... 23
Figure 4.3.1: Batch Image Processing and Authorization Log (48 Images, Orange Pi 5 Pro) .... 30
Figure 4.3.2 — Real-Time Video Frame Inference and Dashboard Synchronization (Orange Pi 5 Pro) ..... 31
Figure 4.3.3: Hardware Diagram ..... 32
Figure 4.3.4: Orange Pi Live Pipeline Demonstration (Plate LEB-1481) ..... 33
Figure 4.4: Dual-Modality Access Control Flowchart ..... 37
Figure 4.4.1 — Web Application Frontend Admin Panel ..... 38
Figure 4.4.2 — Web Application Frontend User Panel ..... 39
Figure 4.4.3 — Web Dashboard Detection Event Card (Plate LEH-8357, Suzuki Wagon R — Allowed) ..... 40
Figure 4.4.4 — Web Dashboard Detection Event Card (Plate BQX-687, Changan Alsvin — Denied) ..... 41
Figure 5.1 — YOLOv8 License Plate Detection: Training and Validation Metrics Over 50 Epochs ..... 45
Figure 5.2 — YOLOv8 License Plate Detection: Performance Metrics Heatmap (F1, Recall, .. 46
Figure 5.3 — YOLOv8 License Plate Detection: Test Images ..... 47
Figure 5.4 — YOLOv8 License Plate Detection: Confusion Matrix ..... 47
Figure 5.5: YOLOv8n License Plate Detection: Sample Inference Results..... 48
Figure 5.5 — Character Recognition Dataset: Rob flow Interface Showing 1,235 Annotated Plate Images ..... 51
Figure 5.6 — Character Recognition: Training and Validation Metrics Over 100 Epochs The curves show how the character recognizer's loss and accuracy change with each of 100 epochs. Training and validation losses are shown to be steadily declining with precision, recall, and mAP steadily increasing and plateauing after ~50 epochs, showing that learning is steady without overfitting. ..... 52
Figure 5.7 — Character Recognition: Confusion Matrix (38 Classes) ..... 53
Figure 5.8 — Character Recognition: Validation Performance Metrics Heatmap (Accuracy, Precision, Recall, F1, FPR) ..... 54
Figure 5.9 — End-to-End Plate Detection and Recognition Results on Test Images ..... 55
Figure 5.10 — YOLOv8 Character Localization: Sample Detection on AFR-024 Plate ..... 55
Figure 5.11 — YOLOv8 Character Localization: Sample Detection on LER-1234 Punjab Plate ..... 56
Figure 5.12 — YOLOv8 Character Localization: Sample Detection on AFR-021 Sindh Plate . 56
Figure 5.13 — Character Recognition: Per-Character Detection and OCR Output on Diverse Pakistani Plates ..... 57
Figure 5.14: Character Recognition Dataset: Annotated Plate Images ..... 58
Figure 5.15 — VMMR: Training and Validation Loss, Accuracy, and Learning Rate Over 50 Epochs ..... 61
Figure 5.16 — VMMR: Test Metrics Heatmap ..... 62
Figure 5.17 — VMMR: Full Confusion Matrix (All 245 Classes) ..... 63
Figure 5.18 — VMMR: Sample Test Predictions with Top-5 Probabilities ..... 73
Figure 5.19 — VMMR: Sample Test Predictions with Confidence Scores ..... 74

Figure 5.20 — Original Foggy Vehicle Images ..... 77
Figure 5.21 — After Fog Removal (Preprocessed) ..... 77
Figure 5.22 — Original Rainy Vehicle Images ..... 78
Figure 5.23 — After Rain Removal (Preprocessed) ..... 79
Figure 5.24 — Original Night / Low-Light Vehicle Images (Before Enhancement) ..... 80
Figure 5.25 — Integrated Pipeline Results on Fog-Preprocessed Images ..... 81
Figure 5.26 — Integrated Pipeline Results on Rain-Preprocessed Images ..... 82
Figure 5.27 — Integrated Pipeline Results on Night-Enhanced Images ..... 83
Figure — Appendix E: Gantt Chart ..... 92

# *List of Tables*

Table 1.1 — SDGs in the Light of Objectives ..... 5
Table 2.3 — Literature Review Summary ..... 10
Table 4.1 — Edge AI Pipeline Latency Breakdown (Orange Pi CPU Inference) ..... 33
Table 5.1 — Yolo Licene Plate Dataset Detail ..... 42
Table 5.2 — Yolo Licene Plate Dataset Size ..... 43
Table 5.3 — License Plate Detection: Training Configuration ..... 43
Table 5.4 — License Plate Detection: Performance Metrics ..... 44
Table 5.4.1 — License Plate Detection: Training Progression (YOLOv8n, 50 Epochs) ..... 45
Table 5.5 — License Plate Detection: Performance Metrics ..... 49
Table 5.6 — Character Digits Detection 38 Classes (0-9 Digits, A-Z Characters) ..... 50
Table 5.7 — Character Recognition: Training Configuration ..... 51
Table 5.8 — Character Recognition: Performance Metrics ..... 51
Table 5.8.1 — Character Recognition: Training Progression (YOLOv8n, 100 Epochs) ..... 53
Table 5.9 — VMMR: Dataset Summary..... 59
Table 5.10 — VMMR: Training Configuration ..... 60
Table 5.11 — VMMR: Test Set Performance Metrics ..... 60
Table 5.12 — VMMR: Training Progression ..... 61
Table 5.13 — VMMR Classes 245 Description ..... 72
Table 5.13.1 — Layer-wise Working of YOLOv8n and EfficientNet-B4 ..... 75
Table 5.14 — Integrated Pipeline: End-to-End Performance Summary ..... 84
Table 5.15 — Comparison with Existing Systems ..... 84
Table 5.16— Overall System Performance Summary ..... 86

# *List of Symbols, Abbreviations and Acronyms*

| Abbreviation | Meaning |
|---|---|
| **ALPR** | Automatic License Plate Recognition |
| **VMMR** | Vehicle Make and Model Recognition |
| **YOLO** | You Only Look Once |
| **OCR** | Optical Character Recognition |
| **CNN** | Convolutional Neural Network |
| **CLAHE** | Contrast Limited Adaptive Histogram Equalization |
| **DCP** | Dark Channel Prior |
| **IR** | Infrared |
| **mAP** | Mean Average Precision |
| **IoU** | Intersection over Union |
| **FPR** | False Positive Rate |
| **GPU** | Graphics Processing Unit |
| **SDG** | Sustainable Development Goal |
| **FYP** | Final Year Project |
| **NMS** | Non-Maximum Suppression |
| **ESP** | Espressif Microcontroller |
| **ER** | Entity Relationship |
| **API** | Application Programming Interface |

# *Abstract*

The aim of this project is to design a Smart Vehicle Gate Access System that can minimize manual checking for gate access in gated communities, institutional campuses and parking areas. The system integrates a weather-aware pre-processing front end, a dual-stage YOLOv8n license plate recognition (LPR) pipeline and a vehicle make, and model recognizer (VMMR) based on a finetuned EfficientNet-B4 convolutional neural network with 245 vehicle models. Three separate branches are used for the restoration: a Dark Channel Prior–based dehazing pipeline with guided filtering for fog and haze; a multi-scale vertically oriented median filter with edge-aware protection and Telea inpainting for rain streak suppression; and a LAB-space Contrast Limited Adaptive Histogram Equalization (CLAHE) pipeline for night and low-light enhancement. Lightweight, Pakistani-alphanumeric character trained YOLOv8n detectors are utilized for license plate localization and 38-class alphanumeric character detection, and deterministic post processing is used to construct region-specific plate tokens. Access is granted via a dual-modal policy that cross-references the reconstructed license plate text, and the predicted vehicle make and model against a registered vehicle database, adding another layer of verification that is not susceptible to plate spoofing and plate-vehicle swapping attacks. All three models are deployed as edge AI on an Orange Pi 5 Pro (16 GB), and all process the data from a single, onboard USB camera, at the gate lane, with a consistent Python inference pipeline, with the recognized text of the plate, vehicle ID, weather context, and annotated snapshots sent over WiFi to a production React dashboard running on Vercel via a secure gateingest API. The web platform provides live feed monitoring, event logging, vehicle registration, administrative override and automated allow/deny gate actions based on the registry rules and confidence thresholds. On held-out test data, the on-device inference time is around 700–1000 ms per frame on the Orange Pi CPU, and achieved a quantitative evaluation result of 98.07% mAP@50 for license plate detection, 98.07% mAP@50 for character recognition, and 96.24% Top-1 for vehicle classification, covering a full hardware–software pipeline of image capture in adverse weather conditions, followed by character recognition and, finally, access control via the cloud.

# CHAPTER 01

# 1. Introduction

In secure areas like car parks, gated communities, business or institutional campuses, an integral part of the challenge is vehicle identification. The traditional methods of credential verification (manual, RFID authentication, barcode passes) are labour intensive, prone to human error, easily misused and hard to scale to high traffic entry points. New technology in computer vision and deep learning has enabled the ability to recognize vehicles contactlessly via camera imagery, which presents a scalable, auditable solution to physical credential-based access control.

The Automatic Vehicle Gate Access System proposed in our Final Year Project combines the capabilities of detecting license plates, fine-grained vehicle make and model recognition, and character recognition from the license plate using the YOLOv3 series. It is designed to be reliable in the real world, supporting fog, rain, and low light conditions, with a pre-processing dedicated to weather-awareness before being applied to the inference. It enhances security over plate-only systems with a dual-modality authorization policy, which cross-referenced discovered text on the plate and predicted vehicle identity from a registered database. All three AI models run locally on an Orange Pi 5 Pro edge node, data is synced in real time to a production React dashboard hosted on Vercel for live monitoring, event logging and administrative control, and the complete solution is a fully integrated hardware–software platform that is ideal for gated-community and campusstyle deployment.

# 1.1 Historical Background

Artificial intelligence, computer vision and edge computing have made much ground in recent years, resulting in significant advances in the evolution of intelligent vehicle gate systems design. Manual credential checks, using RFID and barcode passes are slow, manual and insecure if lost, duplicated or accessed by an unauthorized user. The restrictions are more stringent in high traffic areas such as parking lots, neighborhood areas, offices, and facilities. It's not very efficient if there is delay and costs.

To address these issues, contemporary smart gates are increasingly implementing Automatic Number Plate Recognition (ANPR) methods for a contactless way of identifying vehicles and authorizing entry. The first ANPR solutions relied on Optical Character Recognition (OCR) engines to recognize the license plate text after the localization of the license plate. In controlled conditions, OCR based methods would be effective but they tend to fail when there is a change of font types, variations in image quality, plate orientation, motion blur, and bad weather. In recent years, deep learning has advanced significantly and object detection techniques like YOLO have been introduced which can accurately detect the location of the license plate and can directly detect and classify each alphanumeric character without relying on traditional OCR engines.

On the other hand, Convolutional Neural Networks (CNN) have been recently integrated into Vehicle Make and Model Recognition (VMMR), another important scope of vehicle verification. These systems can be used to counter plate–vehicle swapping (or spoofing) attacks which are not

possible to detect using just plate recognition. Reliable deployment outside is still challenging, due to the impact of the environmental conditions such as fog and low light, which can cause a significant degradation of image quality and recognition accuracy.

The aim of this proposed project is to develop an All-Weather Intelligent Vehicle Gate Recognition System with the intelligent fusion of the proposed image enhancement algorithm of weatheraware image, dual-stage YOLOv8n License Plate Recognition (LPR), YOLOv8n character and digit detection for 38 alphanumeric classes, and the EfficientNet-B4-based vehicle make model recognition for 245 vehicle categories. The system is fitted on Orange Pi 5 Pro edge platform and combined with a cloud-synchronized monitoring dashboard for a full endtoend solution for secure, automated and weather-proof vehicle access control.

# 1.2 Objectives and Limitations

The goal of this project is to investigate computer vision and deep learning powered automated vehicle access control. The essential purposes are:

- Develop a real-time unit for YOLO based license plate detection for the purpose of license plate identification and location in vehicle image.
- Learn to write an intelligent YOLOv8 pipeline for YOLO Character and Digit Detection and pre-processing techniques like denoising, histogram equalization, CLAHE, detail enhancement and sharpening.
- Use EfficientNet-B0 for vehicle type detection - make and model classification.
- Add authorization logic which cross-checks the text of the identified license plate and the vehicle model to an authorized vehicles store.
- Evaluation of test performance under different conditions (day/night, including glare, fog, rain and motion blur) with above 90% accuracy.
- Real-time gate processing, total system response time < 2 seconds (image capture to access decision using AI inference).

**Limitation**s:

- Full industrial gate-motor integration is site-specific and beyond the scope of the academics; GPIO/LED gate indication is supported.
- Working on Sun Glare Removal and Snow Removal from image.
- However, the accuracy of recognition depends on the quality of the camera and weather conditions and can still be reduced in extreme weather conditions.

# 1.3 Project Scope

This project offers a full chain smart vehicle gate recognition system encompassing model training, weather-sensitive preprocessing, edge deployment and cloud monitoring.

**Software scope includes:**

- The module for license plate detection is YOLOv8n.
- YOLOv8n character and digit recognition (38 classes) with adaptive weather preprocessing ( fog, rain, night )
- Convolutional neural network EfficientNet-B4 for classification of vehicle make and model
- (245 classes)
- Permission control logic associated to a database of registered vehicles (allow / deny / temporary open/close gate) with automatic gate control decision
- Web dashboard built using React + Node.js/Express and deployed on Vercel (Live Feed, event logs, vehicle registry, admin override)
- Orange Pi integration – Pi processes all three AI models locally and uploads the results to the cloud using gate_sync.py.
- Hardware scope includes:
- Edge AI node: Orange Pi 5 Pro (16 GB)
- 1080p camera with USB for images at the gate-lane
- Secure upload of results to Vercel via Wi-Fi.
- Optional GPIO/LED to indicate physically the gate status.
- Precision, recall, mAP@0.5 and F1 are metrics used for the detection modules, while Top1/Top-5 accuracy is used for VMMR and the on-device end-to-end latency on the Orange Pi.

# 1.4 Assumptions and Dependencies

**Assumptions and dependencies of the project are:**

- On both development workstations (Google Colab/CUDA) and the Orange Pi 5 Pro edge node, the following software is installed and configured: Python, PyTorch, OpenCV, and Ultralytics YOLO. Model weights are shared through Hugging Face and loaded dynamically both by models_loader.py and the unified pipeline.py.
- System Integration: All modules (YOLOv8n license plate detector, YOLOv8n character/digit recognizer, EfficientNet-B4 vehicle classifier, weather preprocessing, authorization logic and Vercel upload) are integrated into a single pipeline on Pi (pipeline.py), and cloud synchronization is performed by gate_sync.py.
- Network and Security: Wi-Fi at deployment site for uploading detection events. A shared GATE_DEVICE_TOKEN authenticates Pi-to-cloud communication through the
- Vercel's /api/gate/ingest endpoint.
- Performance Assumption: The Orange Pi CPU can complete full inference (preprocess →
- LPD (OCR → VMMR) within ≤ 2 seconds per captured frame (~700 – 1000ms measured).
- Data Accuracy: The access decisions rely on a correct registry, which is maintained on the authorized vehicles, temporary visitors and allow/deny/temp policies through the web dashboard. Based on plate match and vehicle confidence the system returns gate_action: opened or closed.
- Image Quality: Input images are expected to have a visible plate area and have enough surrounding vehicle to use for VMMR.

.

# 1.5 Broader Impact (UN SDGs)

Several UN Sustainable Development Goals are addressed by this project:

**SDG 9 —** Industry, Innovation, and Infrastructure: Integrates YOLO (licence plate recognition), YOLO (Character and Digit Detection), and CNN (vehicle recognition) to create smart infrastructure system for access security.

**SDG 11 —** Sustainable Cities and Communities: Reduces traffic jams by optimizing vehicle access to residential, corporate and public establishments.

**SDG 12**: Responsible Consumption and Production — Automated recognition cuts down on vehicle idle time, digital monitoring enables resource management.

**SDG 13** — Climate Action: AI models and image processing consume less energy and less vehicle idle time results in fewer carbon emissions.

**Table 1.1 — SDGs in the Light of Objectives**

| SDG | Included | Level | Project Goal | How Goal Meets SDG | Remarks |
|---|---|---|---|---|---|
| SDG 3: Good Health | No | N/A | N/A | N/A | N/A |
| SDG 4: Quality Education | Yes | Average | Practical AI/CV learning platform | Allows students to explore real-world ML/CV pipelines | Promotes hands-on STEM learning |
| SDG 8: Decent Work | No | N/A | N/A | N/A | N/A |
| SDG 9: Industry, Innovation | Yes | Major | AI-driven automated vehicle gate system | Introduces YOLO, OCR, CNN for infrastructure automation | Scalable to gated communities and industrial facilities |
| SDG 11: Sustainable Cities | Yes | Average | Urban safety and efficient vehicle access control | Reduces human errors and improves traffic flow | Can be extended for smart city projects |
| SDG 12: Responsible Consumption | Yes | Partial | Minimize paperbased logs via digital processing | Maintains digital logs and automated decision-making | Expandable with IoT energy-efficient components |

This table maps the project's objectives onto the relevant United Nations Sustainable Development Goals, showing how the system's technical aims align with broader social and infrastructural impact.

# 1.6 Thesis Key Contributions

This research has two important contributions. First, it introduces an intelligent framework for the management of vehicles access control based on multiple artificial intelligence techniques such as

Three key steps involved: YOLObased license plate localization, YOLO driven character and digit recognition, and Convolutional Neural Network (CNN) based vehicle make and model classification. The system consists of a combination of these elements that are integrated into a single software system, allowing for automated and efficient vehicle verification that can be deployed in real-world settings.

Second, the study conducts an extensive evaluation of the proposed system under various operating conditions including different illuminations, reflections and moderate weather disturbances.

# 1.6.1 Thesis Outline

The theoretical background and literature review are presented in Chapter 2, focusing on automatic license plate recognition, vehicle make and model recognition and the intelligent access control system. It also points out the existing gaps and restrictions in approaches.

The design and development of the proposed system is explained in chapter 3. Describes the general approach, system requirements, technologies chosen and the general system architecture for the solution.

Chapter 4 explains the operation of the software modules, which include the license plate detection, character and digit detection, classification of vehicle makes and models, and access control system. System testing procedures, performance assessment and experimental results are also addressed in the chapter.

Chapter 5 presents an analysis and discussion of the results obtained from the study, which includes the effectiveness, strengths and limitations of the proposed framework. A comparison is also made with existing solutions to assess the overall performance.

The research findings and contribution are summarized in chapter 6. It also presents possible future upgrades, hardware integration options, and suggestions for making the system more capable and feasible..

# CHAPTER 02

# 2. Literature Review

Main technologies and research areas of Automated License Plate Recognition (ALPR), Vehicle Make and Model Recognition (VMMR) and intelligent vehicle access control systems are discussed in this chapter. It provides an overview of the up-to-date techniques in the field of object detection,

YOLO based models, image enhancement techniques and vehicle classification models. An analysis on the existing commercial and academic solutions is also presented which indicates the gaps on the research space and justifies the design decisions taken in the proposed system.

## 2.1 Related Technologies

Vehicle authentication and Gate access is proposed to be automated using recent computer vision and deep learning techniques. The YOLO based localization model is employed to rapidly and accurately localize the license plates and a separate YOLO Character and Digit Detection model is employed to detect the alphanumeric information from the localized license plates. In order to enhance the recognition performance under harsh environmental conditions, a number of image enhancement techniques are used prior to inference. In addition, a deep learning vehicle classification model is added to give an extra verification layer which identifies the make and model of the vehicle.

### 2.1.1 Object Detection Networks

Object detection is a basic part of the modern computer vision systems. In this context, the YOLO (You Only Look Once) family of approaches have become popular due to the ability of localising and performing classification in a single forward pass through the network. Unlike traditional two-stage detectors which propose regions and classify them afterwards, YOLO directly outputs the location and category of an object from the input image.

Since its inception, YOLO has come a long way, and there are notable improvements in the feature extraction, detection accuracy, and efficiency of YOLOv8. These properties make YOLO particularly useful in applications such as licence plate recognition where the plates are likely to be a relatively small portion of the image. YOLO is a multi-scale feature learning model which can detect license plate robustly at different scale, view and environment with real-time performance.

### 2.1.2 Character Recognition Approaches

Character recognition is one of the most crucial processes of any ALPR system. Generally, two main approaches are used for textual information extraction of a license plate.

**Classical OCR Methods**

In most of the traditional Optical Character Recognition (OCR) systems, character segmentation and feature based classification are followed. These techniques are suitable for images containing well separated and aligned text. They are however poor in performance on non-standard fonts,

distorted images, poor lighting and the distances between characters, which are common in the real world traffic scenarios.

**Deep Learning-Based Recognition**

With emergence of deep learning techniques, character recognition have been significantly improved in recent times. The proposed system will employ the YOLO Character and Digit Detection model which will directly detect individual character and digits from the license plate image. The characters identified are then sorted based on their spatial information to get the complete sequence of the license plate. This detection based method has shown to be more tolerant of blur, low resolution, partial occlusion and illumination changes than are conventional OCR techniques.

**The enhancement and pre-processing of images.**

Outdoor scenes come with a host of imaging issues, such as noise, motion blur, uneven illumination, glare, and bad weather. To overcome these problems, several pre-processing methods are used prior to the detection and recognition processes.

- Noise Reduction: Bilateral Filtering and NonLocal Means are applied to reduce noise without compromising on important edges.
- Contrast Enhancement: Using the Contrast Limited Adaptive Histogram Equalization (CLAHE) makes it easier to see in low-light and glare situations and reduces the artifacts.
- Detail Enhancement: Morphological processing and Edge Enhancing techniques enhance the characters of the license plate to make them more distinguishable.
- and their background.
- Image Sharpening: Sharpening filter like Laplacian based filter enhances the readability by sharpening the character boundaries while not too much enhancing the noise.

**Recognizing Vehicle Make and Model (VMMR)**

Vehicle Make and Model Recognition is a fine-grained image classification task, where the difference between the similar class needs to be determined out. The current way of tackling this challenge is the Deep Convolutional Neural Networks (CNNs) that can learn discriminative visual features automatically.

**ResNet18**

ResNet18 is a residual network (ResNet) which uses residual learning and skip connections to aid the training of deep neural networks. These architectural features help to alleviate optimization challenges and enhance the classification performance. The model has a relatively medium-low computational requirements that can be used to identify common vehicle classes.

**EfficientNet-B0**

EfficientNet-B0 is an efficient balance of performance and accuracy by compound scaling of the network depth, width, and resolution. In conjunction with data augmentation and transfer learning, it yields a high performance over different vehicle poses, lighting conditions and occlusion rates. It is also light, and thus can be deployed on edge-computing platforms.

The current systems are briefly described below:

# 2.2.1 Commercial ALPR Systems

Automated License Plate Recognition (ALPR) is now widely adopted by commercial applications in the field of traffic monitoring, toll collection and parking management. Usually systems in the commercial market have a high recognition rate in a controlled environment and can include integrated management features. But, some are hardware-dependent or require cloud infrastructure for deployment, which can reduce flexibility and contribute to deployment costs and their performance may suffer in harsh weather.

# 2.2.2 Academic ALPR Research (2023–2025)

In the recent studies, there is attempt to improve the accuracy and robustness of the recognition under real world conditions.

YOLOv5 with OCR based recognition is combined, which is efficient but the performance is insufficient in the low-light condition –Thapliyal et al. (2023).

Li et al. (2023): suggested a segmentation-based framework with CNN recognition modules, and it can enhance the detection performance in a complicated scene and is sensitive to severe weather effects.

Agarwal et al. (2024): Combined YOLO with transformer-based models to allow for the recognition of plates in a flexible manner, which added complexity to the computation.

To improve the blurred and weathered image recognition, Chen et al. (2024) suggested image enhancement techniques.

Semihomemade et al. (2025) introduced a modified ALPR system featuring vehicle recognition modules that enhance vehicle verification through multiple factors, thereby boosting the system's security capabilities.

Literature review on the subject revealed that there are no such studies yet.

While there have been some good advances in studies of ALPR and VMMR, there are also some challenges which have yet to be addressed.

1. **Environmental Sensitivity**: Performance in recognition can drop significantly when affected by fog, rain, high reflected light and low light levels.

**2. Single-Factor Authentication**: Certain systems rely on just license plate verification, which may be vulnerable to fake or duplicate license plates.

3. **Computational Demands**: Highly performance detection and tracking model may require a lot of computational resources that may hinder the deployment in low cost hardware.

4. L**imited System integration** – a lot of literature is of individual recognition components and not full Vehicle access control solutions.

5. **Dataset Generalization Issu**es: Some dataset are publicly available with a major contribution of the European, North American or Chinese vehicle and license plate formats, which limits the adaptability to the traffic environment in South Asia..

**Table 2.3 — Literature Review Summary**

| Ref No. | System / Paper (Year) | Approach | Main Objective | Key Technologies | Challenges Addressed | Limitations |
|---|---|---|---|---|---|---|
| 1 | **Proposed System** (2025–2026) | Hybrid Edge–Cloud Pipeline (Preprocessing → Dual YOLOv8n LPR → EfficientNet-B4 VMMR → Database) | Automated gate access control using license plate and vehicle make/model recognition with <2s latency. | • Orange Pi 5 Pro<br>• YOLOv8n<br>• EfficientNet-B4<br>• DCP/CLAHE<br>• React + Node.js<br>• SQLite / MySQL | Weather adaptation (fog, rain, night), Pakistani plates, cloned-plate detection, edge deployment. | ~1 FPS on Orange Pi 5 Pro, Wi-Fi dependency, limited edge resources. |
| 2 | **Thapliyal et al.** (2023) | YOLOv5 + Tesseract OCR | License plate detection and recognition. | • YOLOv5<br>• Tesseract OCR | Good performance on clear images. | Poor performance under glare and low-light conditions. |
| 3 | **Chinese LPR** (2023) | Traditional Computer Vision + Edge Detection | Plate localization. | • Sobel Edge Detection<br>• Morphological Operations | Low implementation cost. | Weak performance on angled and complex plates. |
| 4 | **MDPI Qatar** (2024) | YOLOv8 + CRNN | Multi-national plate recognition. | • YOLOv8<br>• CRNN | Supports multiple plate styles. | Sensitive to occlusion and small plates. |
| 5 | **Agarwal et al.** (2023) | Layout-Independent CNN + Transformer OCR | Multi-row plate recognition. | • CNN<br>• Transformer | Handles complex plate layouts. | High computational complexity. |
| 6 | **Agarwal Direct** (2024) | Image Enhancement + CRNN | Recognition of blurred and tilted plates. | • Enhancement Filters<br>• CRNN | Motion blur and tilt correction. | Additional preprocessing overhead. |
| 7 | **Plavac et al.** (2024) | Commercial ALPR Evaluation | Benchmarking ALPR systems. | • Commercial ALPR Platforms | Real-world benchmarking. | Reduced performance in harsh weather. |
| 8 | **YOLOv5s-LPRNet** (2025) | Lightweight End-to-End CNN | Edge-optimized LPR. | • YOLOv5s<br>• LPRNet | Fast edge inference. | Slightly lower accuracy. |
| 9 | **Video ALPR** (2025) | Object Tracking + Best-Frame OCR | Motion blur reduction in video streams. | • YOLO<br>• Frame Selection Algorithms | Moving vehicle recognition. | Requires continuous video feed. |

A comparative study of research studies reviewed in literature is given in Table 2.3. It provides a brief overview of the main features of each study, including methodology, data sources, performance measures, and limitations, as indicated by the authors. This comparison will help to identify the shortcomings and strengths of the existing vehicle recognition and access control techniques. As shown in the table, most of the previous research has been targeted on either LPR or vehicle classification, and few work has been done on combining these two technologies into a single verification system. It also shows that there are challenges like limited real-time performance, reliance on cloud processing, limited datasets, and a lack of security measures. From these observations, the identified research gap is the development of the proposed system that is a hybrid system of dual verification where dual verification is done using automated vehicle make-model classification and license plate recognition coupled with edge computing for providing accurate, secure, efficient and real-time automated vehicle access control..

# CHAPTER 03

# 3. Modeling and Simulation

The chapter introduces the modeling, designing and simulation of the proposed Intelligent Vehicle Gate Recognition System. To validate the proposed architecture and assess it with real-world vehicle access scenarios, a prototype implementation was developed. The automated gate access decision is based on the combination of License Plate Recognition (LPR), Vehicle Make and Model Recognition (VMM), Weather-aware image enhancement, and authorization systems.

## 3.1 Conceptual Framework

The idea is to introduce a multi-stage computer vision and artificial intelligence system for automated vehicle verification. The process starts with the acquisition of the image and then followed by the weather-aware enhancement, license plate localization, character recognition, vehicle classification, and authorization verification. Each of the processing stages is a standalone module and plays a role in a common decision making mechanism. The modular design adds to maintainability, scalability, and adjustability to different weather, including fog, rain, and low-light.

## 3.2 High Level Design

### 3.2.1 Block Diagram

The general system architecture is edge-to-cloud based, and is deployed on Orange Pi 5 Pro with cloud monitoring on Vercel. The three trained models (YOLOv8n for license plate detection, YOLOv8n for character and digit recognition (38 classes) and EfficientNet-B4 for vehicle make and model recognition (245 classes) as well as the registered vehicle database (vehicles.json) are loaded at startup. The imagery of the vehicles captured is fed to the weatheraware pre-processing module that processes the image for fog removal (Dark Channel Prior), rain removal (vertical median filtering + inpainting) or night/low-light enhancement (CLAHE + gamma), depending on scene conditions. The improved image is then handed over to the AI inference step, which involves three models: EfficientNet-B4 for classifying the entire image of the vehicle and identifying the predicted make and model, YOLOv8n for the localization of the License Plate area and YOLOv8n for detecting individual characters on the cropped license plate and arranging them from left to right in the format of a Pakistani license plate (e.g., ABC-1234). The text on the plate and vehicle identity is detected and sent to the authorization engine for a dual-modality comparison to the database (allow, deny or temporary access rules) and return of a final allow/deny decision.

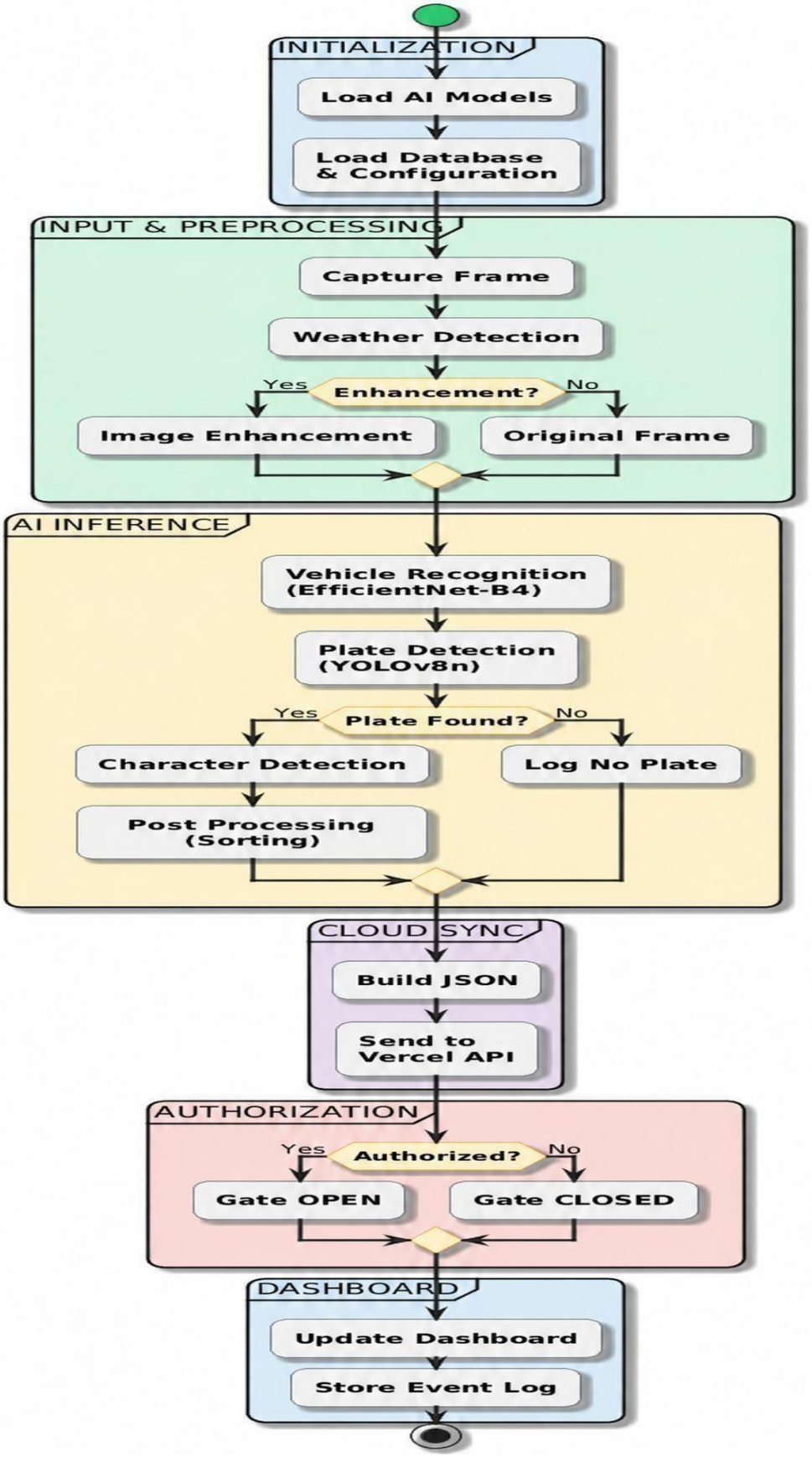



**Figure 3.2.1: High-Level Block Diagram**

## 3.2.2 Flow Diagram

When the image of a vehicle is obtained from the camera, the system operation is started. Firstly, the license plate is detected using the YOLOv8 detection model. Detected part is cut out and enhanced for better visuals. The extracted information of the plate is then sent to the YOLO character and digit detection model, and from the complete vehicle image, the vehicle make and model is detected using the EfficientNet-B4 model. The information obtained is matched with the authorised information in the database. Based on the verification results, the system will either make an authorized or unauthorized access decision, and it is instantly reflected on the monitoring dashboard..

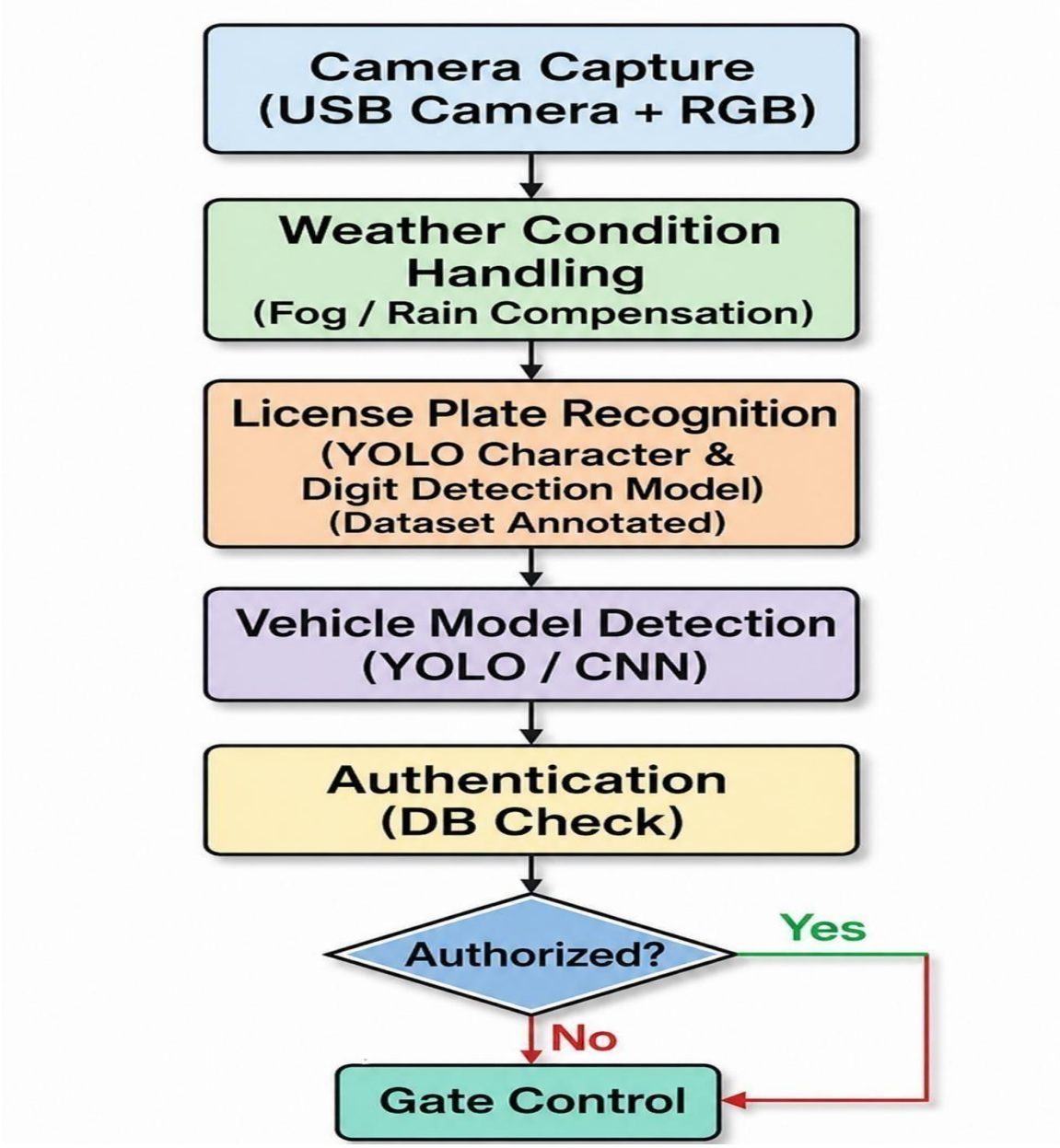



**Figure 3.2.2: Flowchart of the Proposed System**

The flowchart shows the entire process of the system, from image acquisition to the final decision for access control. It also features a dual-verification system which involves license plate recognition and vehicle recognition, this is for security purposes.

# 3.2.3 Performance

The system is designed to provide reliable recognition performance and yet be fast enough to be used for gate automation. The desired goals are roughly 85% detection rate for license plates, 90% detection rate for vehicle classes and less than 3 seconds processing time on standard computing hardware. Performance evaluation is based on the detection accuracy, recognition consistency, effectiveness of the pre-processing, reliability of the classification and system response time.

# 3.2.2 Flow Diagram

In order to be easy to implement and maintain, the complete solution is broken down into several functional subsystems. Each of the subsystems implements a particular operation in the vehicle recognition and access control chain.

# 3.3.1 List of Subsystems

**Image Acquisition Subsystem**: This subsystem is responsible for acquiring the images from the vehicles through a camera (USB 1080p) mounted on the Orange Pi 5 Pro. It can be used to monitor gates in real-time or to process images stored on a computer or video frames extracted from a video.

**Weather-Aware Preprocessing Subsystem**: Environmental factors like clear weather, fog, rain and nighttime scenes are analysed by the preprocessing subsystems. Depending on the condition detected, adaptive enhancement techniques are used to enhance image quality prior to AI inference. These include Dark Channel Prior for fog removal, filtering for rain suppression and CLAHE, gamma correction for low light.

**License Plate Detection Subsystem:** License Plate Localization is done using YOLOv8n model trained specifically for license plate detection. The model detects plates in the images of the vehicles and returns a list of bound boxes along with confidence scores.

**Character and Digit Recognition Subsystem:** A second YOLOv8n model trained on 38 alphanumeric classes (digits and uppercase English letters) is used in this module. The characters are then sorted based on their spatial position, to obtain the entire sequence of characters for the license plate.

**Vehicle Classification Subsystem:** A fine-tuned EfficientNet-B4 network is used for the vehicle make and model recognition. The model is designed to classify vehicle type from the whole image of the vehicle and also to provide the confidence score for the classification results.

**Edge Deployment Subsystem:** All AI models and the pre-processing operations run in the Orange Pi 5 Pro. This edge-computing approach removes reliance on cloud inference and allows for quick decision making with processing times typically less than the specified system requirement.

**Cloud Synchronization Subsystem:** Recognition results are securely sent to the cloud server as a structured data object. The information uploaded is plate text, vehicle identity, confidence values, environmental conditions, timestamps, and annotated images.

**Authorization and Gate Control Subsystem:** The authorization module checks the information of vehicles detected with the information stored in the database. Access control decisions are made based on pre-defined allow, deny and temporary entry rules. The subsystems decide whether or not to keep the gate shut or allow the vehicles in.

**Web Dashboard and Logging Subsystem**: Web based monitoring interface is created to monitor all the activities pertaining to the vehicle recognition in a centralized manner in real-time. The interface shows live recognition events as they happen, and administrators can immediately see recognitions and the details, including the make/name of the detected vehicle, the registration number, the time of the recognition and the confidence score. This real-time visualization allows quick monitoring and immediate response to any suspicious or unauthorised vehicle activities.

Apart from live monitoring, the system also offers a complete vehicle registration management module, which enables the administrator to register, update, delete and manage authorized vehicles with an easy to use and intuitive interface. This helps to ease the management of such a database and maintain accurate and current vehicle information.

The interface also offers an event history tracking tool whereby all past vehicle events detected are stored in an organized database. Users can search, filter and view previous records based on various criteria such as Date, Time, Vehicle Registration Number or Recognition Status. This will be very helpful for investigations, reporting and performance evaluation.

## 3.3.2 Flowchart of Subsystems

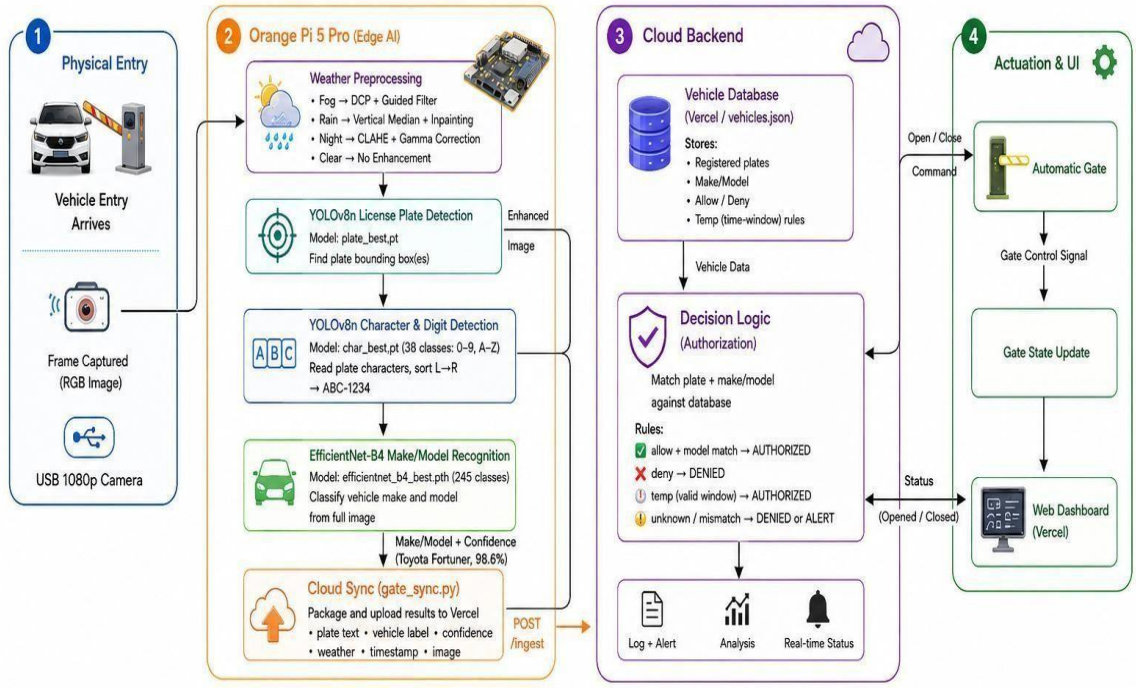



**Figure 3.3.1: Subsystem Diagram**

The subsystem diagram depicts the relationship between the key functional components and the flow of information through the recognition, verification and monitoring processes. When the high level design has been completed, the next step is to select the components and to complete the low level design. Each of the selected software and hardware components for system implementation is described in this section. The selection of the components was based on their performance, compatibility, availability, deployment needs, and cost efficiency.

# 3.4.1 Image Acquisition — USB CAMERA is an application that can capture images from a USB camera.

The main image acquisition device is a Full HD USB camera. It is installed at the entry point of the gate and can take pictures of vehicles under various lighting conditions and then send them to the processing unit for analysis. Software-based image enhancement techniques, not dedicated illumination hardware, are used to enhance night time visibility.

## 3.4.2 This is a tutorial for training a YOLOv8 detection model to detect license plates in images.

YOLOv8 detection framework is chosen, since it can complete high-speed object localization and high detection accuracy. It is a single stage system or device that can be operated efficiently in real-time vehicle access applications.

## 3.4.3 Pre-processing of images — OpenCV Image Processing Library

The image enhancement functionality needed to increase the quality of the images prior to recognition is offered by OpenCV. Preprocessing operations involved are contrast enhancement, low-light correction, fog removal and rain reduction depending on the weather condition detected.

## YOLO — Character and Digit Detection: The image below shows the output of running YOLO on the test image.

Character recognition is done with a special YOLOv8 model, which is trained to recognize the characters separately. Detection-based method makes it more tolerant of skewed text, font variations, and partially obscured characters.

## 3.4.5 Vehicle Classification — EfficientNet-B4

In this study, EfficientNet-B4 was chosen as it provides a good compromise between classification performance and cost. The experimental evaluation showed that recognition performance was high, and that there was no problem with the inference speed for use on the edge.

## 3.4.6 Authorization — JSON-based Supabase

A structured database holds authorized vehicle information and can be accessed via the web dashboard. Records contain license plate data, temporarily users and validity periods, and vehicle identity and access permissions. Recognized vehicle information is compared to stored information to produce access decisions.

## 3.4.7 Orange Pi 5 is a Embedded Processing device.

The Orange Pi 5 Pro acts as the main edge computing device. It performs locally weather-aware pre-processing, license plate detection, character recognition and vehicle classification. It is capable of real-time image processing and has a secure communication to send results to the cloud platform.

## 3.4.8 Web-Based Dashboard for User Interface

The dashboard offers an intuitive platform that allows for easy visualization of the system's activity and control of registered vehicles. The operators can view the recognition result in real time, get historical events, make manual overrides and set access permissions in one place.

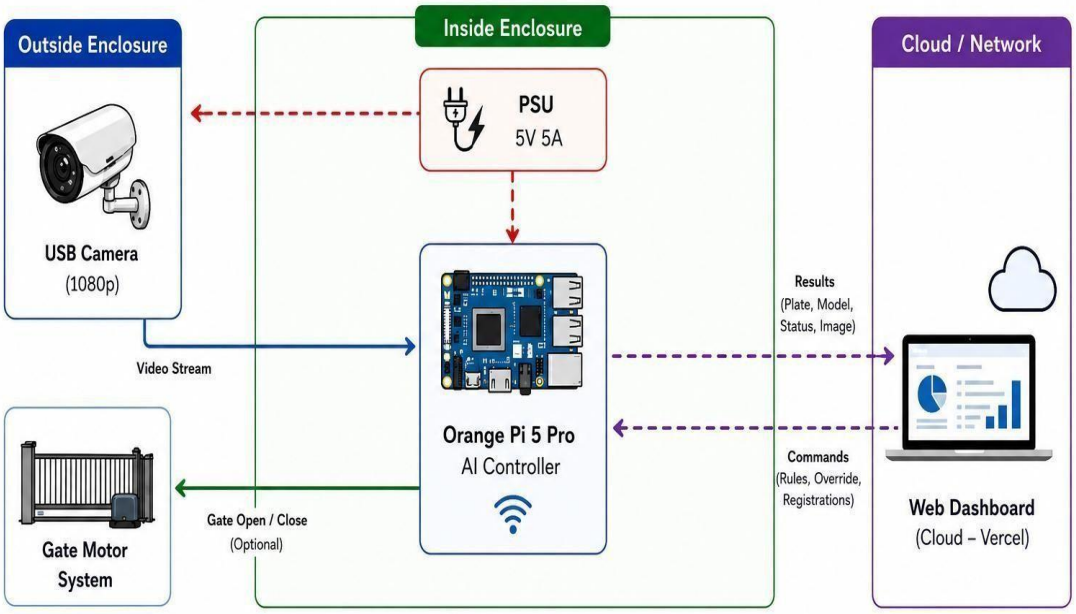



**Figure 3.4 — Prototype Application Diagram**

# 3.5 System Architecture and Module Interaction

The overall system architecture provides a high-level view of the interactions between all the hardware and software elements in the automated vehicle verification and gate control system. It specifies the relation between camera, edge computing device, recognition model, database, web application and gate controlling mechanism. The architecture diagram visualizes the data flow from image capturing of the vehicles to recognition, verification and decision of access to the gate. It also identifies the relationships and communication between different modules and between edge devices and cloud services. This structured design provides for efficient data processing, reliable communication, real-time decision-making, system scalability, and seamless integration of all components, leading to secure and accurate vehicle access control...

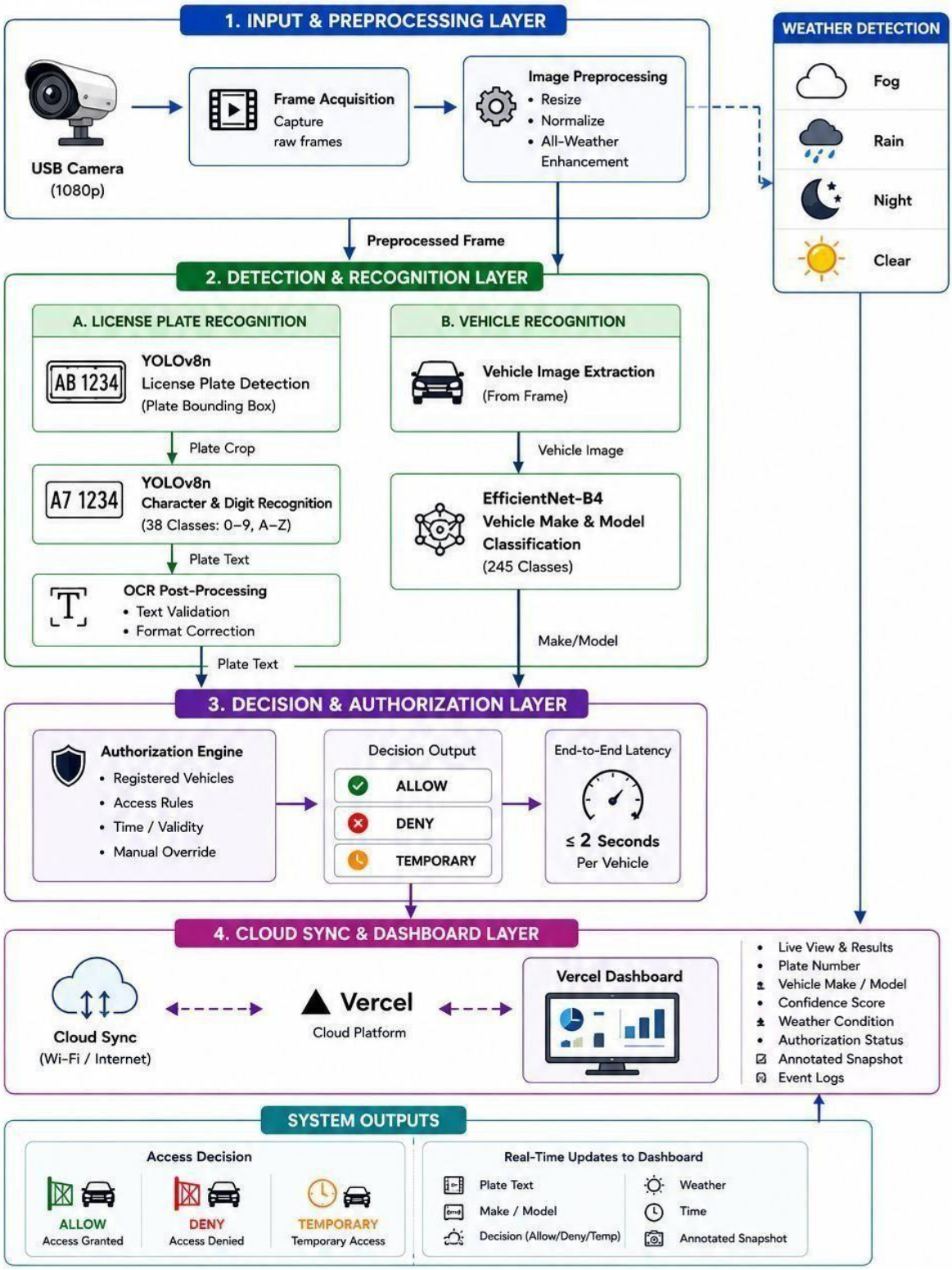



**Figure 3.5 — System Architecture Diagram**

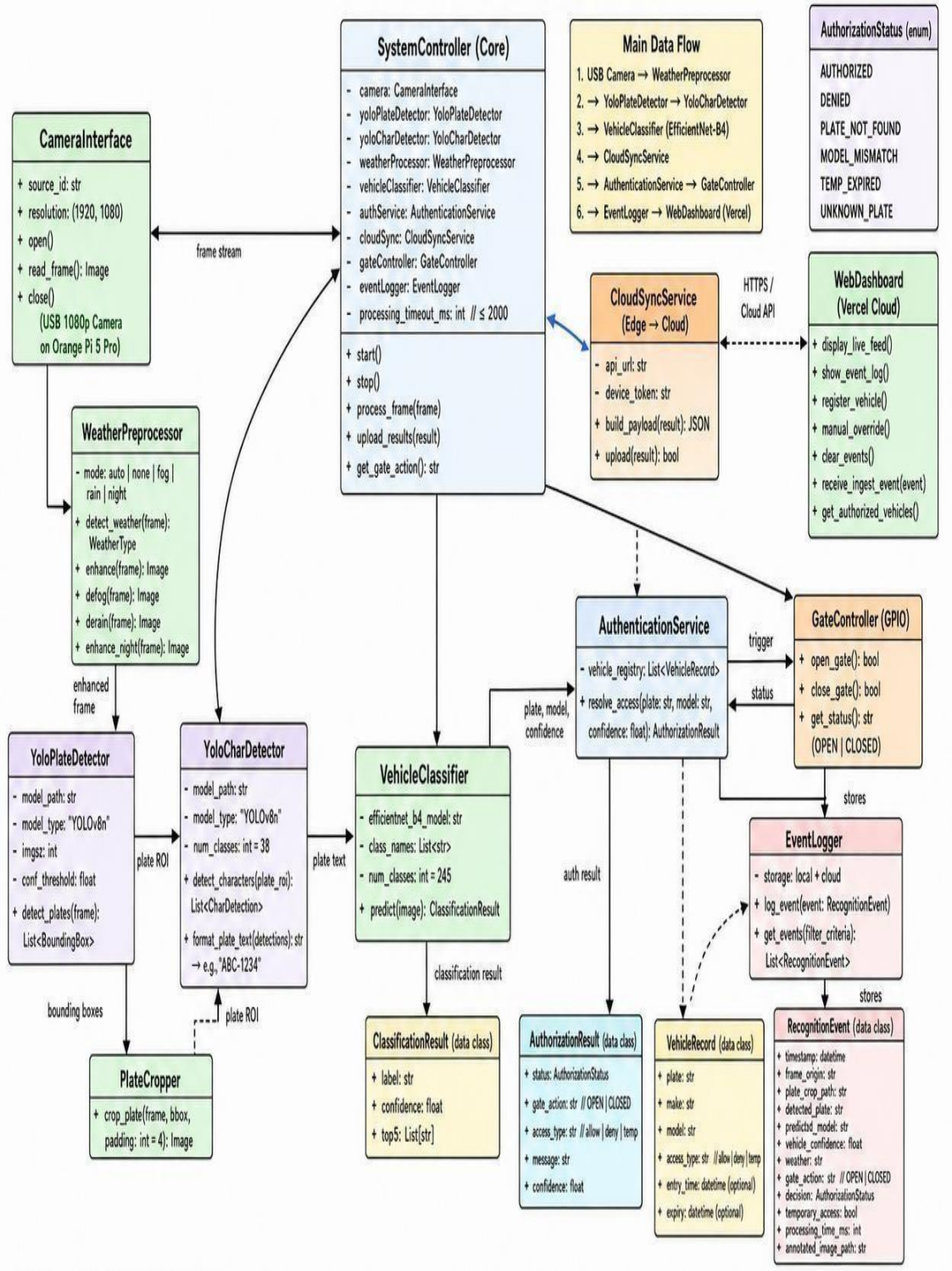



**Figure 3.5.1 — Class Diagram**

The class diagram represents the software structure of the proposed system. It illustrates the relationships among major software entities responsible for image acquisition, preprocessing, recognition, authorization, and event management.

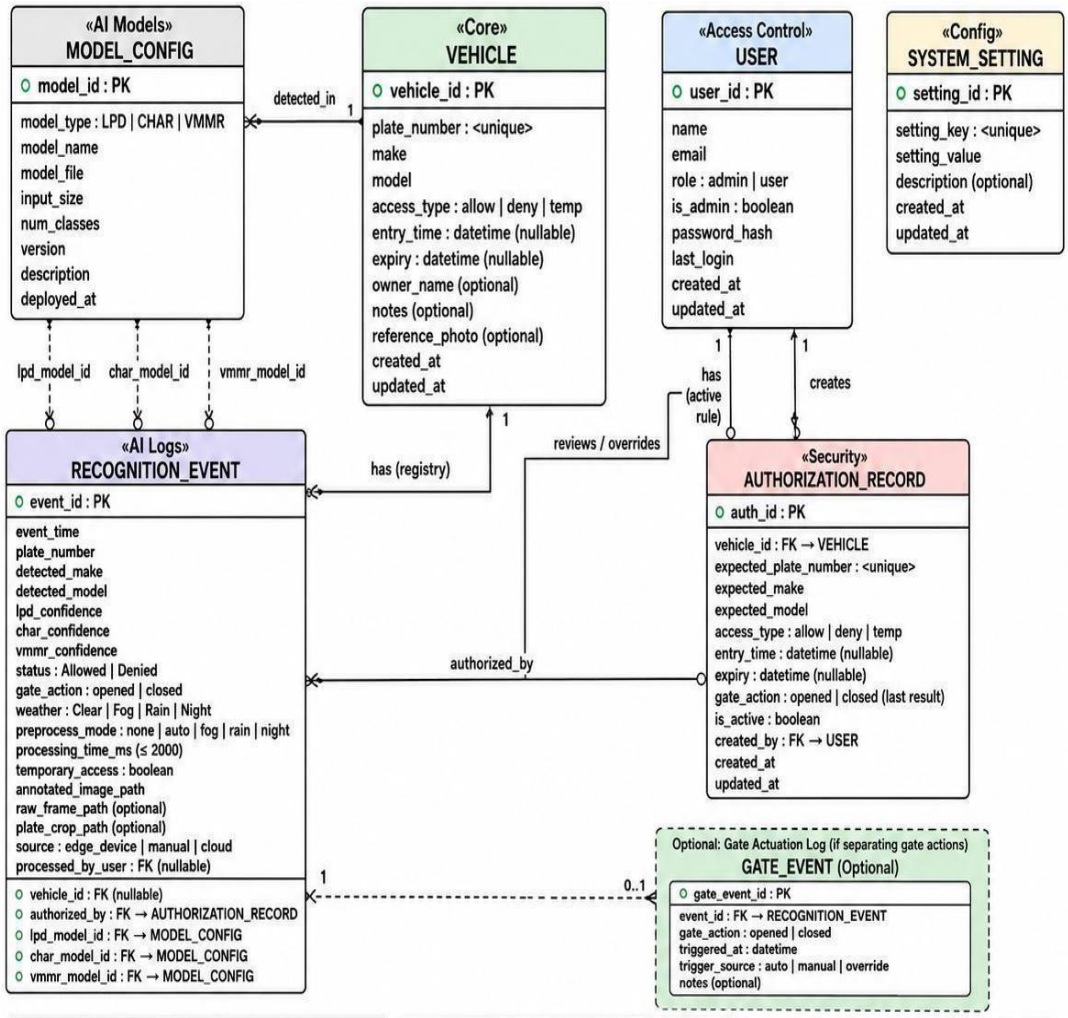



**Figure 3.5.2 — ER Diagram**

This is the Entity Relationship (ER) diagram that shows the logical structure of a database for the proposed intelligent vehicle recognition and access control system. It establishes the relationships among the entities of the main database, such as users, registered vehicles, recognition events, and authorization records, allowing for efficient organization and management of the data. Each user is allowed to register and manage several cars and each detected car will create a recognition event with things like the license plate number, the car make and model, the confidence score, the timestamp and the access decision. Users and vehicles both have authorization records that identify whether or not a particular user is authorized to move a particular vehicle into the secured area. This relational database design reduces data duplication, ensures data consistency and integrity, and allows for rapid retrieval of data records in real-time verification process. Moreover, since well-defined relationships are established among all entities, the ER diagram will enable secure event logging, historical record management, auditing and reporting. This relational database architecture guarantees system performance, scalability, and efficient management of vehicle access operations..

# CHAPTER 04

# 4. Implementation and Testing

. chapter Demonstrates the deployment of the proposed all-weather smart vehicle gate recognizing system. It's a characterization of the development, integration and deployment of individual software components into a fully operating pipeline. The chapter also highlights testing methods, performance analysis, and experimental results to evaluate the accuracy, efficiency, and reliability of the system under various environmental and operating conditions.

## 4.1 Software Implementation

## The YOLOv8 model is used to detect license plates.YOLOv8 is used to detect license plates.

The localization of vehicle license plates is done using the Ultralytics YOLOv8 object detection framework. A vehicle dataset specifically for license plates has been used to fine-tune a YOLOv8-Nano (YOLOv8n) model. The images are scaled to 640 × 640 pixels and then input into the network in the inference stage. The detector can approximate the position of licence plate BBs, and provide a confidence score. In order to avoid duplicate predictions by YOLOv8, Non-Maximum Suppression (NMS) is used to ensure that only the most reliable prediction is kept. The acquired plate area is cut and handed over to the subsequent stages of the pre-processing and character recognition.

## 4.1.2 Adaptive Image Preprocessing

Before character recognition the detected license plate regions are subjected to an adaptive image enhancement process to increase the image quality and thus helps in recognition. The preprocessing pipeline consists of several image processing operations such as noise reduction, Contrast Limited Adaptive Histogram Equalization (CLAHE), global histogram equalization and image sharpening. Such techniques enhance contrast and edge clarity, as well as minimize the effect of the rain streaks, low light, motion blur, reflection and haze in the atmosphere. The preprocessing stage will facilitate the recognition stage in the downstream with the betterment of character boundary and suppression of unwanted characters. The entire enhancement process with the traditional CPU hardware currently takes 100-200 milliseconds per image of the plates.

## 4.1.3 The problem statement is related to License Plate Text Recognition (LPTE).

A special YOLOv8n model for character and digit detection is used to extract license plate text. The suggested method is not based on the traditional Optical Character Recognition (OCR)

procedures but each alphanumeric symbol is one target to be detected. The class label, confidence score and bounding box location is given for each character detected. This network is trained to identify 38 characters (0-9 and A-Z) and thus there is no need to use any external OCR engine to recognize the characters of Pakistani license plate.

Known symbols once detected are arranged in ascending order by their x-coordinate. A deterministic post processing module then arranges letters and digits in the anticipated format of a Pakistani license-plate to create formatted output (e.g., ABC-1234). Unlike character segmentation and template matching, the detection of the object itself makes it robust in case of partially occluded, skewed, dirty and challenging environmental conditions. On the validation set, the precision, recall, and mAP@0.5 are 98.99%, 96.88%, and 98.07%, respectively; this is a sufficient level of character recognition accuracy for deployment on the Orange Pi edge platform.

# 4.1.4 The classification of the make and model of the vehicle.

The EfficientNet-B4 Convolutional Neural Network (CNN) is used to identify vehicles. A custom made fine grained dataset containing 245 categories of vehicles (both Pakistani market vehicles and Japanese Domestic Market (JDM) vehicles) was used for training the classifier. This was done by transferring the weights learned from ImageNet and changing the last layer to correspond to the classes of the project - vehicles.

The used model was EfficientNet-B4, the selected model is efficient to give a trade off with recognition accuracy and computation cost for an embedded model. The effectiveness of the model was also evaluated on a held out test set of 17,254 images, with a Top-1 accuracy of 96.24%, Top-5 accuracy of 99.39% and a macro F1-score of 96.18%. The result found in this paper confirms the model for the classification of similar visually classes of vehicles, and also confirms the dualverification access control system where the image of the vehicle is matched with the recognized license plate before access decision.

# 4.1.5 Web-Based Dashboard for User Interface

The module which determines whether or not a vehicle can pass through the gate is called an authorization module. The decision is made with reference to a database of registered license plate/vehicle model combinations. Access will only be granted if the found plate number and calculated vehicle type and model are in the database and a valid entry. If the license plate is right, but the model of the vehicle is not a match with the one kept in the database, then it is suspected.

### 4.1.6 Visualization and Logging

To make the system more transparent and easy to monitor, visual annotations are added for each image processed. These overlays include license plate boundaries detected, license plate text recognized and confidence information along with the final authorization result. In addition a structured log entry is produced for every vehicle event. The log is used to store the time of the incident, the license plate number detected, the make and model of the vehicle predicted, the confidence levels, and the access decision made. These records offer a thorough audit trail that can be evaluated for operational monitoring, troubleshooting, and security investigations.

## 4.2 Testing Procedures

### 4.2.1 Dataset Description

- To evaluate the proposed smart vehicle gate recognition system, a set of data with different categories of vehicles, license plate formats, and under different environmental conditions were used. The evaluation was focused on the performance of the individual models and of the entire end-to-end pipeline in realistic deployments.
- The license plate detection module is trained and tested using the dataset of about 23,000 annotated vehicle images in YOLOv8 format. Data set consists of vehicles taken from different view angles and distances in different lighting conditions, and license plate is manually annotated with a bounding box. These are images of a variety of real-world outdoor settings that can be encountered at vehicle entry and exit points.
- For character and digit recognition, a special database of 1235 images of Pakistani license plate was used. Each image was annotated on both the character and digit level with the annotation format YOLOv8 allowing each letter and digit in the image to be a separate detection object. There are 38 classes (results are digits 0-9 and letters A-Z). The collected samples are diversified in font style, plate color, image quality and environmental factors to make the samples more robust.
- To test the vehicle make and model recognition a fine-grained vehicle dataset was created. The data set includes both market cars sold in Pakistan and the Japanese Domestic Market (JDM) cars which are available on the roads of Pakistan. The test set was a held out set of 17,254 images. The images have a lot of variation in background, occlusion, illumination, viewpoint, etc., and the color of the vehicles.
- In addition to the model specific ones, qualitative end-to-end testing was conducted on another set of images and video frames captured from the deployed Orange Pi camera setup. For example, during the day with plenty of light, at night, during the rain with running water, in foggy weather where there isn't a lot of visibility, and in real time shots from live gate shows. Testing scenarios included all the various type of license plates as per the usual practice in Pakistan, which are a maximum of three alphabetic characters followed by 4 numeric digits (as in ABC-1234).

# 4.2.2 Testing Methodology

Evaluation process was done by the following three major phases: Unit Testing, Integration Testing and Performance Testing. The modular design provided a good opportunity to test each module and the whole system.

**Unit Testing:** In addition to the model specific ones, qualitative end-to-end testing was conducted on another set of images and video frames captured from the deployed Orange Pi camera setup.

- Prior to the system integration, all the software components have been tested individually for correctness and performance.
- A reserved test data set was used to test the YOLOv8n license plate detection model. Precision, recall, and mean Average Precision (mAP@0.5) with a confidence threshold of 0.25 were used to evaluate the detection quality.
- The adaptive preprocessing module was used for the analysis of images in which bad weather (such as night lighting, rain and fog) caused image degradation. Original and enhanced images were compared, and the level of improvement in readability of the plates as well as visibility of the vehicles was determined.
- The results of character and digit detection model YOLOv8n were analyzed on cropped License Plate Regions. The individual character detection accuracy and the overall plate reconstruction accuracy following post processing was used to measure performance.
- Vehicle classification model EfficientNet-B4 was tested on the entire 17254image test set. Different metrics of performance were taken into account including Top-1 accuracy, Top-5 accuracy, macro precision, macro recall and macro F1-score.

**Integration Testing :** After a successful validation of individual modules, the whole recognition pipeline was tested as a single system.

- End-to-end testing was carried out with the images acquired under various weather and illumination conditions. The aim was to see that the pre-processing, plate detection, character recognition, vehicle classification and authorization components work properly.
- To validate the edge to cloud communication, the inference was tested on Orange Pi 5 Pro and then confirm if the recognition result, authorization decision, and annotated image are uploaded to the cloud based dashboard.

- Different vehicle records were tested with authorization function (allow, deny and temporary access). It was found that system decisions were compared to the expected outcomes to be able to determine if access-control behavior was correct.
- Real-time image processing and event synchronization was monitored to ensure the results of the recognition performed on the edge devices are reflected in the updates on the dashboard.

**Performance Testing :** In addition to the model specific ones, qualitative end-to-end testing was conducted on another set of images and video frames captured from the deployed Orange Pi camera setup.

# 4.3 Deploying and setting up the hardware.

Model development and initial validation was done on a desktop workstation, whereas the whole recognition system was ported to specialized edge devices for operation in the real world. The deployment architecture was created based on edge-AI principles with all the heavy computational inference workload being performed on-site and only light results being sent up to the cloud services. The Orange Pi 5 Pro is equipped with three AI models: YOLOv8 license plate detector, YOLOv8 character/digit recognition model, and EfficientNet-B4 vehicle make and model classifier. Only processed data is uploaded to the cloud by the edge device, including the recognized text on plates, predicted vehicle identity, confidence data, weather classification and authorization status, time stamps and compressed image snapshots. With this architecture, bandwidth usage is reduced, privacy is enhanced, and it enables the system to function even if the connection is unreliable.

Carries out the entire AI recognition process including license plate recognition, character recognition, vehicle classification and authorization processing.

- Makes all inferences locally – no cloud based AI computation required.

- Delivers enough processing power to allow real time gate monitoring and decision making.

- At average, processes about 1 image per second with CPU based inference.

- Primary controller during live deployment demos & field tests.

**USB Camera (1080p)**

The Full HD USB camera is used for image acquisition and is directly connected to the Orange.

- Takes pictures of cars and frames of video at the gate entry point.

- Computes real-time visual data for the recognition pipeline.

Works in Live Operation or Batch Processing.

Supports real-time vehicle in/out tracking.

**USB-C Power Supply (5V, 5A)**

High current power supply designed to operate at stable currents over long running interference workloads.

- Supplies power to the Orange Pi 5 Pro in a regulated manner.
- Allows running of multiple deep-learning models continuously.
- Helps prevent intermittent power outages caused by fluctuations and/or heavy power demands.
- Helps to ensure system reliability for lengthy deployments.
- Network and Cloud Link.
- Wireless networking that is implemented with Wi-Fi between edge device and cloud dashboard.
- All recognition events are sent secure over https connection.
- Only the compact results payloads are uploaded to the cloud.

- The raw image streams, trained models and inferences calculations are stored on the local device.
- After every recognition, Dashboard is automatically synced.

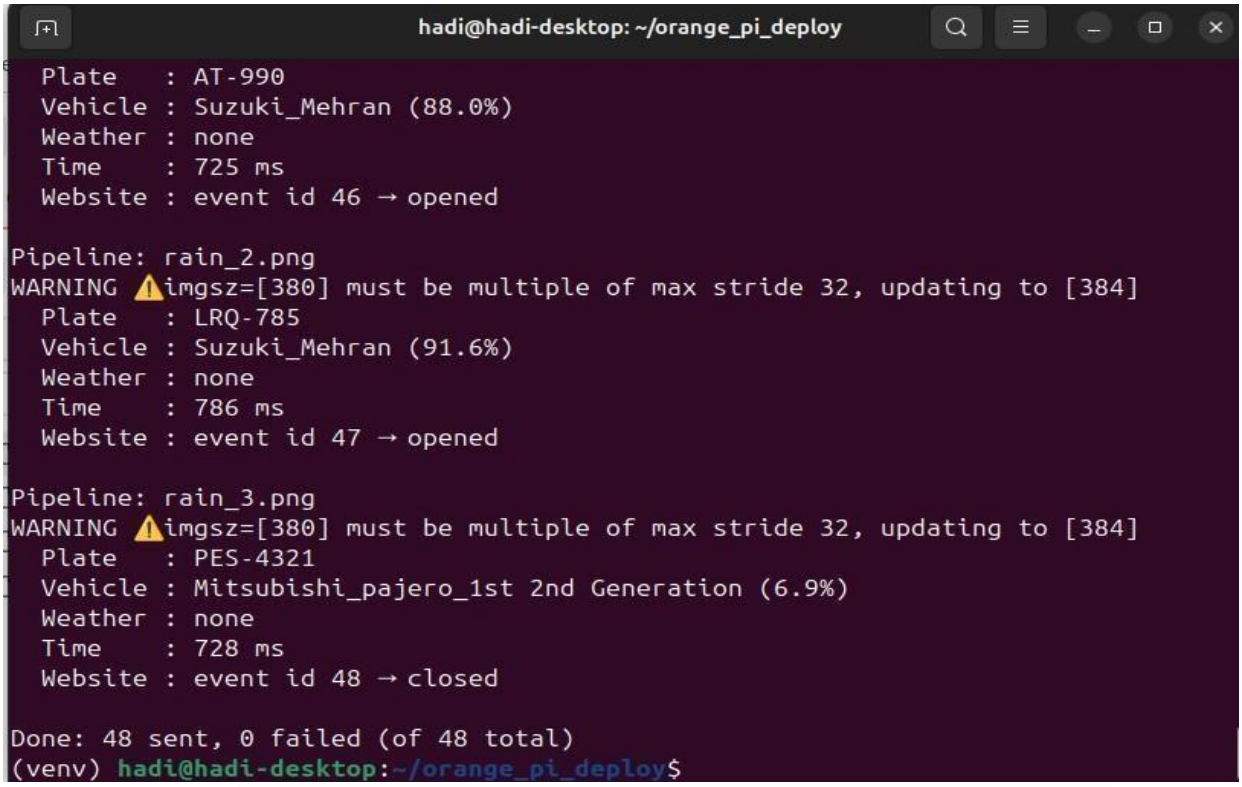



**Figure 4.3.1: Batch Image Processing and Authorization Log (48 Images, Orange Pi 5 Pro)**

This is a large-scale batch inference experiment on Orange Pi 5 Pro to test the realtime processing capacity and stability of the system. The total number of vehicles images processed during the experiment was 48. In every image, the system was able to correctly recognize the license plate, classify the vehicle make and model, provide a confidence score and verify the authorization by comparing the detected license plate with the registered vehicle database. Furthermore, the processing time for every image was also measured to assess its inference performance on the edge device. All the recognition results were automatically uploaded to the cloud dashboard for monitoring and storage after completing the analysis, such as license plate number, predicted vehicle class, confidence value, authorization status, timestamp and processing statistics. The experiment was successfully completed without any processing interruption, upload failure or synchronization problem, which shows the reliability, robustness, and scalability of the proposed edge based vehicle recognition and access control system for continuous operation.

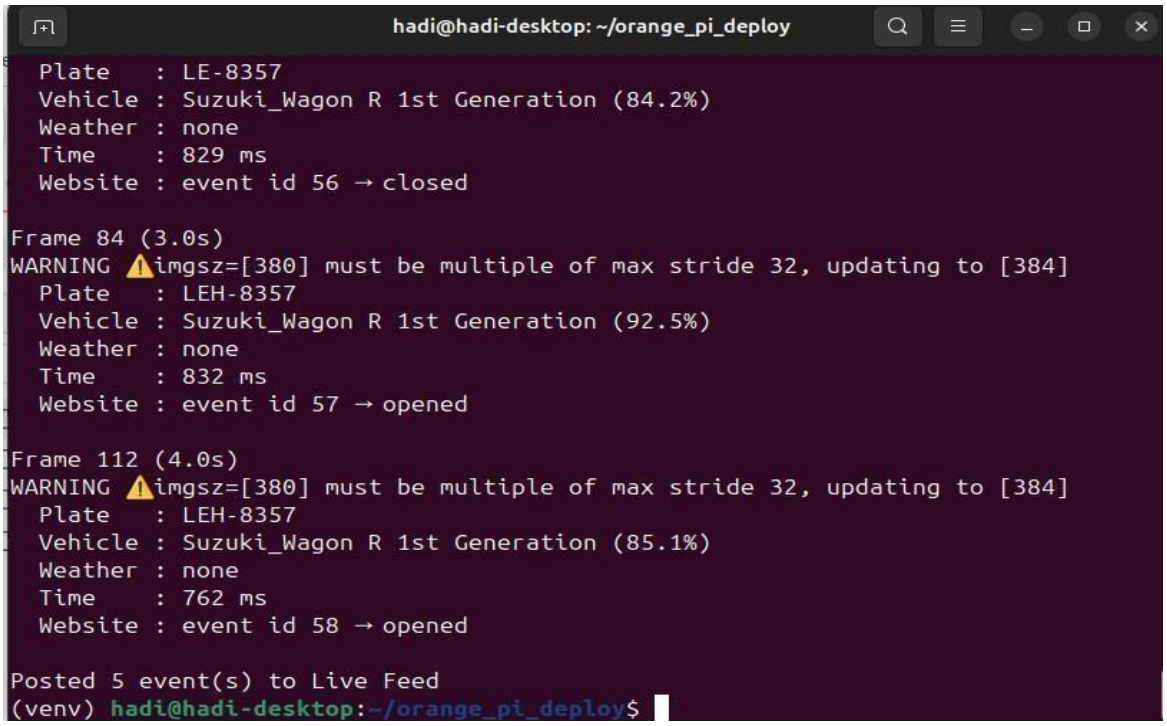



**Figure 4.3.2 — Real-Time Video Frame Inference and Dashboard Synchronization (Orange Pi 5 Pro)**

This figure demonstrates the real-time video-frame processing performed on the Orange Pi edge computing platform. Video frames captured by the surveillance camera are processed locally, where vehicle detection, license plate recognition, and vehicle classification are executed without relying on cloud-based inference. Once the recognition process is completed, the generated detection results are instantly transmitted to the cloud dashboard through the network for real-time visualization and storage. This experiment confirms the system's ability to maintain seamless synchronization between edge-based processing and remote monitoring services. The results demonstrate efficient data transmission, low-latency communication, and reliable real-time performance, ensuring continuous monitoring and accurate vehicle access management.

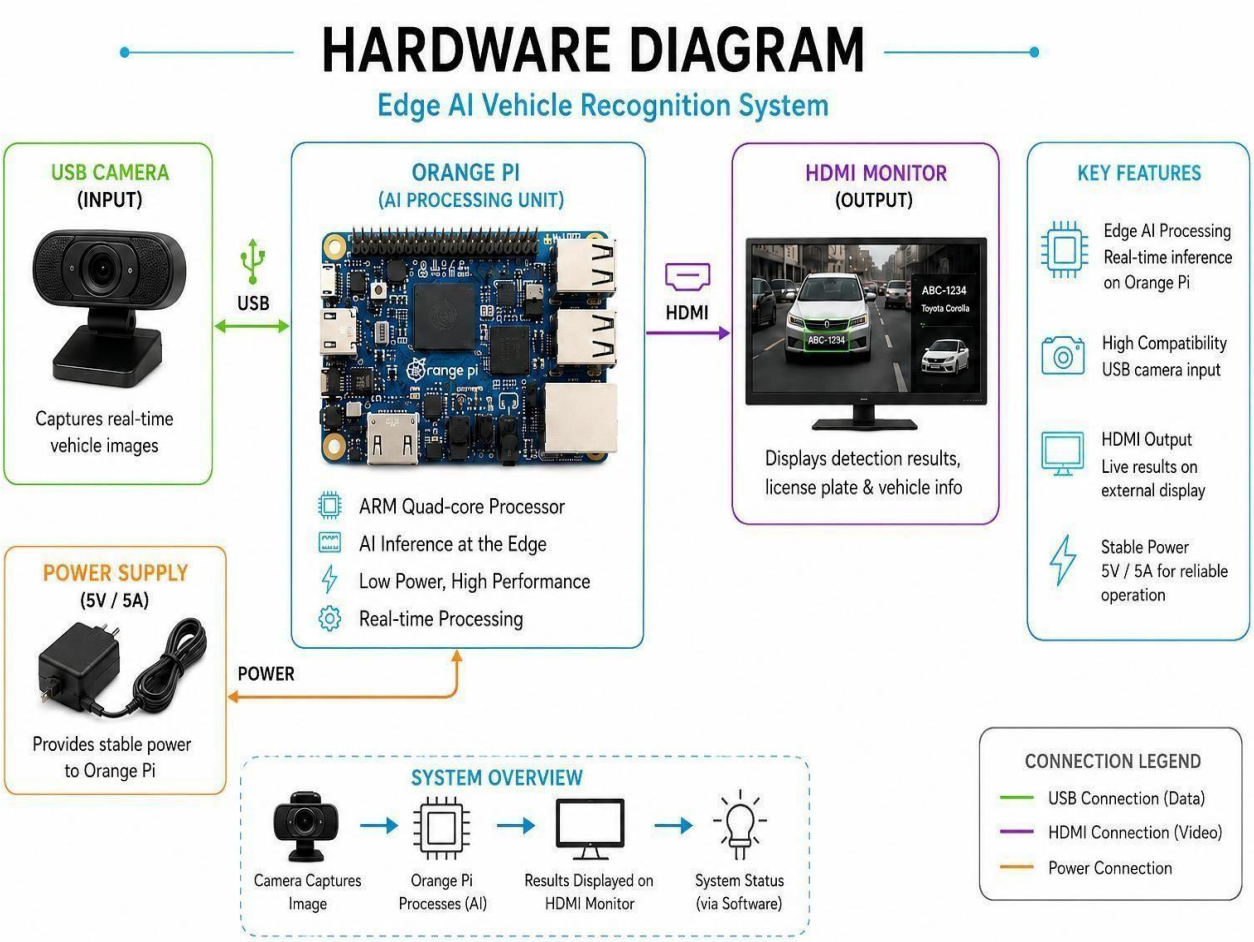



**Figure 4.3.3: Hardware Diagram**

The overall hardware architecture of the proposed Vehicle recognition and Gate control system is shown in this figure. It features a surveillance camera, an Orange Pi processing unit, network infrastructure, a cloud-based dashboard and an automated gate-control mechanism. Images of the vehicles are captured by the camera and sent to the Orange Pi for real time processing and recognition. The processed result data is sent via the network to the cloud based dashboard for monitoring, data storage and administration. Depending on the recognition result, the gate-control unit is given commands to open or close the gate so that the vehicle can enter the site efficiently, in security and in a fully automated manner.




**Figure 4.3.4: Orange Pi Live Pipeline Demonstration (Plate LEB-1481)**

Figure 4.3.4 illustrates a live deployment scenario in which the system successfully recognized license plate LEB-1481 and completed full inference in approximately 846 milliseconds. Following authorization evaluation, the resulting denied-access event was uploaded to the cloud dashboard in real time, demonstrating the responsiveness of the deployed solution.

# 4.3.2 Pipeline Time:

| Processing Stage | Execution Time |
|---|---|
| Plate Detection | 80–200 ms |
| Digits Character Detection | 30–100 ms |
| Vehicle Classification | 200–800 ms |
| Total Pipeline | 700–1000 ms |

**Table 4.1 — Edge AI Pipeline Latency Breakdown (Orange Pi CPU Inference)**

Table 4.1 Displays the processing time of the key steps of the edge-AI pipeline. Experimental measurement shows that the detection of license plate requires about

80-200ms to recognize the word, 30-100ms to recognize the character,

It takes 200-800 milliseconds to classify a vehicle. The whole pipeline (including overhead for preprocessing, database verification and cloud synchronization) offers around 700-1000 milliseconds endtoend latency per frame. These results meet with the project requirement of a total processing time of less than 2 seconds and still achieve good recognition accuracy.

# 4.4 Web Dashboard Implementation

The Web Dashboard is an all-weather smart vehicle gate cloud-based monitoring, administration and access-control application for proposed all-weather smart vehicle gate system. It has been developed using current web technology stack consisting of both on the front end and back end.

Node.js/Express backend. Vercel is where the application is deployed and it's connected to the Orange Pi edge device via a wireless network connection. The results of the edge pipeline (license plate text, vehicle make and model predictions, confidence score, weather data, authorization decisions, time stamps, and compressed annotated image snapshots) are fed in real-time into the dashboard where they can be visualized and managed.

The frontend app offers a Live Feed interface, which continually displays the incoming recognition events. Each of these events includes details of the license plate detected (image, type, etc.), the class of vehicle predicted (car, truck, motorcycle, etc.), confidence values, environmental sensing (weather, etc.) and access decision. The dashboard also offers details on gate status, showing operators if a barrier is open or closed, following barrier authorization evaluation. Administrators can use this interface to see the system in action and take action on any suspicious or unusual events.

A separate administration module for vehicle registration and access management. The system database can be populated with authorised vehicles, make and model details. Access permissions can be limited to visitor temporary access, permanent deny access or permanent access. Entries can be made for a specific period of time and only let visitors into the facility for certain time periods. This gives flexibility and at the same time a strict system of access control.

The dashboard also has an event history that stores all the vehicle detections that have been processed and is searchable for auditing and historical analysis purposes. Every entry contains the results of the recognition, the results of the attempt to authorize, the time of the recognition and additional information from the inference process. Administrative users can view these to see what has happened in the past, to investigate security matters, and to audit system activity. In addition, there is a manual override setting, allowing a person to assume full control of the robot in cases where unusual situations arise which cannot be dealt with using automated decision making.

A lightweight vehicle registry, based on JSON data format, is used to manage recognition data, which is stored as permanent access-control records. Detection events are kept in runtime to update user interface as quickly as possible. Communication between the frontend and backend components is done via RESTful API with JSON Web Token (JWT) authentication. This architecture guarantees that just legitimate users are able to access administrative functions as well as sensitive car data.

Cloud services integration with edge computing resources is an important aspect of the system design. The Orange Pi 5 Pro is used to locally compute all the heavy tasks of AI inference models, like license plate detection, character recognition, vehicle classification and authorization evaluation. Only processed outputs go to the cloud dashboard, conserving bandwidth and respecting user privacy. With this architecture, efficient monitoring and centralized management are achieved with site deployment real-time performance.

The dashboard can also be used for actual gate-control application. The user interface will show gate-open and gate-closed events from authorization decisions made by the recognition pipeline. These decisions can also be tied to Orange Pi GPIO relay outputs, to directly actuate the gate-control hardware if physical barrier automation is required. As such, the dashboard can be used not only to monitor access but also as a part of the access-control solution.

The front end web application has been developed using React 18, TypeScript, Vite and Tailwind CSS, the back end is based on a node and express rest API.

The app is hosted on Vercel, with a single page application (SPA) served on the front end, and API endpoints on the back end that handle vehicle registration, event processing, user authentication and access-control operations.

Vehicle registration records are stored in an in-memory data structure for low latency updates and a JSON database file (vehicles.json) will contain the records while the system is running. Role management is carried out with JWT and authentication and authorisation is implemented. There are two types of user roles: administrator and regular user. The admin has more privileges (vehicle registration, user management, manual override, etc.) while normal users have less privileges (operational monitoring, visitors).

For future scalability, the software architecture also features a Supabase PostgreSQL Database schema that can deliver persistent cloud storage. The prototype deployed, however, is simple and easy to deploy APIs based on express and file storage using json. The gate_sync.py utility runs after the inference is completed, and uploads the recognition events to the ingestion endpoint on the cloud platform to synchronize the Orange Pi and the cloud platform. This mechanism will allow for the information available on the dashboard to be consistent with the activities on edge devices.

**Key features:**

**1 Dashboard** — Dashboard is the main dashboard and provides a real-time view of the day to day activities of the gates. Vehicle detection, number of entries granted, number of entries denied, gate status and environmental conditions are displayed. While Dashboard statistics are automatically updating at regular intervals for up to date operational awareness.

**2. Live Feed** — Live Feed module displays a stream of recognition events received from Orange Pi edge device which is continuously updated. They contain the information of the detected vehicle license plate number, the predicted model and make of the vehicle, the confidence score, the weather label, the authorization result and the time. Filtering can be used to show certain types of events—such as authorized vehicles, denied vehicles, or temporary visitor entries. Of course, there can also be an optional camera feed for added situational awareness.

**3. Vehicle History** — The Vehicle History module keeps a searchable and paginated history of gate events. Previous records can be retrieved by the operators for auditing, investigation and performance analysis. Search capability enables events to be quickly looked up by license plate, type of vehicle, time, or result of authorization.

**Manual Override** — Manual Override allows the administrator to take manual control of the system if there are exceptional events. Registration of new vehicles, access rights, upload of reference pictures and override of automatic gate decisions can be made by authorised persons if necessary.

**5. Add Temporary Visitor** — Short term access authorization for visitors, contractors and service personnel is provided by the Temporary Visitor module. Administrators can have records of visitors, as well as different times/dates that the temporary permissions expire, so they are only current during authorized access hours.

**6.Weather** — Environmental conditions are shown in the Dashboard to give contextual information on recognition events. Parameters such as temperature, humidity and visibility provide insight to the operators as to how the environment affects system performance and the preprocessing to be used.

**7. Admin Access Privileges** — Secure access to the administration of the platform with the help of role based access management. Administrative users can increase the privilege level of trusted operators, change their privileges or revoke them without compromising the overall security of the system.

**8. Settings** — The Settings module includes a number of parameters such as notifications, display, camera integration and so on. These controls allow administrators to set how the dashboard will act if needed in a deployment. All these modules form a complete management platform with real-time monitoring and historical auditing, visitor administration, manual intervention and secure gate-control. The dashboard will be used together with the Orange Pi edge inference pipeline, to provide a comprehensive end-to-end solution for intelligent vehicle access management..

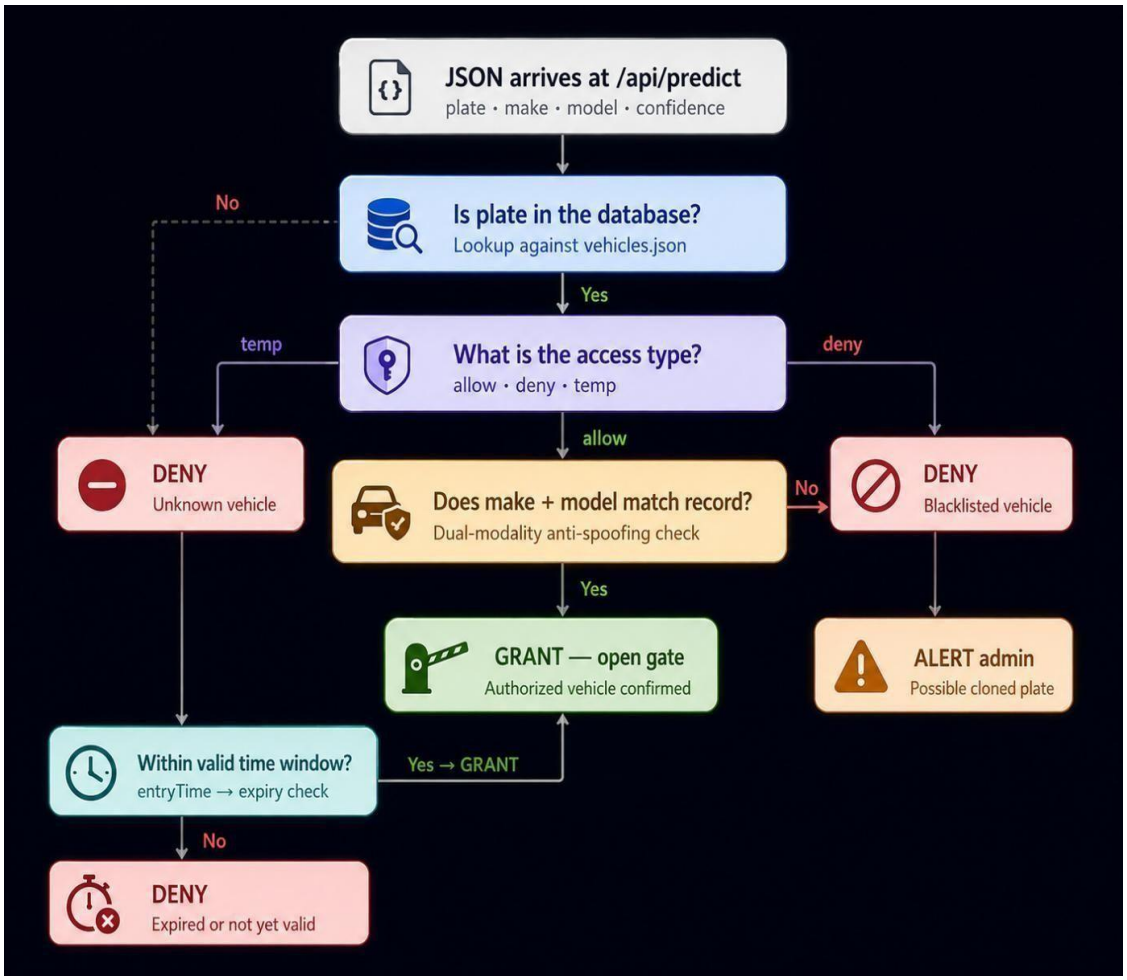



**Figure 4.4: Dual-Modality Access Control Flowchart**

When an inference result that includes the plate number, make and model of the vehicle ocr and the corresponding confidence scores are received on the backend API endpoint, the decision making process shown in figure 4.4 is performed. A system check of the vehicle registry database for a matching plate record is performed first. It then assesses the access policy given, which could be granting or denying access to a visitor permanently or temporarily. Validity periods are checked prior to additional processing for temporary entries. Finally, the vehicle identity detected is matched with the make and model information registered. Only when all the verification conditions are met, access is granted, otherwise access is denied.

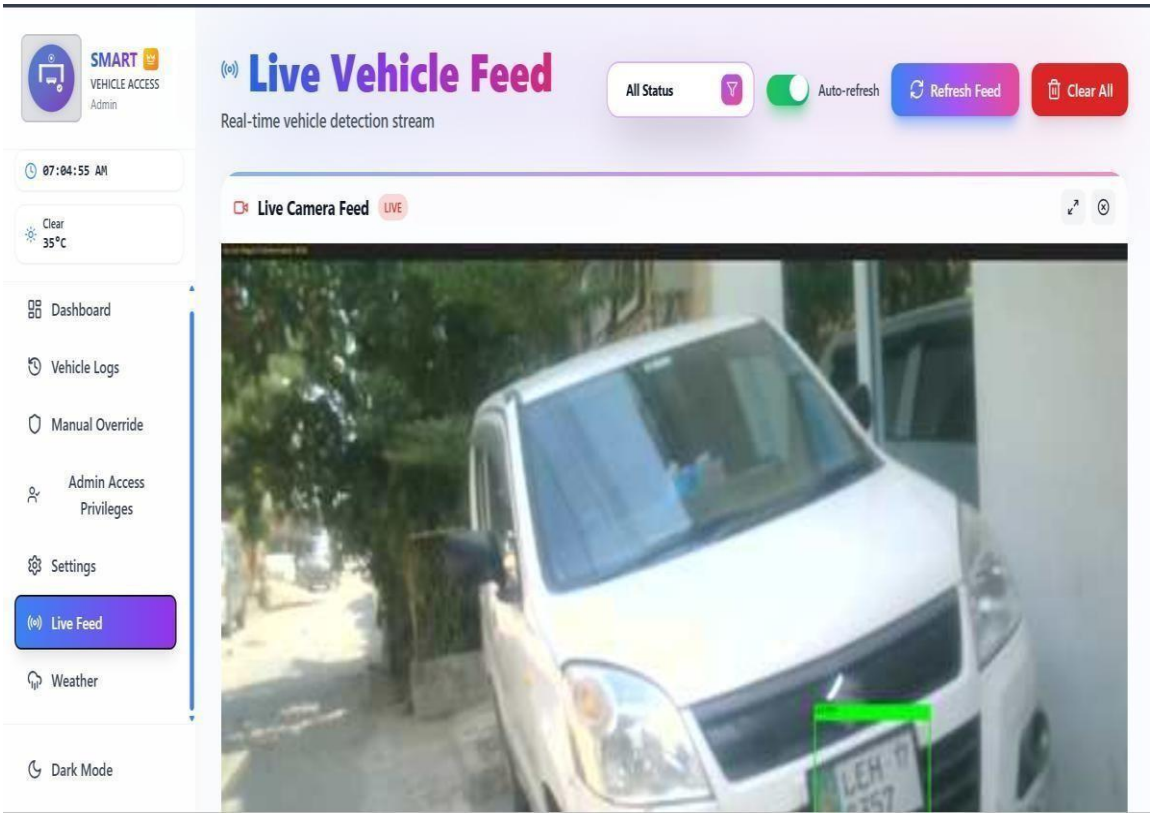



**Figure 4.4.1 — Web Application Frontend Admin Panel**

One sample vehicle detection event is shown in the proposed intelligent vehicle access control system via the Live Feed interface as shown in Figure 4.4.3. In this instance, the Orange Pi processing unit managed to capture the license plate of the vehicle, LEH-8357 and, at 85.1% confidence, determine the make and model of that vehicle to be Suzuki Wagon R. Once the recognition process was done, the identified license plate was automatically matched with the registered vehicle database to ascertain whether the vehicle was authorised to come inside the premises. A matching record was found, and the system verified the vehicle as authorized and automatically sent a gate-open command – no manual intervention required. The dashboard immediately provided all necessary information such as the plate number, the vehicle make and model, the confidence level, authorization or not, the time and the access decision. This real-time visualization enables operators to track vehicle movements in real time, verify accuracy of vehicle recognition, and storage of an accurate event history

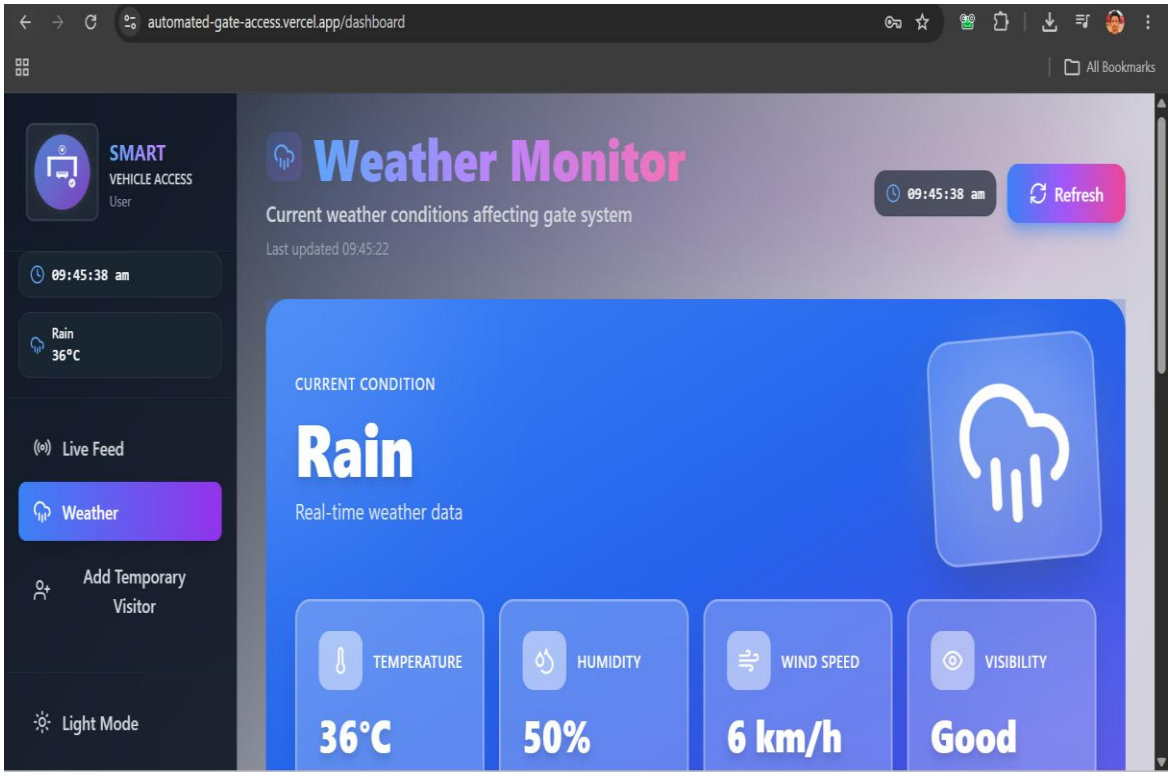



**Figure 4.4.2 — Web Application Frontend User Panel**

The standard-user dashboard appears as shown in Figure 4.4.2, and is designed to offer key monitoring capabilities and limit access to advanced administrative functions. The user interface is different from the administrator dashboard, it will be a simplified navigation structure that will be easier to use and will focus on day to day operations. Every standard user is able to view the vehicle recognition events in real-time and access decisions made by the system, as well as the vehicle verification status. The dashboard also allows users to register temporary visitors by entering the necessary vehicle and visitor details and giving them access to the vehicle while not changing the vehicle database. This role-based design guarantees that users are able to carry out day-to-day activities effectively while maintaining the system's protection and integrity. Normal users still cannot access administrative functionalities like user management, system configuration, database maintenance, and model settings. In summary, the standard-user dashboard offers a safe, easy-to-navigate and efficient means of tracking vehicle activity.

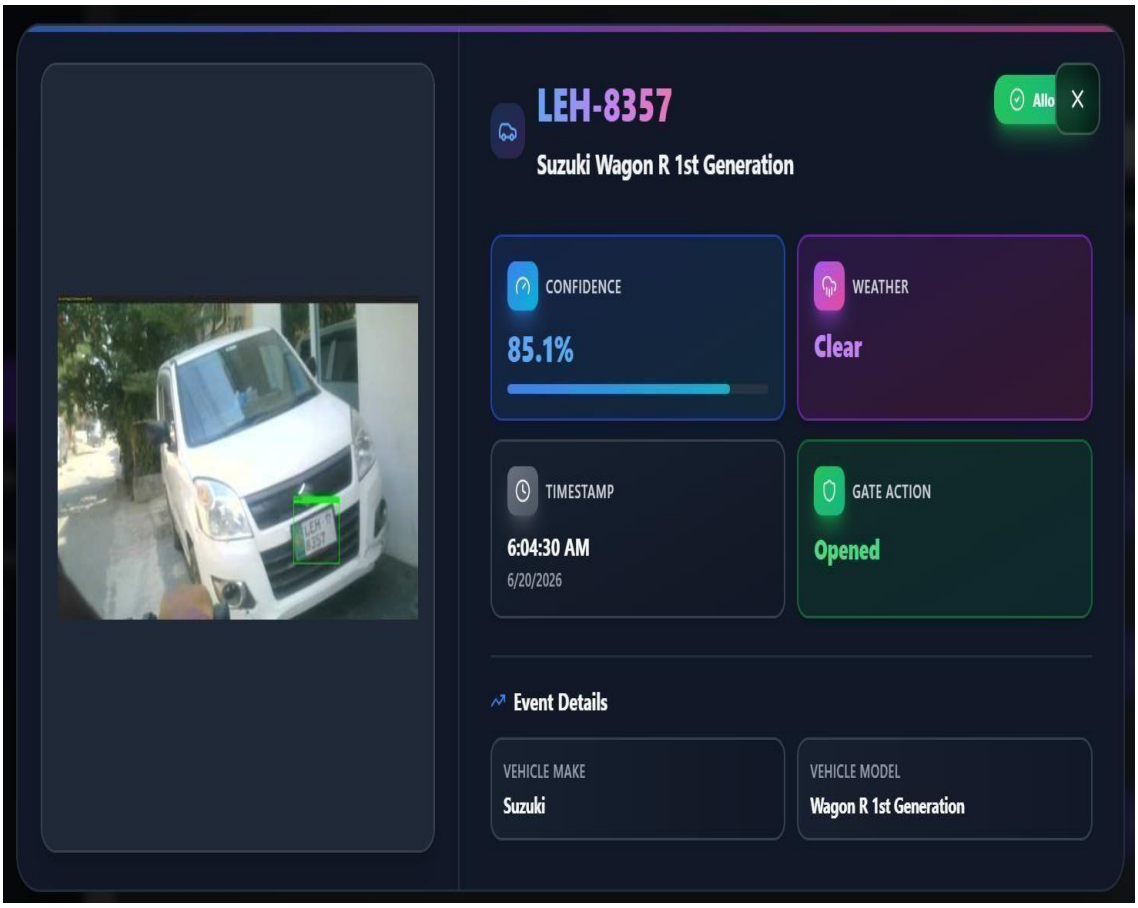



**Figure 4.4.3 — Web Dashboard Detection Event Card (Plate LEH-8357, Suzuki Wagon R — Allowed)**

Figure 4.4.3 shows an example of a vehicle detection event as shown by the Live Feed interface of the proposed system. In the latter case, the Orange Pi processing unit could successfully read and identify the car's license plate, LEH-8357, and classify it as a Suzuki Wagon R with 85.1% confidence. Once the recognition process is done, the system automatically checks detected license plate for its authorized status in the vehicle register database. The vehicle had been recorded in the database and verification was marked as correct; the system also sent a gate-open command, thus letting the vehicle in without manually switching the gate. The relevant data, such as detected licence plate, predicted vehicle make and model, confidence level, authorised/not authorised status and time were immediately displayed on the dashboard. The real-time event visualization helps operators keep track of system activity, check the recognition results, and keep a precise record of vehicle access for security, auditing and future analysis...

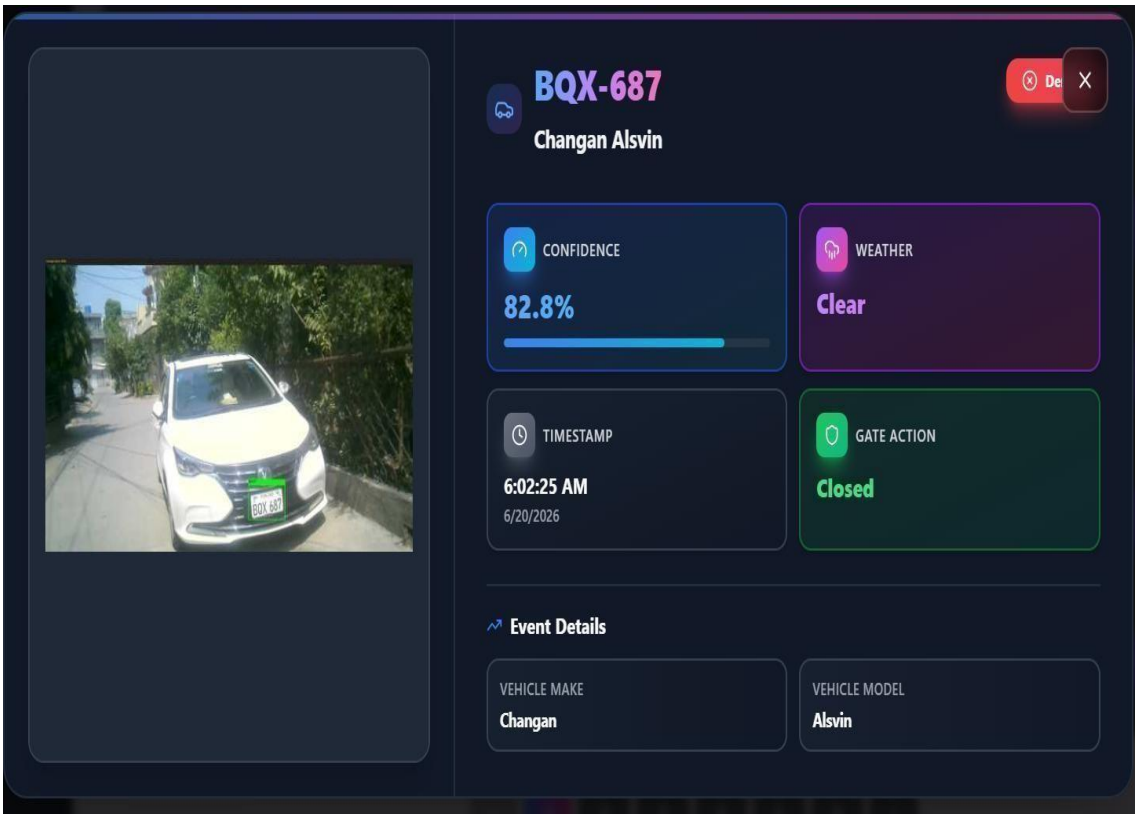



**Figure 4.4.4 — Web Dashboard Detection Event Card (Plate BQX-687, Changan Alsvin — Denied)**

The denied-access event shown in Figure 4.4.4 is created by the system. The Orange Pi was able to successfully identify the license plate BQX-687, while identifying the vehicle as a Changan Alsvin with 82.8% confidence. The vehicle was not allowed to pass the accesscontrol requirements and an accesscontrol decision 'gate-closed' was issued after the authorization evaluation. This event was then recorded and added to the real-time monitoring stream on the dashboard.

# CHAPTER 05

# 5. Results and Discussion

In this chapter, an extensive analysis of the proposed all-weather smart vehicle gate recognition system is done. This assessment is to conduct a performance analysis of the individual License plate detection model, Character recognition model and Vehicle make-and-model recognition (VMMR) model. Standard quantitative measures, based on training logs, validation results and held out test data are used for each component. The full system path is evaluated in conjunction with stand-alone testing and applied to real-world environmental changes like rain, fog and low visibility conditions to test the robustness of the system to be deployed in real world situations..

## 5.1 License Plate Detection Results

The first step of the proposed system is to locate the license plates of vehicles in the input images. A YOLOv8-Nano (YOLOv8n) object detection model, which has been trained on a locally created dataset of Pakistani cars and bikes, is used in this task. The nano version of YOLOv8 was chosen because of its lightweight architecture that allows for efficient training and deployment on edge devices while maintaining high detection accuracy.

The detector is a very important preprocessing step, because the subsequent recognition stages are all based on correct plate regions. Hence the reliability of this module is directly dependent on its accuracy and stability.

### 5.1.1 Dataset:

| Item | Detail |
|---|---|
| **Source** | Roboflow — *License Plate Project (Version 1)* |
| **License** | `CC BY 4.0` |
| **Format** | **YOLOv8** (images + normalized bounding box labels) |
| **Task** | Object detection |
| **Number of classes** | 1 (`plate`) |
| **Annotation type** | Axis-aligned bounding box per license plate |

**Table 5.1 — Yolo Licene Plate Dataset Detail**

Table 5.1 shows the composition of the set of data utilized for training and evaluation. It contains images of vehicles captured under different lighting, camera angles, occlusions and background complexity scenarios. Supervised learning is provided by manually annotating each license plate using YOLO format bounding boxes.

| Split | Images | Allocation (%) | Purpose |
|---|---|---|---|
| **Training** | 20,580 | 87.5% | Model weight learning |
| **Validation** | 1,973 | 8.4% | Hyperparameter tuning and epoch selection |
| **Test** | 978 | 4.1% | Held-out evaluation and inference demonstration |
| **Total** | **23,531** | **100.0%** | *Complete Dataset Pool* |

**Table 5.2 — Yolo Licene Plate Dataset Size**

Table 5.2 shows the sampling distribution between training, validation and testing sets. The partitioning of the data set has been designed around the following principles to make the experiment sound and unbiased, and to avoid any data leakage between training and test sets

### 5.1.2 Training Confguration

| Parameter | Value |
|---|---|
| Model Architecture | YOLOv8n (Nano) |
| Task | Object Detection |
| Number of Classes | 1 (License Plate) |
| Input Image Size | 380 × 380 pixels |
| Batch Size | 32 |
| Total Epochs | 50 |
| Warmup Epochs | 3 |
| Training Platform | Google Colab (Tesla T4 GPU) |

**Table 5.3 — License Plate Detection: Training Configuration**

To train the YOLOv8n detector, the hyperparameters and training configuration are summarized in Table 5.3. This involves input resolution, batch size, optimizer type, learning rate schedule, number of epochs, and augmentation strategies.

To boost the convergence, the model was trained for 50 epochs with the transfer learning strategy, which allows for the use of the feature representations trained in previous epochs. The training and validation losses were recorded during training to ensure the stable learning behavior and prevent overfitting.

### 5.1.3 Detection Performance

**Table 5.4 — License Plate Detection: Performance Metrics**

| Metric | Value |
|---|---|
| Precision | 98.50% |
| Recall | 95.53% |
| mAP@50 | 98.07% |
| mAP@50-95 | 70.73% |
| Min Box Loss (Train) | 1.032 |
| Min Cls Loss (Train) | 0.384 |
| Min Box Loss (Val) | 1.067 |
| Min Cls Loss (Val) | 0.384 |

The results of the evaluation confirm that the detector YOLOv8n is working properly for the test set which it has not been trained on. 98.50% of accuracy means that the model gives very few false alarms, which means that the majority of the areas predicted are in agreement with the actual licensed plates. The high recall value of 95.53% means that the model can detect most plates in the input images while the rest are not detected. This is particularly important in the context of gate automation, where failure to detect gate issues or delays in access to the gate could happen. Moreover, the localization accuracy based on the standard IoU threshold of 50% is very high which can be seen from the obtained mAP@50 score of 98.07%. These results demonstrate a good precision and stability and the detector is suitable to be used for real-time application..

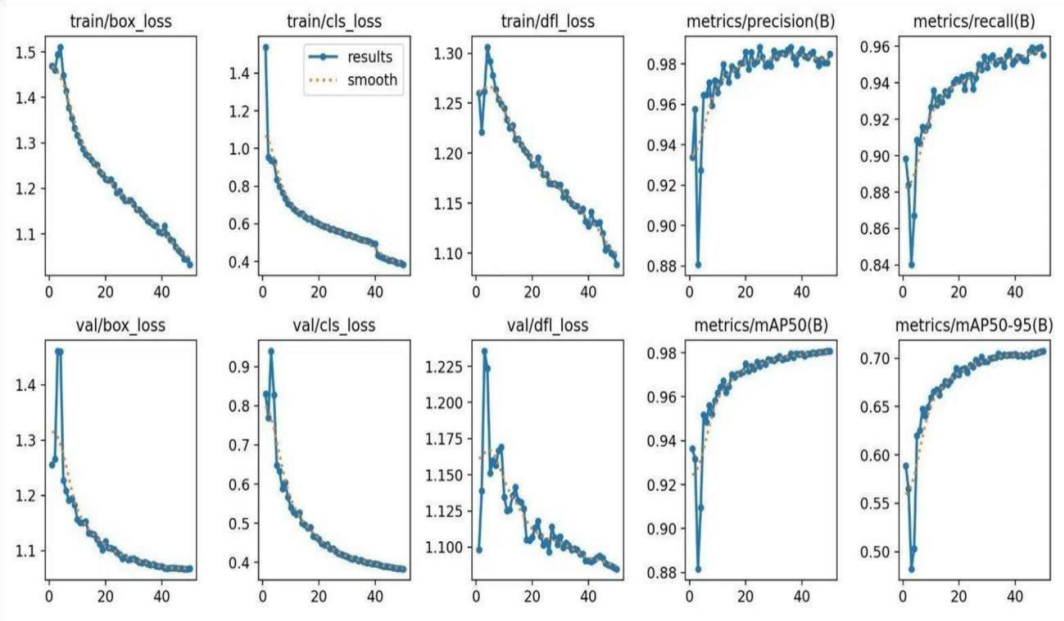



**Figure 5.1 — YOLOv8 License Plate Detection: Training and Validation Metrics Over 50 Epochs**

The training and validation metrics throughout the entire training period are shown in Figure 5.1. As the training proceeds, loss values are consistently decreasing, signifying good optimization. Aside from any training/validation curve differences, the model seems to perform well and generalizes to unseen data. The metrics of performance like precision, recall and mAP begin to increase with each epoch and then level off, indicating that the model is not overfitting.

| Epoch | Box Loss | Cls Loss | Val Precision (%) | Val Recall (%) | mAP@0.5 (%) | mAP@0.5:0.95 |
|---|---|---|---|---|---|---|
| 1 | 1.469 | 1.539 | 93.40 | 89.80 | 93.60 | 58.90 |
| 5 | 1.448 | 0.836 | 96.40 | 90.90 | 95.20 | 62.00 |
| 10 | 1.316 | 0.698 | 96.60 | 92.70 | 96.20 | 66.00 |
| 15 | 1.263 | 0.638 | 97.90 | 93.50 | 97.00 | 67.60 |
| 20 | 1.220 | 0.601 | 98.60 | 94.10 | 97.50 | 68.30 |
| 25 | 1.194 | 0.573 | 98.80 | 93.70 | 97.40 | 69.10 |
| 30 | 1.165 | 0.542 | 98.60 | 94.80 | 97.70 | 69.70 |
| 35 | 1.129 | 0.518 | 98.70 | 95.20 | 97.80 | 70.20 |
| 40 | 1.103 | 0.495 | 98.70 | 95.20 | 97.90 | 70.40 |
| 45 | 1.072 | 0.408 | 98.20 | 95.70 | 98.00 | 70.20 |
| **50** | **1.032** | **0.384** | **98.50** | **95.53** | **98.07** | **70.73** |

**Table 5.4.1 — License Plate Detection: Training Progression (YOLOv8n, 50 Epochs)** The sample training statistics are shown in table 5.4.1 for the 50 epochs. The trend of decreasing losses and increasing validation scores shows stable learning behavior along the training process. However, the results also validate that transfer learning made a substantial contribution towards quicker and stable convergence.

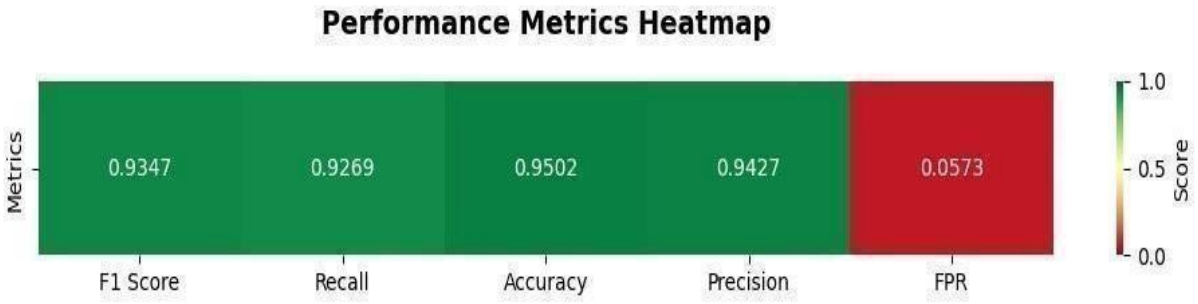



**Figure 5.2 — YOLOv8 License Plate Detection: Performance Metrics Heatmap (F1, Recall, Accuracy, Precision, FPR)**

The heatmap in Figure 5.2 visualizes key evaluation metrics including F1-score (0.9347), recall (0.9269), accuracy (0.9502), precision (0.9427), and false positive rate (0.0573).

All first four values are very high, which means that the detection capability is balanced. The low false positive rate indicates that the model rarely classifies background regions as license plates, an important factor in minimizing the amount of processing that needs to be done in an automated gate system.

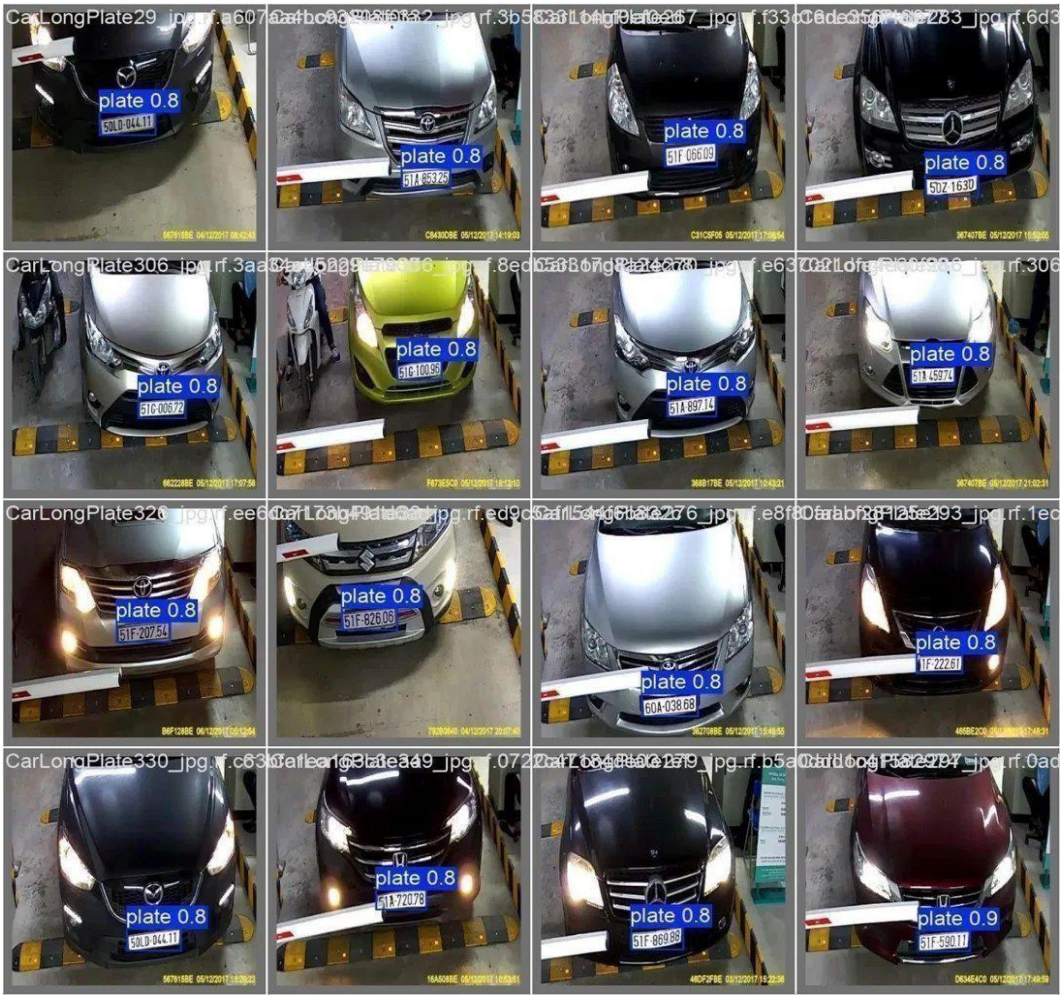


**Figure 5.3 — YOLOv8 License Plate Detection: Test Images**

The figure 5.3 shows a few sample result images of the trained detector on unseen images. The model has been able to successfully recognize license plates in a range of scenarios such as

different lighting conditions, angle of view, and vehicle types. These results validate that the detector is capable of continuing to deliver strong performance in real-world situations that are not ideal.

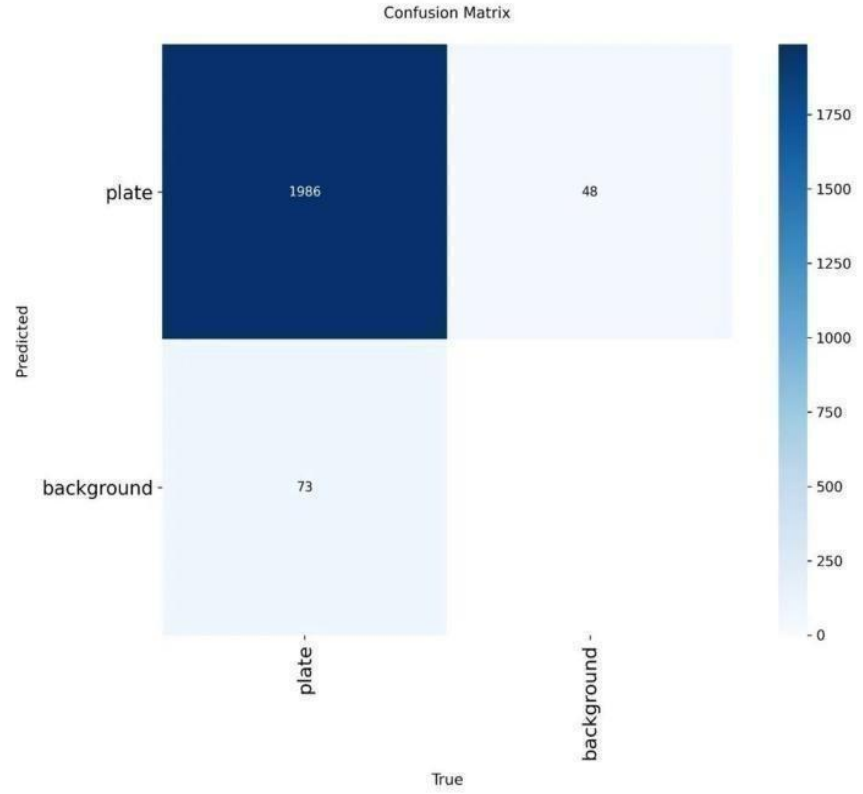



**Figure 5.4 — YOLOv8 License Plate Detection: Confusion Matrix**

Figure 5.18 presents the top five vehicle class predictions generated by the proposed vehicle recognition model for a representative test image. Each predicted vehicle class is associated with a confidence probability that reflects the model's certainty regarding the classification. The class with the highest confidence score is selected as the final recognition result because it is considered the most probable match for the input image. The remaining four predictions are ranked in descending order of confidence, providing alternative vehicle class possibilities identified by the model. Displaying the top five predictions offers a comprehensive understanding of the model's classification behavior and confidence distribution. It also demonstrates the model's capability to differentiate between visually similar vehicle categories while maintaining reliable performance. Furthermore, this representation helps assess the robustness and effectiveness of the recognition system by illustrating how it prioritizes competing predictions, especially in complex situations where different vehicle models exhibit similar shapes, structures, colors, or other visual features.




**Figure 5.5: YOLOv8n License Plate Detection: Sample Inference Results**

The samples include a range of real world situations such as parked cars, moving cars, forward plate, rear plate, and different car types (Sedan, SUV, Van). There are also several plate formats that are included in the dataset in different regions and low light conditions as a form of robustness testing. The clearly visible plates have a confidence score within the range of 0.72 to 0.83, indicating consistent detection performance across various conditions. Overall, the results indicated that the YOLOv8n network is suitable for deployment in a real-time vehicle recognition system with a gate.

# 5.2 Character Recognition Results

The character recognition module recognizes each alphanumeric character in the cropped license plate region. This system uses an object detection model that has been trained to identify each character and digit as its own box - the YOLOv8n model.

## 5.2.1 Dataset

| Property / Metadata | Detail |
|---|---|
| **Source** | Roboflow — *My First Project (Version 4)* |
| **License** | `CC BY 4.0` |
| **Format** | **YOLOv8** (images + per-character bounding box labels) |
| **Task** | Multi-class object detection (character-level) |
| **Number of Classes** | 38 |
| **Annotation Type** | Axis-aligned bounding box per character/digit |
| **Dataset Split Configuration** | |
| **Split Type** | Training |
| **Images** | 1,235 |
| **Character Instances** | 7,883 |
| **Notes** | Primary learning set |

**Table 5.5—License Plate Detection: Performance Metrics**

| # | Class | Category | Description |
|---|---|---|---|
| 0 | 0 | Digit | Numeric digit '0' (0–9 range) |
| 1 | 1 | Digit | Numeric digit '1' (0–9 range) |
| 2 | 2 | Digit | Numeric digit '2' (0–9 range) |
| 3 | 3 | Digit | Numeric digit '3' (0–9 range) |
| 4 | 4 | Digit | Numeric digit '4' (0–9 range) |
| 5 | 5 | Digit | Numeric digit '5' (0–9 range) |
| 6 | 6 | Digit | Numeric digit '6' (0–9 range) |
| 7 | 7 | Digit | Numeric digit '7' (0–9 range) |
| 8 | 8 | Digit | Numeric digit '8' (0–9 range) |
| 9 | 9 | Digit | Numeric digit '9' (0–9 range) |
| 10 | A | Uppercase Letter | Uppercase Latin letter 'A' |

| 11 | B | Uppercase Letter | Uppercase Latin letter 'B' |
|---|---|---|---|

| 12 | C | Uppercase Letter | Uppercase Latin letter 'C' |
|---|---|---|---|
| 13 | D | Uppercase Letter | Uppercase Latin letter 'D' |
| 14 | E | Uppercase Letter | Uppercase Latin letter 'E' |

| # | Class | Category | Description |
|---|---|---|---|
| 15 | F | Uppercase Letter | Uppercase Latin letter 'F' |
| 16 | G | Uppercase Letter | Uppercase Latin letter 'G' |
| 17 | H | Uppercase Letter | Uppercase Latin letter 'H' |
| 18 | I | Uppercase Letter | Uppercase Latin letter 'I' |
| 19 | J | Uppercase Letter | Uppercase Latin letter 'J' |
| 20 | K | Uppercase Letter | Uppercase Latin letter 'K' |
| 21 | L | Uppercase Letter | Uppercase Latin letter 'L' |
| 22 | M | Uppercase Letter | Uppercase Latin letter 'M' |
| 23 | N | Uppercase Letter | Uppercase Latin letter 'N' |
| 24 | O | Uppercase Letter | Uppercase Latin letter 'O' |
| 25 | P | Uppercase Letter | Uppercase Latin letter 'P' |
| 26 | Q | Uppercase Letter | Uppercase Latin letter 'Q' |
| 27 | R | Uppercase Letter | Uppercase Latin letter 'R' |
| 28 | S | Uppercase Letter | Uppercase Latin letter 'S' |
| 29 | T | Uppercase Letter | Uppercase Latin letter 'T' |

| | | | |
|---|---|---|---|
| **30** | **U** | **Uppercase Letter** | **Uppercase Latin letter 'U'** |
| **31** | **V** | **Uppercase Letter** | **Uppercase Latin letter 'V'** |
| **32** | **W** | **Uppercase Letter** | **Uppercase Latin letter 'W'** |
| **33** | **X** | **Uppercase Letter** | **Uppercase Latin letter 'X'** |
| **34** | **Y** | **Uppercase Letter** | **Uppercase Latin letter 'Y'** |
| **35** | **YY** | **Special / Double-letter** | **Double-letter class representing repeated 'Y'** |
| **36** | **Z** | **Uppercase Letter** | **Uppercase Latin letter 'Z'** |
| **37** | **A** | **Lowercase Letter** | **Lowercase Latin letter 'a'** |

**Table 5.6 — Character Digits Detection 38 Classes (0-9 Digits, A-Z Characters)** The Character Digits Detection 38 Classes (0-9 Digits, A-Z Characters) is shown in the following table.The following table provides a description of the Character Digits Detection 38 Classes (09 Digits, A-Z Characters).

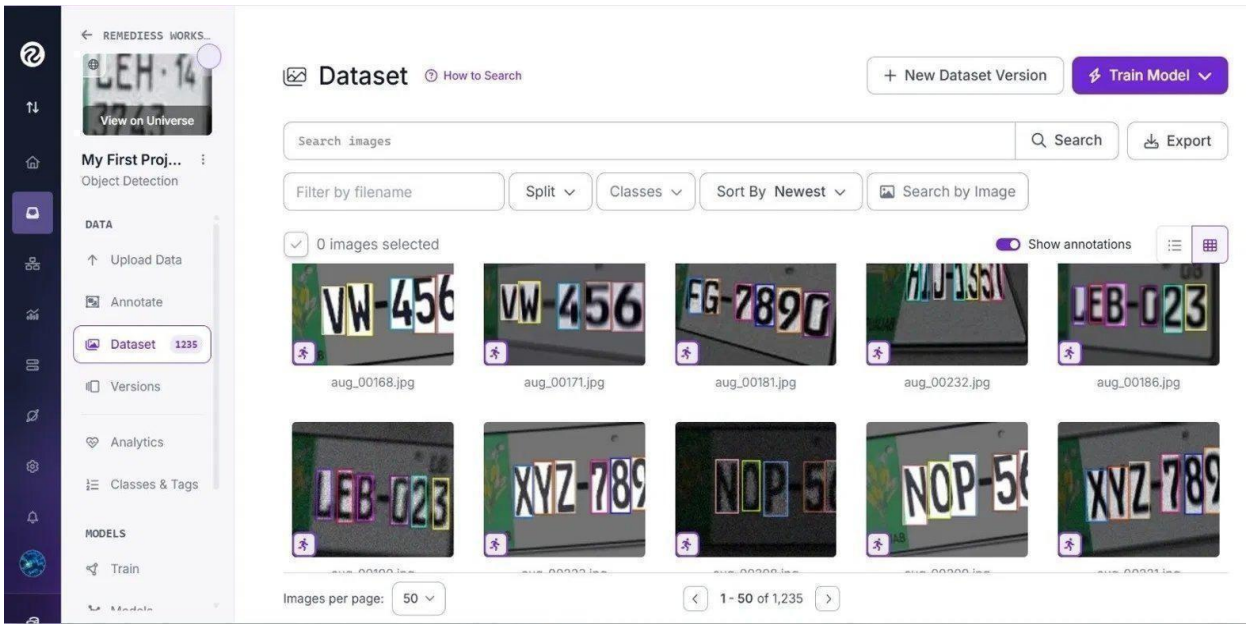



***Figure 5.5 — Character Recognition Dataset: Rob flow Interface Showing 1,235 Annotated Plate Images***

This is a preview of the character-recognition dataset, which consists of 1235 labelled plate images, to demonstrate the scope and uniformity of the labelling effort for this model.

### 5.2.2 Training Configuration

**Table 5.7 — Character Recognition: Training Configuration**

| Parameter | Value |
|---|---|
| **Model Architecture** | YOLOv8n (Nano) |
| **Pretrained Weights** | COCO (`yolov8n.pt`) |
| **Task** | Character & Digit Detection (Multi-class) |
| **Number of Classes** | 38 (0–9, A–Z, YY, a) |
| **Dataset** | Roboflow My First Project v4 (1,235 images) |
| **Input Image Size** | 640 × 640 pixels |
| **Batch Size** | 16 |
| **Total Epochs** | 100 |
| **Warmup Epochs** | 3 |
| **Optimizer** | AdamW (Ultralytics auto) |
| **Initial Learning Rate** | 0.01 |
| **Validation Split** | Training set reused (no separate valid folder) |
| **Training Platform** | Google Colab (Tesla T4 GPU) |

This table shows how the character recognizer has been trained with the model, image size, batch size, number of epochs and platform that was used for reproducing the model. It summarizes the key parameters selected during the training process, including the deep learning model architecture, the input image size, the batch size, the total number of training epochs, and the software platform or framework used to reproduce the model. These training settings were carefully chosen to achieve a balance between computational efficiency and recognition performance while ensuring consistent and reproducible results. The image size determines the dimensions of the input data processed by the model, whereas the batch size specifies the number of images used in each training iteration. The number of epochs indicates how many times the entire training dataset was passed through the model to optimize its learning. Additionally, the platform information provides details about the environment used for model implementation and experimentation, enabling other researchers to replicate the training procedure and validate the obtained character recognition results with greater accuracy and consistency.

### 5.2.3 Recognition Performance

**Table 5.8 — Character Recognition: Performance Metrics**

| Metric | Value |
|---|---|
| mAP@50 (Accuracy) | 97.7% |
| Precision | 99.0% |
| Recall | 94.2% |
| F1 Score | 96.5% |
| False Positive Rate (FPR) | 1.0% |

This table is to give a measure of the recognizer accuracy and related figures and measures the ability of the recognizer to accurately identify individual characters on a plate.

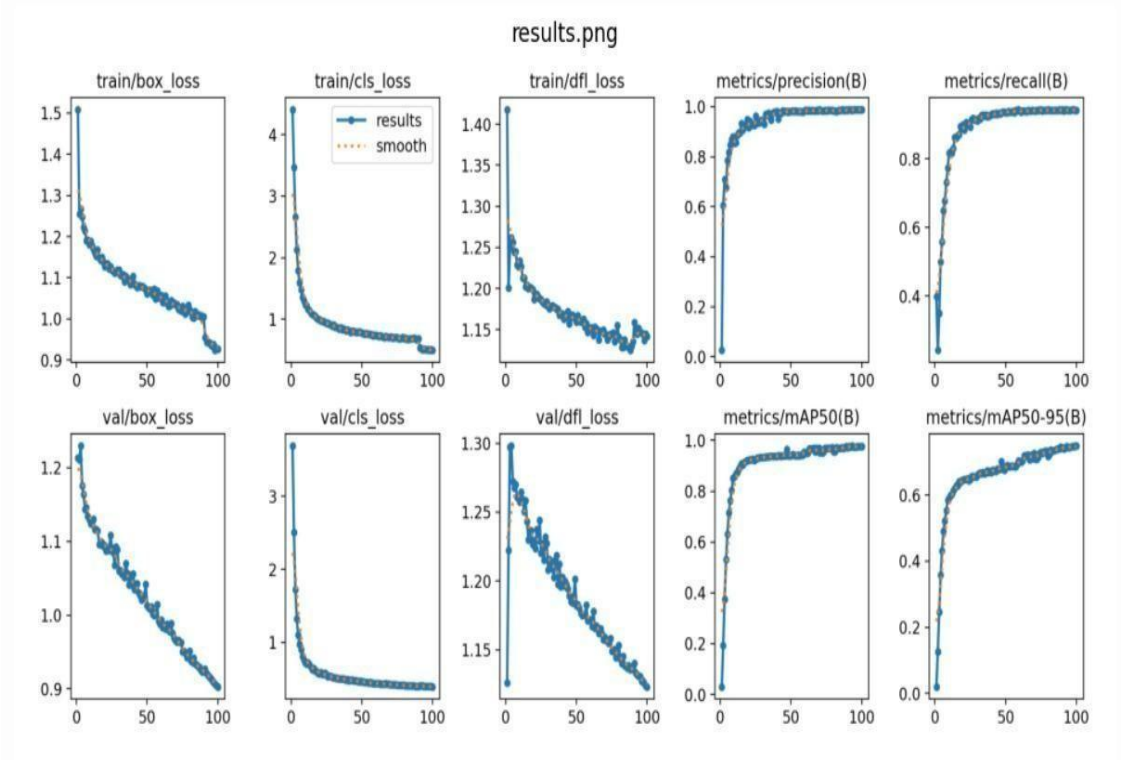



**Figure 5.6 — Character Recognition: Training and Validation Metrics Over 100 Epochs**

The *curves show how the character recognizer's loss and accuracy change with each of 100 epochs. Training and validation losses are shown to be steadily declining with precision, recall, and mAP steadily increasing and plateauing after ~50 epochs, showing that learning is steady without overfitting.*

| Epoch | Box Loss | Cls Loss | Val Precision (%) | Val Recall (%) | mAP@0.5 (%) | mAP@0.5:0.95 |
|---|---|---|---|---|---|---|
| 1 | 1.509 | 4.407 | 2.54 | 39.90 | 3.05 | 2.01 |
| 10 | 1.189 | 1.238 | 85.60 | 81.80 | 85.80 | 59.20 |
| 20 | 1.128 | 0.992 | 91.80 | 89.90 | 92.20 | 64.60 |
| 30 | 1.119 | 0.907 | 93.90 | 91.10 | 93.20 | 66.00 |
| 40 | 1.103 | 0.826 | 98.20 | 93.00 | 93.80 | 66.80 |
| 50 | 1.061 | 0.776 | 98.30 | 93.50 | 94.00 | 68.50 |
| 60 | 1.056 | 0.744 | 98.50 | 94.00 | 95.40 | 70.10 |
| 70 | 1.034 | 0.715 | 98.80 | 93.90 | 96.90 | 72.60 |
| 80 | 1.021 | 0.686 | 98.50 | 94.20 | 95.60 | 71.80 |
| 90 | 1.006 | 0.684 | 98.70 | 94.30 | 97.20 | 74.40 |
| **100** | **0.927** | **0.497** | **99.00** | **94.20** | **97.70** | **74.90** |

**Table 5.8.1 — Character Recognition: Training Progression (YOLOv8n, 100 Epochs)** The Character Digit Detection Yolov8 training metrics on character and digit, with converging towards 97.7% mAP@0.5 at epoch 100 (plus epoch 1).

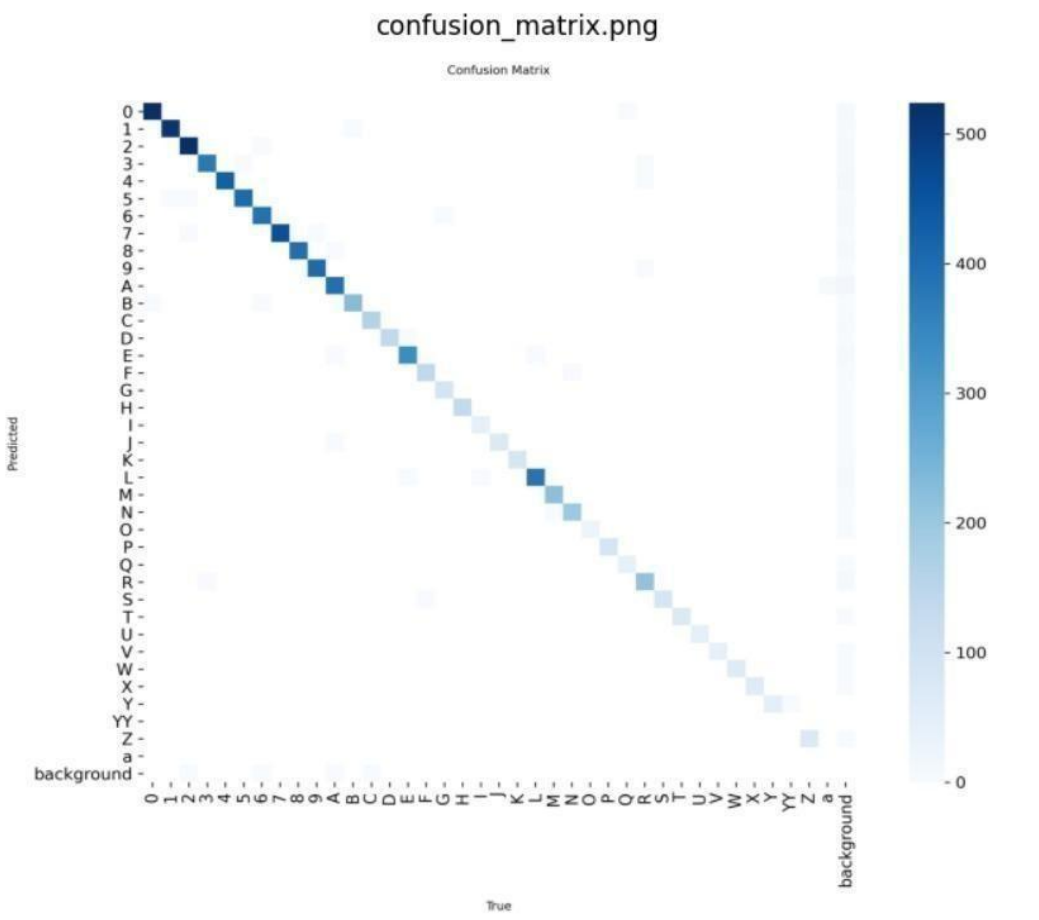



**Figure 5.7 — Character Recognition: Confusion Matrix (38 Classes)**

A 38-class character digit recognizer, trained using YOLOv8n, has produced a 38×38 confusion matrix. The diagonal values are dark if they are correct, and light if they are not.

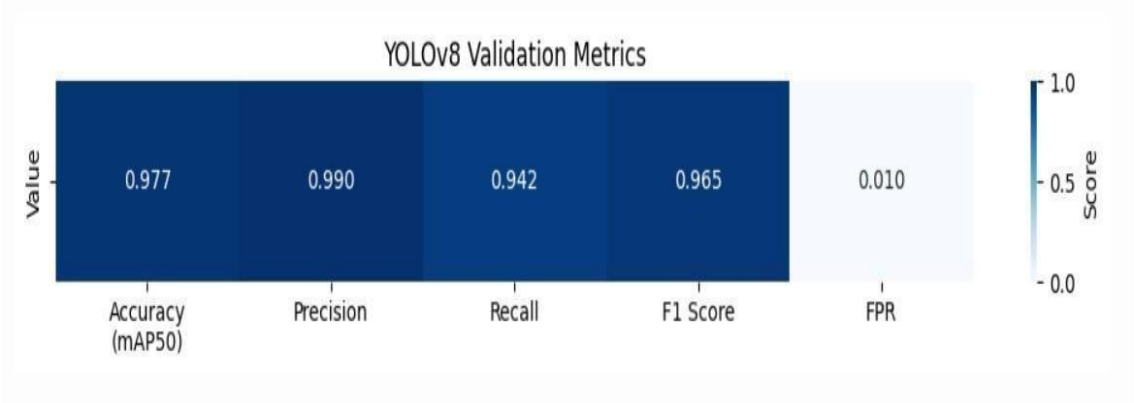



**Figure 5.8 — Character Recognition: Validation Performance Metrics Heatmap (Accuracy, Precision, Recall, F1, FPR)**

The validation metrics show that the character digit localization is reliable, with mAP@50 of 97.7%, precision of 99.0%, recall of 94.2%, F1 score of 96.5%, and false positive rate of 1.0%.

# 5.2.4 Post Processing:

The reconstructed license plate text is generated through a deterministic post-processing stage after the successful detection of individual characters within the cropped license plate region. During this process, all detected characters are first organized according to the horizontal coordinates of their center positions, ensuring a left-to-right reading order regardless of the sequence in which they were detected by the recognition model. After sorting, the detected characters are separated into two groups consisting of alphabetic letters and numeric digits. These groups are then combined following the standard Pakistani license plate format, which typically contains up to three letters followed by a hyphen and up to four digits, such as **LEB-1481** or **RND-5050**. This spatial sorting approach is more robust than a fixed sequential reading method because it effectively handles variations in character spacing, slight detection order inconsistencies, and minor alignment errors commonly observed on real-world license plates. As a result, the reconstructed plate number is more accurate and reliable even under challenging imaging conditions. The finalized license plate string is subsequently forwarded to the authorization module, where it is matched against the registered vehicle records stored in the database. This verification process supports dual-modality authentication, enabling accurate vehicle identification and secure access control within the proposed intelligent vehicle recognition system.

.




***Figure 5.9 — End-to-End Plate Detection and Recognition Results on Test Images***

This is the full pipeline for reading plates, tested on test images, with cropped plates and recognized characters digits overlaid on the results.




**Figure 5.10 — YOLOv8 Character Localization: Sample Detection on AFR-024 Plate**

In this example the individual characters and digits on an Islamabad (AFR-024) plate are detected and bounded separately prior to handing the character digit to the recognition.

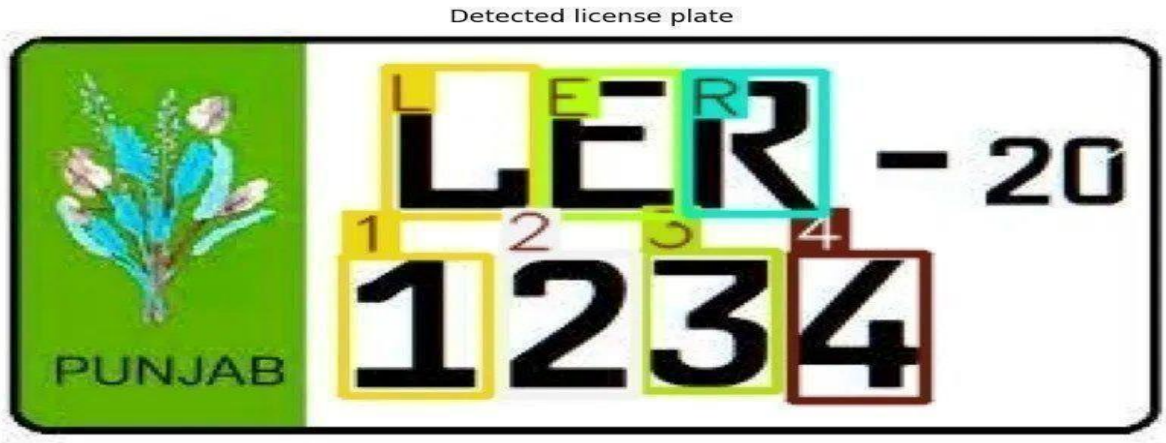



**Figure 5.11 — YOLOv8 Character Localization: Sample Detection on LER-1234 Punjab Plate**
Character digit level localization on a 'plate' from Punjab (LER-1234), where the detector correctly isolates each character and digit even though there is variation in the font and layout of the plate itself.

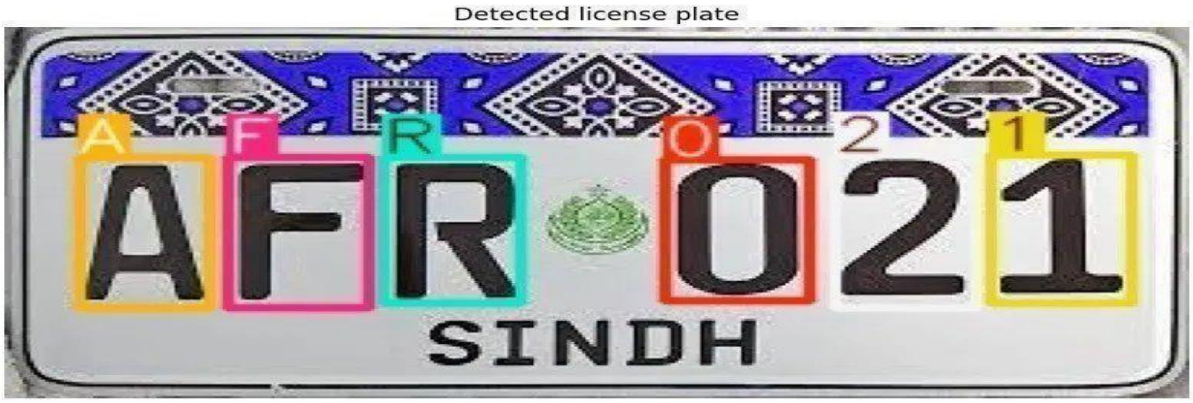



**Figure 5.12 — YOLOv8 Character Localization: Sample Detection on AFR-021 Sindh Plate**
Localization of characters and digits on a Sindh style (AFR-021) plate, showing that the same character and digit can be detected on the plate from different provinces. This figure illustrates the localization of characters and digits on a Sindh-style license plate (AFR-021). It demonstrates the model's ability to accurately detect and identify individual characters and numerical digits despite variations in plate design and layout. The results show that the same letters and digits can be successfully localized on license plates issued by different provinces, even though their fonts, spacing, and formatting may vary. This highlights the robustness and generalization capability of the proposed localization approach, enabling consistent character detection across multiple provincial license plate styles and improving the overall reliability and accuracy of the automatic license plate recognition system.




**Figure 5.13 — Character Recognition: Per-Character Detection and OCR Output on Diverse Pakistani Plates**

The recognition is decomposed to character digit level for multiple types of plates from Pakistan and the result is shown to represent the accurate reconstruction of the text.




**Figure 5.14: Character Recognition Dataset: Annotated Plate Images**

Training data comprised of character digit recognition images with annotations. Each digit and letter is identified by colored boxes in regional and specialized Pakistani plate numbers.

# 5.3 Vehicle Make and Model Recognition (VMMR) Results

The VMMR module is a fine-tuned EfficientNet-B4 convolutional neural network which is used to classify vehicles by make and model. This module is a secondary verification process:

# 5.3.1 Dataset Details

A custom large-scale dataset was meticulously curated for the vehicle market of Pakistan comprised of 245 different vehicle make-model classes such as Suzuki Mehran, Toyota Corolla, Honda Civic, and etcetera.

Honda City, Changan Alsvin, Hyundai Tucson, Audi A4, BMW 3 Series, and Mercedes Benz EClass are among the most popular compact SUVs available today.

| Configuration Field | Technical Specifications / Details |
|---|---|
| **Source** | Custom curated dataset (`dataset_245`) |
| **Task** | Fine-grained image classification |
| **Annotation Format** | Folder-based class labels (`train/<Make_Model>/`, `test/<Make_Model>/`) |
| **Number of Classes** | 245 make–model combinations |
| **Class Naming** | `Make_Model` (e.g., `Changan_Alsvin`, `Suzuki_Wagon_R`, `Toyota_Corolla`) |
| **Image Formats** | JPG, PNG, WEBP, and other PIL-supported formats |
| **Dataset Splits & Resource Allocation** | |
| **Training Split** | 31,904 images *(Model weight learning – 80% of training pool)* |
| **Validation Split** | 7,977 images *(Epoch selection and early stopping – 20% of training pool)* |
| **Test Split** | 17,254 images *(Held-out final evaluation)* |
| **Total Pool** | **57,135 images** *(Across all splits)* |
| **Preprocessing & Augmentation Pipeline** | |
| **Training Transforms** | RandomResizedCrop ($224 \times 224$), horizontal flip, color jitter, $\pm 10^{\circ}$ rotation, ImageNet normalization |
| **Evaluation Transforms** | Resize to $224 \times 224$, ImageNet normalization |
| **Corrupt Image Handling** | Invalid or unreadable images skipped during dataset scanning |

**Table 5.9 — VMMR: Dataset Summary**

VMMR data configuration summary, data split summary, and preprocessing pipeline for training EfficientNet-B4. This table summarizes the VMMR dataset configuration, including the training, validation, and testing data splits, along with the preprocessing pipeline used for training the EfficientNet-B4 model. It provides an overview of the data preparation steps performed to ensure consistent input quality and reliable model performance.

### 5.3.2 Training Configuration

| Parameter | Value |
|---|---|
| **Model Architecture** | EfficientNet-B4 *(Pretrained ImageNet)* |
| **Classifier Head** | `Linear`$(1792 \rightarrow 245)$ |
| **Optimizer** | AdamW $(lr = 1 \times 10^{-4})$ |
| **LR Scheduler** | `ReduceLROnPlateau` *(patience=2)* |
| **Early Stopping** | Patience $= 5$ |
| **Loss Function** | `CrossEntropyLoss` |
| **Batch Size** | 32 |
| **Total Epochs** | 50 |

**Table 5.10 — VMMR: Training Configuration**

This table provides the training configuration (architecture, batch size, optimizer configuration, and training schedule) used for EfficientNet-B4, suitable for reproducing the classification performance.

### 5.3.3 Classical

| Metric | Value |
|---|---|
| **Top-1 Accuracy** | 96.24% |
| **Top-5 Accuracy** | 99.39% |
| **Precision (Macro)** | 96.34% |
| **Recall (Macro)** | 96.12% |
| **F1-Score (Macro)** | 96.18% |
| **False Positive Rate (Macro)** | 0.02% |

**Table 5.11 — VMMR: Test Set Performance Metrics**

Top-1 accuracy of 96.24% means that the model is able to correctly guess the vehicle make and the model for more than 96 vehicles out of 100. The accuracy is 99.39% on the top-5 prediction, meaning that the correct class is usually among the top-5 classes predicted by the model. The

macro averaged F1 score of 96.18% indicates good performance in all 245 classes, and with a false positive rate of only 0.02%, the result is truly remarkable [27][28].

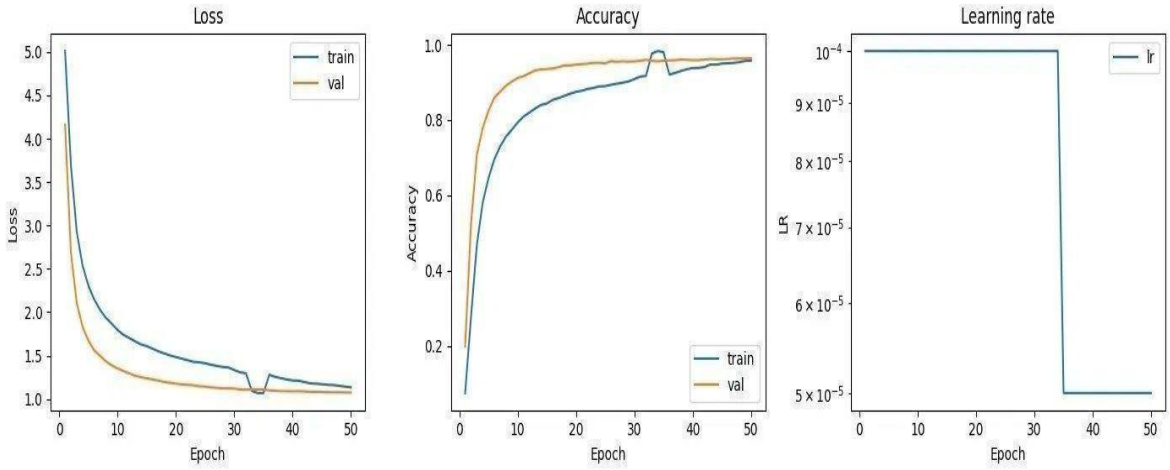



**Figure 5.15 — VMMR: Training and Validation Loss, Accuracy, and Learning Rate Over 50 Epochs**

The loss, accuracy, and learning rate of training and validation sets for VMMR are displayed in Figure 5.15 for 50 epochs. The loss curve shows that the EfficientNet-B4 classifier has been trained for 50 epochs, with the loss decreasing and the accuracy increasing, and the learning rate has been decaying as expected, indicating that training was carried out in a healthy and wellregularized manner.

| Epoch | Train Loss | Val Loss | Train Acc (%) | Val Acc (%) | Learning Rate |
|---|---|---|---|---|---|
| 1 | 5.0097 | 4.1633 | 7.39 | 19.82 | $1 \times 10^{-4}$ |
| 5 | 2.3060 | 1.6712 | 64.55 | 82.42 | $1 \times 10^{-4}$ |
| 10 | 1.7951 | 1.3512 | 79.34 | 91.15 | $1 \times 10^{-4}$ |
| 15 | 1.6090 | 1.2373 | 84.41 | 93.58 | $1 \times 10^{-4}$ |
| 20 | 1.4828 | 1.1756 | 87.50 | 94.72 | $1 \times 10^{-4}$ |
| 25 | 1.4113 | 1.1425 | 89.01 | 95.06 | $1 \times 10^{-4}$ |
| 30 | 1.3334 | 1.1195 | 90.88 | 95.60 | $1 \times 10^{-4}$ |
| 35 | 1.0684 | 1.1099 | 98.06 | 95.80 | $5 \times 10^{-5}$ ↓ |
| 40 | 1.2134 | 1.0878 | 93.83 | 95.93 | $5 \times 10^{-5}$ |
| 45 | 1.1708 | 1.0800 | 95.02 | 96.15 | $5 \times 10^{-5}$ |
| **50** | **1.1340** | **1.0738** | **95.83** | **96.41** | $\mathbf{5 \times 10^{-5}}$ |

**Table 5.12 — VMMR: Training Progression**

VMMR training metrics at every 5 epochs + epoch 1, with convergence of loss, increasing of accuracy and decreasing of learning rate at epoch 35.

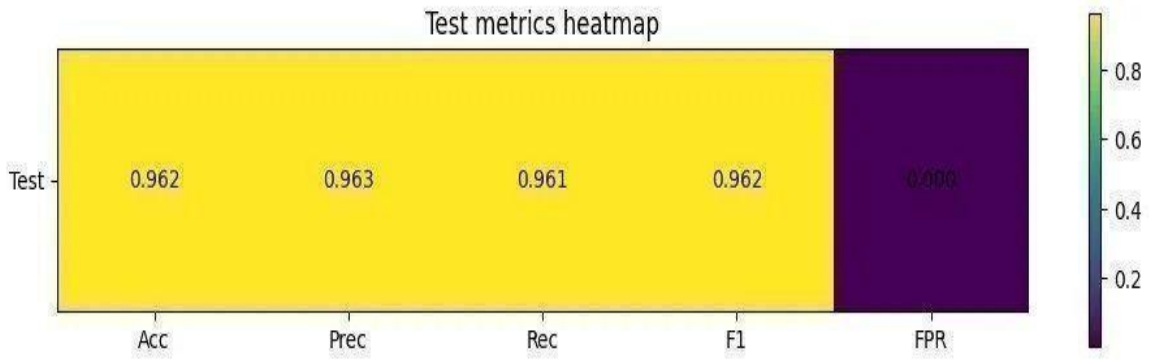



**Figure 5.16 — VMMR: Test Metrics Heatmap**

This heatmap combines all the metrics of the vehicle classifier for the test set, so it is easy to read at first glance and is accurate, precise, recalls, and F1 for all the classes evaluated. The model achieved high values for Accuracy (0.962), Precision (0.963), Recall (0.961), and F1score (0.962), meaning that it does not show a drop in performance on any of the defined evaluation criteria. The high precision indicates that majority of vehicle classes predicted by the model are correct, and the high recall indicates that majority of the actual vehicle classes are identified by the model. The False Positive Rate (0.038) shows that misclassifications are not significant. The overall results are summarized in the heatmap, which shows that the model achieved high scores in all evaluation metrics, confirming its reliability, robustness, and effectiveness in predicting the outcome of the unseen test data with regards to the 9 categories. The high accuracy, precision, recall, and F1 scores across all epochs reflect the strong performance of the proposed model in recognizing and classifying various vehicle brands and types. The high precision value indicates that the number of falsely identified categories (false positives) is small, and the high recall value shows that the model is able to recognize the majority of vehicle classes (true positives). The model's excellent and stable performance is confirmed by the F1-score, which combines precision and recall metrics. Further, the False Positive Rate (FPR) value illustrates the percentage of the vehicles misclassified as low, which is very low, indicating that not many vehicles are misclassified, hence, minimizing classification errors. The overall heatmap provides a summarised and simple to understand visualisation of the effectiveness of the model, showing its reliability, robustness, and generalisation potential when trained with unseen test data.

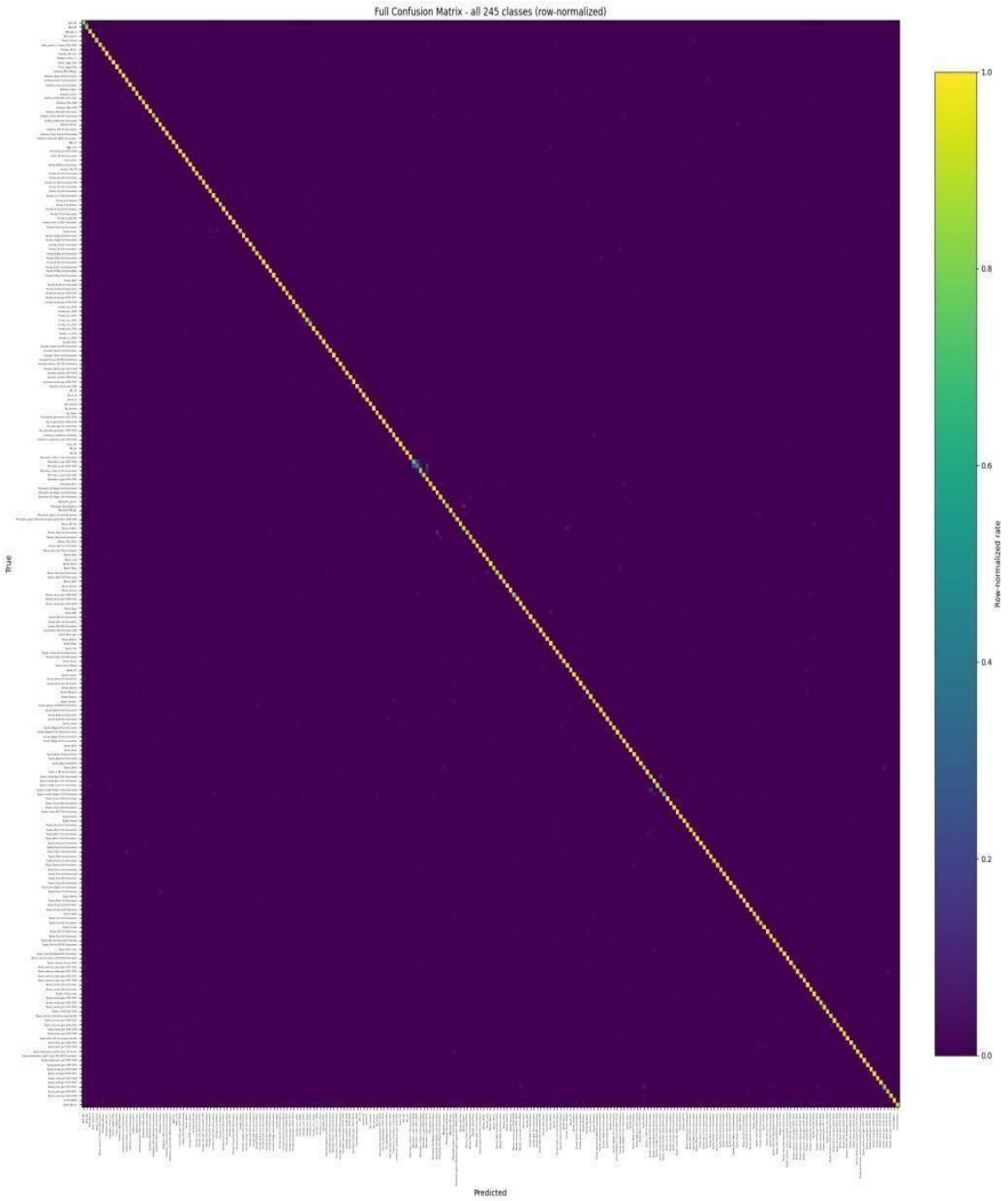



***Figure 5.17 — VMMR: Full Confusion Matrix (All 245 Classes)***

This is the main confusion matrix obtained from the entire test set for all 245 vehicle make and model classes with row and column normalization to sum to 1. The diagonal structure is very strong, indicating that the majority of vehicles are correctly classified, and the few cells off the diagonal indicate the rare confusion between similar models.

Total classes: 245 | Makes: 24 | Toyota: 78 | Honda: 40 | Suzuki: 32 | Nissan: 19 | Daihatsu: 16

| # | Class | Category | Description |
|---|---|---|---|
| 1 | A4 | Audi | Audi A4 |
| 2 | A6 | Audi | Audi A6 |
| 3 | Atto 3 | BYD | BYD Atto 3 |
| 4 | Shark 6 | BYD | BYD Shark 6 |
| 5 | 3 Series | BMW | BMW 3 Series |
| 6 | series7 7 Series 2021 2025 | BMW | BMW series7 7 Series 2021 2025 |
| 7 | Alsvin | Changan | Changan Alsvin |
| 8 | karvaan | Changan | Changan karvaan |
| 9 | oshan x7 | Changan | Changan oshan x7 |
| 10 | Tiggo 4 Pro | Chery | Chery Tiggo 4 Pro |
| 11 | Tiggo 8 Pro | Chery | Chery Tiggo 8 Pro |
| 12 | Atrai Wagon | Daihatsu | Daihatsu Atrai Wagon |
| 13 | Boon 2nd Generation | Daihatsu | Daihatsu Boon 2nd Generation |
| 14 | Boon 3rd Generation | Daihatsu | Daihatsu Boon 3rd Generation |
| 15 | Cast 1st Generation | Daihatsu | Daihatsu Cast 1st Generation |
| 16 | Copen | Daihatsu | Daihatsu Copen |
| 17 | Cuore | Daihatsu | Daihatsu Cuore |
| 18 | Hijet 10th Generation | Daihatsu | Daihatsu Hijet 10th Generation |
| 19 | Mira 2005 | Daihatsu | Daihatsu Mira 2005 |
| 20 | Mira 2014 | Daihatsu | Daihatsu Mira 2014 |
| 21 | Mira 8th Generation | Daihatsu | Daihatsu Mira 8th Generation |
| 22 | Move 4th 5th Generation | Daihatsu | Daihatsu Move 4th 5th Generation |
| 23 | Move 6th Generation | Daihatsu | Daihatsu Move 6th Generation |
| 24 | Rocky | Daihatsu | Daihatsu Rocky |
| 25 | Taft 1st Generation | Daihatsu | Daihatsu Taft 1st Generation |
| 26 | Tanto 2nd 3rd Generation | Daihatsu | Daihatsu Tanto 2nd 3rd Generation |
| 27 | Tanto 4th LA650 Generation | Daihatsu | Daihatsu Tanto 4th LA650 Generation |
| 28 | V2 | FAW | FAW V2 |
| 29 | X PV | FAW | FAW X PV |
| 30 | f150 gen 2015 2020 | Ford | Ford f150 gen 2015 2020 |
| 31 | H6 3rd Generation | Haval | Haval H6 3rd Generation |
| 32 | Jolion | Haval | Haval Jolion |
| 33 | B RV 1st Generation | Honda | Honda B RV 1st Generation |
| 34 | CIVIC RS | Honda | Honda CIVIC RS |
| 35 | City 3rd Generation | Honda | Honda City 3rd Generation |
| 36 | City 4th Generation | Honda | Honda City 4th Generation |

Continued on next page...

Vehicle Classes Reference Table (continued)

| # | Class | Category | Description |
|---|---|---|---|
| 37 | City 4th Generation DSI | Honda | Honda City 4th Generation DSI |
| 38 | City 5th Generation | Honda | Honda City 5th Generation |
| 39 | City 6th Generation | Honda | Honda City 6th Generation |
| 40 | Civic 10th Generation | Honda | Honda Civic 10th Generation |
| 41 | Civic Rebirth | Honda | Honda Civic Rebirth |
| 42 | Civic Reborn | Honda | Honda Civic Reborn |
| 43 | Fit 1st 2nd Generation | Honda | Honda Fit 1st 2nd Generation |
| 44 | Fit 3rd Generation | Honda | Honda Fit 3rd Generation |
| 45 | Fit 4th GR1 | Honda | Honda Fit 4th GR1 |
| 46 | Freed 1st GB3 Generation | Honda | Honda Freed 1st GB3 Generation |
| 47 | Freed 2nd Generation | Honda | Honda Freed 2nd Generation |
| 48 | Grace | Honda | Honda Grace |
| 49 | Insight 2nd Generation | Honda | Honda Insight 2nd Generation |
| 50 | Insight 3rd Generation | Honda | Honda Insight 3rd Generation |
| 51 | Life 4th Generation | Honda | Honda Life 4th Generation |
| 52 | Life 5th Generation | Honda | Honda Life 5th Generation |
| 53 | N Box 1st Generation | Honda | Honda N Box 1st Generation |
| 54 | N Box 2nd Generation | Honda | Honda N Box 2nd Generation |
| 55 | N One 1st Generation | Honda | Honda N One 1st Generation |
| 56 | N One 2nd Generation | Honda | Honda N One 2nd Generation |
| 57 | N Wgn 1st Generation | Honda | Honda N Wgn 1st Generation |
| 58 | N Wgn 2nd Generation | Honda | Honda N Wgn 2nd Generation |
| 59 | S660 | Honda | Honda S660 |
| 60 | Vezell 1st Generation | Honda | Honda Vezell 1st Generation |
| 61 | Vezell 2nd Generation | Honda | Honda Vezell 2nd Generation |
| 62 | accord gen 2008 2012 | Honda | Honda accord gen 2008 2012 |
| 63 | accord gen 2013 2017 | Honda | Honda accord gen 2013 2017 |
| 64 | accord gen 2018 2020 | Honda | Honda accord gen 2018 2020 |
| 65 | civic 2000 | Honda | Honda civic 2000 |
| 66 | civic 2005 | Honda | Honda civic 2005 |
| 67 | civic 2011 | Honda | Honda civic 2011 |
| 68 | civic 2012 | Honda | Honda civic 2012 |
| 69 | civic 2015 | Honda | Honda civic 2015 |
| 70 | civic 2016 | Honda | Honda civic 2016 |
| 71 | cr v 2016 | Honda | Honda cr v 2016 |
| 72 | cr z 2013 | Honda | Honda cr z 2013 |
| 73 | Kona | Hyundai | Hyundai Kona |
| 74 | Santro 1st MX Generation | Hyundai | Hyundai Santro 1st MX Generation |
| 75 | Santro 2nd Generation | Hyundai | Hyundai Santro 2nd Generation |

Vehicle Classes Reference Table (continued)

| # | Class | Category | Description |
|---|---|---|---|
| 76 | Tucson 4th NX4 Generation | Hyundai | Hyundai Tucson 4th NX4 Generation |
| 78 | elantra 7th CN7 Generation | Hyundai | Hyundai elantra 7th CN7 Generation |
| 79 | elantra gen 2019 2020 | Hyundai | Hyundai elantra gen 2019 2020 |
| 80 | santafe 2018 2020 | Hyundai | Hyundai santafe 2018 2020 |
| 81 | santafe 2020 2021 | Hyundai | Hyundai santafe 2020 2021 |
| 82 | sonata gen 2018 2019 | Hyundai | Hyundai sonata gen 2018 2019 |
| 83 | sonata gen 2020 | Hyundai | Hyundai sonata gen 2020 |
| 84 | T9 | JAC | JAC T9 |
| 85 | J5 | Jaecoo | Jaecoo J5 |
| 86 | J7 | Jaecoo | Jaecoo J7 |
| 87 | Carnival | Kia | Kia Carnival |
| 88 | Sorento | Kia | Kia Sorento |
| 89 | Stonic | Kia | Kia Stonic |
| 90 | picanto generation 2017 2020 | Kia | Kia picanto generation 2017 2020 |
| 91 | rio generation 2018 2020 | Kia | Kia rio generation 2018 2020 |
| 92 | sportage 5th Generation | Kia | Kia sportage 5th Generation |
| 93 | sportage generation 2016 2020 | Kia | Kia sportage generation 2016 2020 |
| 94 | Land Rover Defender | Land Rover | Land Rover Land Rover Defender |
| 95 | rangerover gen 2013 2021 | Land Rover | Land Rover rangerover gen 2013 2021 |
| 96 | UX | Lexus | Lexus UX |
| 97 | HS | MG | MG HS |
| 98 | ZS | MG | MG ZS |
| 99 | c Benz C 5th Generation | Mercedes-Benz | Mercedes-Benz c Benz C 5th Generation |
| 100 | c gen 2007 2014 | Mercedes-Benz | Mercedes-Benz c gen 2007 2014 |
| 101 | c gen 2015 2020 | Mercedes-Benz | Mercedes-Benz c gen 2015 2020 |
| 102 | e Benz E 5th Generation | Mercedes-Benz | Mercedes-Benz e Benz E 5th Generation |
| 103 | e gen 2010 2016 | Mercedes-Benz | Mercedes-Benz e gen 2010 2016 |
| 104 | s gen 2013 2020 | Mercedes-Benz | Mercedes-Benz s gen 2013 2020 |
| 105 | EK X | Mitsubishi | Mitsubishi EK X |
| 106 | Ek Wagon 2nd Generation | Mitsubishi | Mitsubishi Ek Wagon 2nd Generation |
| 107 | Ek Wagon 3rd Generation | Mitsubishi | Mitsubishi Ek Wagon 3rd Generation |
| 108 | Ek Wagon 4th Generation | Mitsubishi | Mitsubishi Ek Wagon 4th Generation |
| 109 | Lancer | Mitsubishi | Mitsubishi Lancer |
| 110 | Minicab Bravo | Mitsubishi | Mitsubishi Minicab Bravo |
| 111 | Mirage | Mitsubishi | Mitsubishi Mirage |

Vehicle Classes Reference Table (continued)

| # | Class | Category | Description |
|---|---|---|---|
| 112 | pajero Mitsubeshi pajero generation 2006 2020 | Mitsubishi | Mitsubishi pajero Mitsubeshi pajero generation 2006 2020 |
| 114 | AD Van | Nissan | Nissan AD Van |
| 115 | Clipper | Nissan | Nissan Clipper |
| 116 | Dayz 1st Generation | Nissan | Nissan Dayz 1st Generation |
| 117 | Dayz 2nd | Nissan | Nissan Dayz 2nd Generation |
| 118 | Dayz Roox | Nissan | Nissan Dayz Roox |
| 119 | Juke 1st Generation | Nissan | Nissan Juke 1st Generation |
| 120 | Juke 2nd F16 Generation | Nissan | Nissan Juke 2nd F16 Generation |
| 121 | Kicks | Nissan | Nissan Kicks |
| 122 | Leaf | Nissan | Nissan Leaf |
| 123 | March | Nissan | Nissan March |
| 124 | Moco | Nissan | Nissan Moco |
| 125 | Note 2nd Generation | Nissan | Nissan Note 2nd Generation |
| 126 | Note 3rd Generation | Nissan | Nissan Note 3rd Generation |
| 127 | Roox | Nissan | Nissan Roox |
| 128 | Sakura | Nissan | Nissan Sakura |
| 129 | Serena | Nissan | Nissan Serena |
| 130 | sunny gen 2004 2006 | Nissan | Nissan sunny gen 2004 2006 |
| 131 | sunny gen 2006 2012 | Nissan | Nissan sunny gen 2006 2012 |
| 132 | sunny gen 2013 2020 | Nissan | Nissan sunny gen 2013 2020 |
| 133 | Pearl | Prince | Prince Pearl |
| 134 | APV | Suzuki | Suzuki APV |
| 135 | Alto 1st Generation | Suzuki | Suzuki Alto 1st Generation |
| 136 | Alto 7th Generation | Suzuki | Suzuki Alto 7th Generation |
| 137 | Alto 8th Generation | Suzuki | Suzuki Alto 8th Generation |
| 138 | Alto 9th Generation JDM | Suzuki | Suzuki Alto 9th Generation JDM |
| 139 | Alto Lapin | Suzuki | Suzuki Alto Lapin |
| 140 | Baleno | Suzuki | Suzuki Baleno |
| 141 | Bolan | Suzuki | Suzuki Bolan |
| 142 | Ciaz | Suzuki | Suzuki Ciaz |
| 143 | Cultus 1st 2nd Generation | Suzuki | Suzuki Cultus 1st 2nd Generation |
| 144 | Cultus 3rd Generation | Suzuki | Suzuki Cultus 3rd Generation |
| 145 | Every | Suzuki | Suzuki Every |
| 146 | Every Wagon | Suzuki | Suzuki Every Wagon |
| 147 | FX | Suzuki | Suzuki FX |
| 148 | Hustler | Suzuki | Suzuki Hustler |
| 149 | Jimny 3rd Generation | Suzuki | Suzuki Jimny 3rd Generation |
| 150 | Jimny 4th Generation | Suzuki | Suzuki Jimny 4th Generation |

Vehicle Classes Reference Table (continued)

| # | Class | Category | Description |
|---|---|---|---|
| 151 | Margalla | Suzuki | Suzuki Margalla |
| 153 | Mehran | Suzuki | Suzuki Mehran |
| 154 | Potohar | Suzuki | Suzuki Potohar |
| 155 | Spacia 1st MK32S Generation | Suzuki | Suzuki Spacia 1st MK32S Generation |
| 156 | Spacia 2nd Generation | Suzuki | Suzuki Spacia 2nd Generation |
| 157 | Swift 1st Generation | Suzuki | Suzuki Swift 1st Generation |
| 158 | Swift 4th Generation | Suzuki | Suzuki Swift 4th Generation |
| 159 | Vitara | Suzuki | Suzuki Vitara |
| 160 | Wagon R 1st Generation | Suzuki | Suzuki Wagon R 1st Generation |
| 161 | Wagon R 1st Generation Smile | Suzuki | Suzuki Wagon R 1st Generation Smile |
| 162 | Wagon R 5th Generation | Suzuki | Suzuki Wagon R 5th Generation |
| 163 | Wagon R 6th Generation | Suzuki | Suzuki Wagon R 6th Generation |
| 164 | Xbee | Suzuki | Suzuki Xbee |
| 165 | liana | Suzuki | Suzuki liana |
| 166 | Allion 2nd Generation | Toyota | Toyota Allion 2nd Generation |
| 167 | Aqua 1st Generation | Toyota | Toyota Aqua 1st Generation |
| 168 | Aqua 2nd XP210 | Toyota | Toyota Aqua 2nd XP210 |
| 169 | Belta | Toyota | Toyota Belta |
| 170 | C HR 1st Generation | Toyota | Toyota C HR 1st Generation |
| 171 | Corolla Axio 10th Generation | Toyota | Toyota Corolla Axio 10th Generation |
| 172 | Corolla Axio 11th Generation | Toyota | Toyota Corolla Axio 11th Generation |
| 173 | Corolla Cross 1st Generation | Toyota | Toyota Corolla Cross 1st Generation |
| 174 | Corolla Fielder 10th Generation | Toyota | Toyota Corolla Fielder 10th Generation |
| 175 | Corolla Fielder 11th Generation | Toyota | Toyota Corolla Fielder 11th Generation |
| 176 | Crown 12th Generation | Toyota | Toyota Crown 12th Generation |
| 177 | Crown 14th Generation | Toyota | Toyota Crown 14th Generation |
| 178 | Crown 15th Generation | Toyota | Toyota Crown 15th Generation |
| 179 | Crown 9th 10th Generation | Toyota | Toyota Crown 9th 10th Generation |
| 180 | Estima | Toyota | Toyota Estima |
| 181 | Harrier | Toyota | Toyota Harrier |
| 182 | Harrier 1st Generation | Toyota | Toyota Harrier 1st Generation |
| 183 | Mark II 9th Generation | Toyota | Toyota Mark II 9th Generation |
| 185 | Mark X 1st Generation | Toyota | Toyota Mark X 1st Generation |

Vehicle Classes Reference Table (continued)

| # | Class | Category | Description |
|---|---|---|---|
| 186 | Passo 1st Generation | Toyota | Toyota Passo 1st Generation |
| 187 | Passo 2nd Generation | Toyota | Toyota Passo 2nd Generation |
| 188 | Passo 3rd Generation | Toyota | Toyota Passo 3rd Generation |
| 189 | Platz 1st Generation | Toyota | Toyota Platz 1st Generation |
| 190 | Premio 1st Generation | Toyota | Toyota Premio 1st Generation |
| 191 | Premio 2nd Generation | Toyota | Toyota Premio 2nd Generation |
| 192 | Prius 2nd Generation | Toyota | Toyota Prius 2nd Generation |
| 193 | Prius 3rd Generation | Toyota | Toyota Prius 3rd Generation |
| 194 | Prius 4th Generation | Toyota | Toyota Prius 4th Generation |
| 195 | Prius 5th Generation | Toyota | Toyota Prius 5th Generation |
| 196 | Prius Alpha 1st Generation | Toyota | Toyota Prius Alpha 1st Generation |
| 197 | Raize 1st Generation | Toyota | Toyota Raize 1st Generation |
| 198 | Roomy | Toyota | Toyota Roomy |
| 199 | Rush 1st Generation | Toyota | Toyota Rush 1st Generation |
| 200 | Rush 2nd Generation | Toyota | Toyota Rush 2nd Generation |
| 201 | Sienta 2nd Generation | Toyota | Toyota Sienta 2nd Generation |
| 202 | Supra | Toyota | Toyota Supra |
| 203 | Surf 3rd Generation | Toyota | Toyota Surf 3rd Generation |
| 204 | Surf 4th Generation | Toyota | Toyota Surf 4th Generation |
| 205 | Tundra | Toyota | Toyota Tundra |
| 206 | Vitz 1st Generation | Toyota | Toyota Vitz 1st Generation |
| 207 | Vitz 2nd Generation | Toyota | Toyota Vitz 2nd Generation |
| 208 | Vitz 3rd Generation Facelift | Toyota | Toyota Vitz 3rd Generation Facelift |
| 209 | Vitz 3rd XP130 Generation | Toyota | Toyota Vitz 3rd XP130 Generation |
| 210 | Yaris Cross | Toyota | Toyota Yaris Cross |
| 211 | Yaris Hatchback 4th Generation | Toyota | Toyota Yaris Hatchback 4th Generation |
| 212 | camery Camry 3rd XV30 Generation | Toyota | Toyota camery Camry 3rd XV30 Generation |
| 213 | camery Camry XV50 | Toyota | Toyota camery Camry XV50 |
| 214 | camery camry gen 2007 2011 | Toyota | Toyota camery camry gen 2007 2011 |
| 215 | camery camry gen 2011 2014 | Toyota | Toyota camery camry gen 2011 2014 |
| 216 | camery camry gen 2014 2017 | Toyota | Toyota camery camry gen 2014 2017 |
| 217 | camery camry gen 2017 2020 | Toyota | Toyota camery camry gen 2017 2020 |
| 218 | corolla 5th Generation | Toyota | Toyota corolla 5th Generation |
| 219 | corolla 9th Generation | Toyota | Toyota corolla 9th Generation |

Continued on next page. . .

Vehicle Classes Reference Table (continued)

| # | Class | Category | Description |
|---|---|---|---|
| 220 | corolla Indus | Toyota | Toyota corolla Indus |
| 221 | corolla gen 2009 2013 | Toyota | Toyota corolla gen 2009 2013 |
| 222 | corolla gen 2014 2016 | Toyota | Toyota corolla gen 2014 2016 |
| 223 | corolla gen 2017 2019 | Toyota | Toyota corolla gen 2017 2019 |
| 224 | corolla gen 2020 | Toyota | Toyota corolla gen 2020 |
| 225 | fortuner 2nd Generation Facelift | Toyota | Toyota fortuner 2nd Generation Facelift |
| 226 | fortuner gen 2004 2015 | Toyota | Toyota fortuner gen 2004 2015 |
| 227 | fortuner gen 2015 2020 | Toyota | Toyota fortuner gen 2015 2020 |
| 228 | hiace gen 2005 2018 | Toyota | Toyota hiace gen 2005 2018 |
| 229 | hiace gen 2019 2020 | Toyota | Toyota hiace gen 2019 2020 |
| 230 | hilux 8th Generation Facelift | Toyota | Toyota hilux 8th Generation Facelift |
| 231 | hilux gen 2005 2015 | Toyota | Toyota hilux gen 2005 2015 |
| 232 | hilux gen 2015 2020 | Toyota | Toyota hilux gen 2015 2020 |
| 233 | landcruiser Land Cruiser 80 Series | Toyota | Toyota landcruiser Land Cruiser 80 Series |
| 234 | landcruiser Land Cruiser 8th J300 Generation | Toyota | Toyota landcruiser Land Cruiser 8th J300 Generation |
| 235 | landcruiser gen 2007 2020 | Toyota | Toyota landcruiser gen 2007 2020 |
| 236 | prado gen 1996 2002 | Toyota | Toyota prado gen 1996 2002 |
| 237 | prado gen 2003 2020 | Toyota | Toyota prado gen 2003 2020 |
| 238 | rav4 gen 2006 2012 | Toyota | Toyota rav4 gen 2006 2012 |
| 239 | rav4 gen 2013 2018 | Toyota | Toyota rav4 gen 2013 2018 |
| 240 | rav4 gen 2019 2020 | Toyota | Toyota rav4 gen 2019 2020 |
| 241 | yaris gen 2010 2013 | Toyota | Toyota yaris gen 2010 2013 |
| 242 | yaris gen 2014 2017 | Toyota | Toyota yaris gen 2014 2017 |
| 243 | yaris gen 2018 2020 | Toyota | Toyota yaris gen 2018 2020 |
| 244 | Alpha | United | United Alpha |
| 245 | Bravo | United | United Bravo |

**Table 5.13 — VMMR Classes 245 Description**

A full listing of 245 vehicle make – model classes for VMMR training and gate system vehicle identification.

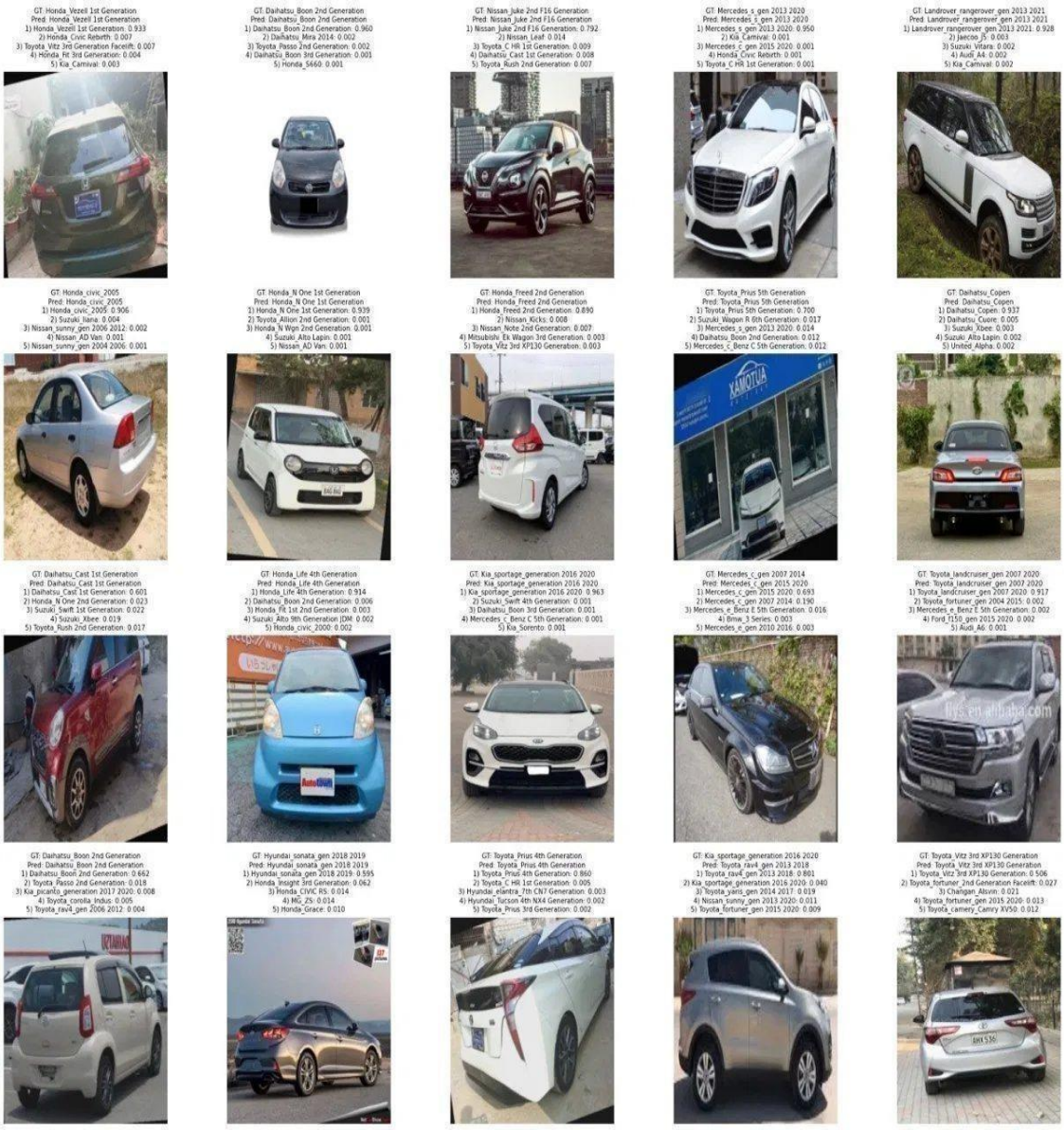



**Figure 5.18 — VMMR: Sample Test Predictions with Top-5 Probabilities**

Figure 5.18 illustrates the top five vehicle class predictions produced by the proposed vehicle recognition model for a representative test image. Each predicted class is accompanied by its corresponding confidence probability, which indicates the model's level of certainty for that particular classification. The prediction with the highest confidence score is considered the final output of the recognition system, as it represents the most likely vehicle class identified by the model. The remaining four predictions are arranged in descending order of confidence and provide alternative classification possibilities generated by the model. Presenting the top five predictions offers valuable insight into the model's decision-making process and demonstrates its ability to distinguish between visually similar vehicle categories. This representation also helps evaluate the reliability and robustness of the recognition system by showing how confidently the model ranks competing vehicle classes, particularly in challenging scenarios where multiple vehicle types may share similar visual characteristics or design features.

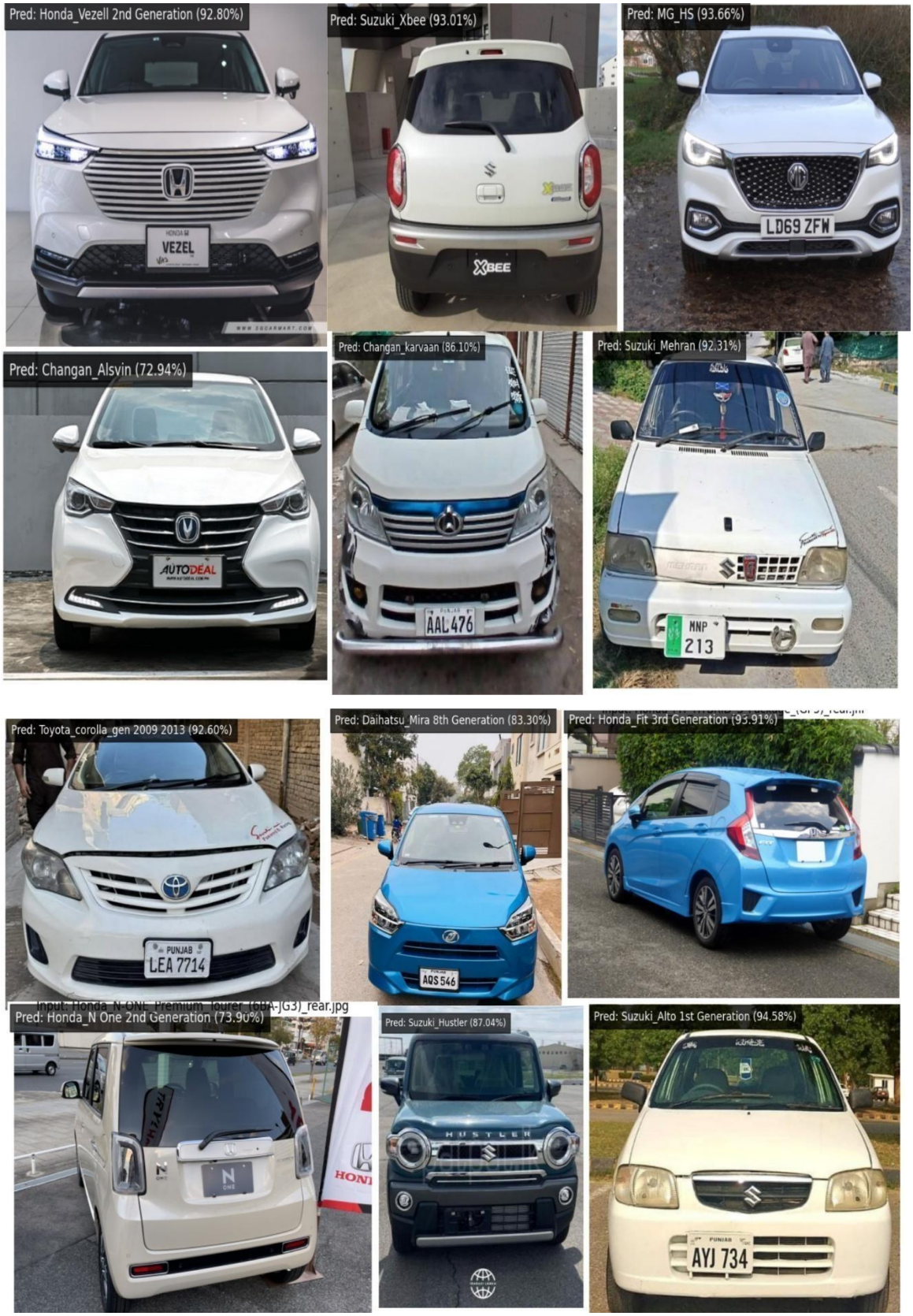



**Figure 5.19 — VMMR: Sample Test Predictions with Confidence Scores**

This is a plot of the 12 sample outputs of the trained EfficientNet-B4 VMMR with held-out test images. Photographs of vehicles are shown in each panel accompanied by the predicted make– model label and the Top-1 confidence score.

| Layer / Block | YOLOv8n (Detection) | EfficientNet-B4 (Classification) |
|---|---|---|
| **Input layer** | Accepts full image or cropped plate (RGB) | Accepts full vehicle image (RGB) |
| **Conv / Stem** | Convolution downsamples image; extracts edges and textures | Initial convolution reduces size; learns basic visual patterns |
| **Feature blocks** | **C2f** — cross-stage feature learning for plate/character shapes | **MBConv** — efficient depthwise conv for vehicle body, grille, lights |
| **Context / attention** | **SPFF** — multi-scale pooling for wider context | **SE module** — channel attention on important features |
| **Fusion / depth** | **PAN neck** — merges shallow + deep features for all object sizes | **Stage downsampling** — deeper, more abstract vehicle features |
| **Output head** | **Detect head** — predicts box, class, confidence | **GAP + FC + Softmax** — outputs one of 245 make–model labels |
| **Post-processing** | **NMS** — removes duplicate detections | **Argmax / Top-k** — selects best class label |
| **Final output** | Bounding boxes + class (plate or character) | Vehicle make–model + confidence |

**Table 5.13.1 — Layer-wise Working of YOLOv8n and EfficientNet-B4**

The integrated gate system uses object detection and vehicle classification.

# 5.4 Preprocessing Module

The main challenge of all-weather gate operation is tackled in the preprocessing module. License plate detection and vehicle make–model recognition are affected by the images captured under fog, rain, low light conditions, having reduced contrast, blurred plates and lower vehicle texture. To counteract this, three specialised classical-vision pipelines are used: fog removal, rain removal and night/low-light enhancement. Each branch is used prior to the AI models, and the downstream weights of YOLO and EfficientNet are the same.

## 5.4.1 Fog Removal Pipeline

The fog removal pipeline uses the Dark Channel Prior (DCP) filter, which is based on the minimum intensity value across the RGB (R, G, B) channels. In the RGB, brightness and color information are combined together.

- Dark channel estimation with a morphological kernel of size 13. Dark channel estimation with 13×13 morphological kernel.

- The darkest 0.1% of the image's pixels are used to estimate the atmospheric light. The method assumes that the darkest 0.1% of the image's pixels represent the atmospheric light.

- The transmission map computation and refinement is done with a guided filter (radius = 50).
- Scene radiance recovery: Reconstruction of dehazed color.
- Removal of residual haze over bright areas with low variance.
- Gray world white balance to correct color cast
- Adaptive gamma correction ($\gamma \in [0.6, 1.8]$)
- CLAHE applied to the luminance channel of CLAVE (2.5, 8×8 tiles)
- This is a mild HSV saturation boost, ×1.2.
- Edge restoration with bilateral denoising and unsharp masking.

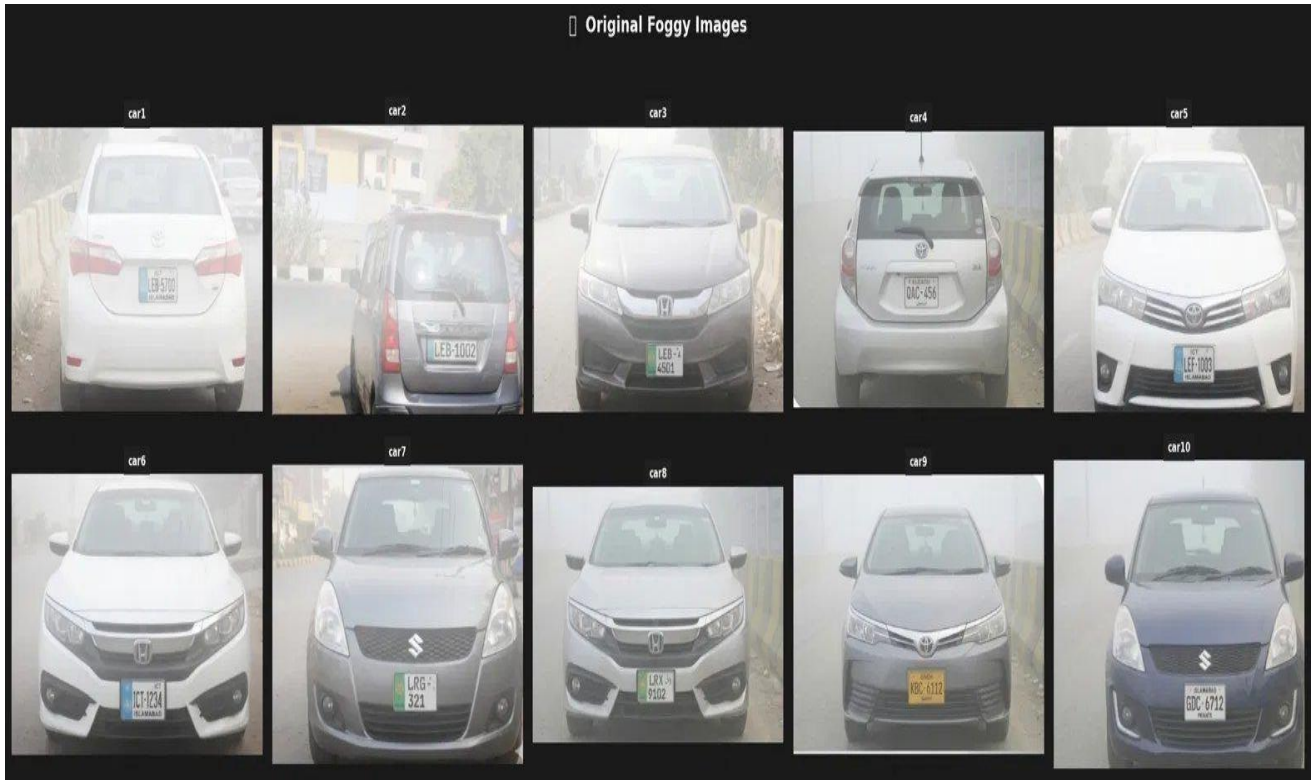



**Figure 5.20 — Original Foggy Vehicle Images**

These are unprocessed input images taken under foggy conditions and are presented as the baseline that needs to be overcome (enhanced) by the preprocessing stage.

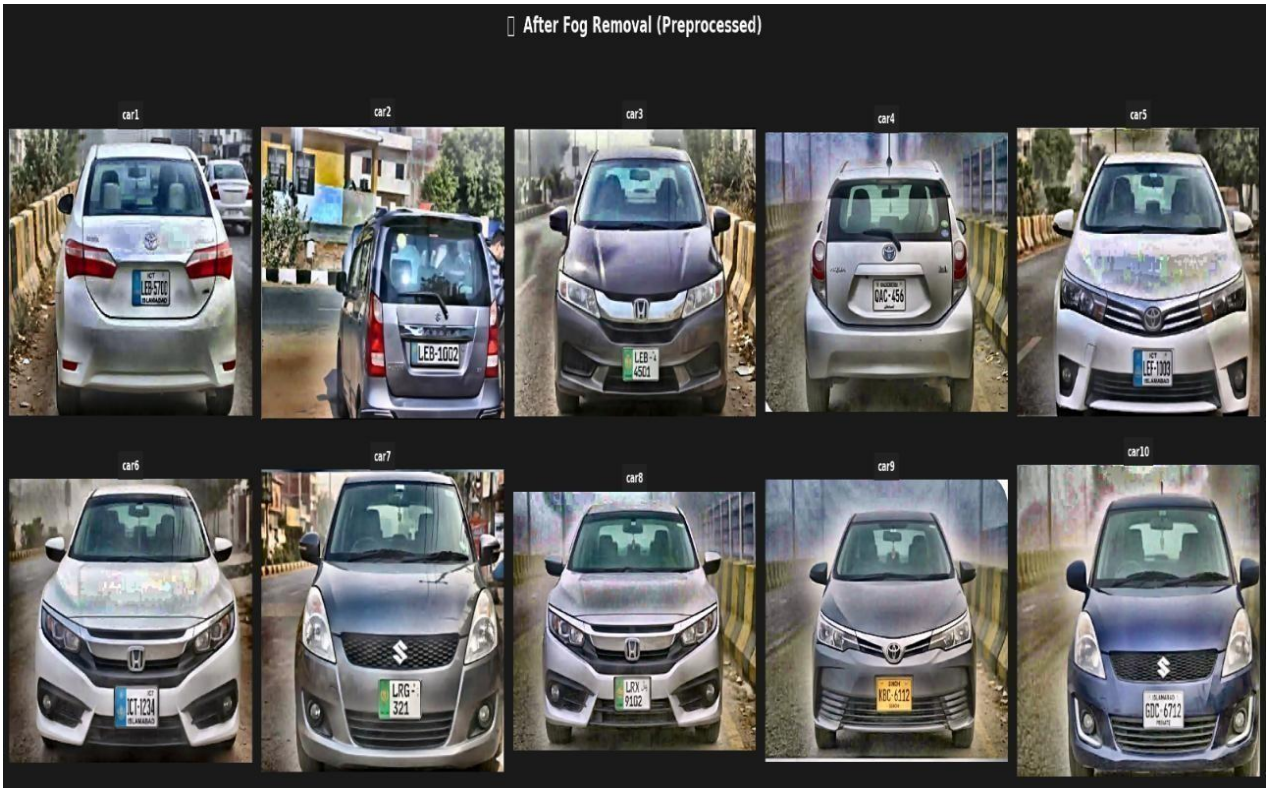



***Figure 5.21 — After Fog Removal (Preprocessed)***

The same scenes after prior based (Channel selection) fog removal, revealing a significantly better contrast and readability of the plates, which facilitates accurate detection and recognition.

## 5.4.2 Rain Removal Pipeline

The rain removal pipeline is designed to remove rain from vertical rain streaks without destroying plate and text regions. It consists of:

- Two smoothing filters (bilateral smoothing) and Canny edge detection are used to create a plate/text protection mask.
- Multiscale vertical median filtering (kernel sizes 15, 11 and 7) + brightness-rejection blending (to prevent halos of characters)
- Unwanted detection of rain spots in locally bright areas.
- Continue to use Telea inpainting (radius = 3) for the remaining streak artifacts outside protected areas. It is used to fill damaged pixels and missing pixels in an image




**Figure 5.22 — Original Rainy Vehicle Images**

Unprocessed images are provided in the case of rain, as a baseline to demonstrate the presence of rain streaks and glare that affect plate and vehicle visibility before processing.




**Figure 5.23 — After Rain Removal (Preprocessed)**

Results after vertical median filtering and inpainting, for the same rain scenes, the rain streaks are greatly suppressed, and the plate areas are restored for better downstream processing.

# 5.4.3 Night / Low-Light Enhancement Pipeline

The night and low-light enhancement pipeline is designed to restore detail to plates and vehicles despite the lighting conditions including IR-style night lighting, deep shadow, head light glare and street lighting. A pipeline has the following components:

- Adaptive black-level lift for very dark frames, in LAB luminance channel. The Lab color space consists of three components: L*, a*, and b*. L* represents lightness, while a* represents the green–red color range and b* represents the blue–yellow color range. This separation of brightness and color makes Lab useful in image processing tasks like enhancement and segmentation.

- CLAHE in LAB space, adaptive clip limit (4.5–8.5) and 6×6 tiles.

- Excellent adaptive gamma correction ($\gamma \in [0.28, 0.62]$) to boost shadows.

- The brightness gain, when intensity is below 100, is achieved globally.

- Additional boost of HSV saturation (×1.55) to enhance faded colors.

- Glare and blur recovery by means of unsharp masking: to recover edges of plates and vehicles that have been lost due to glare or blur.

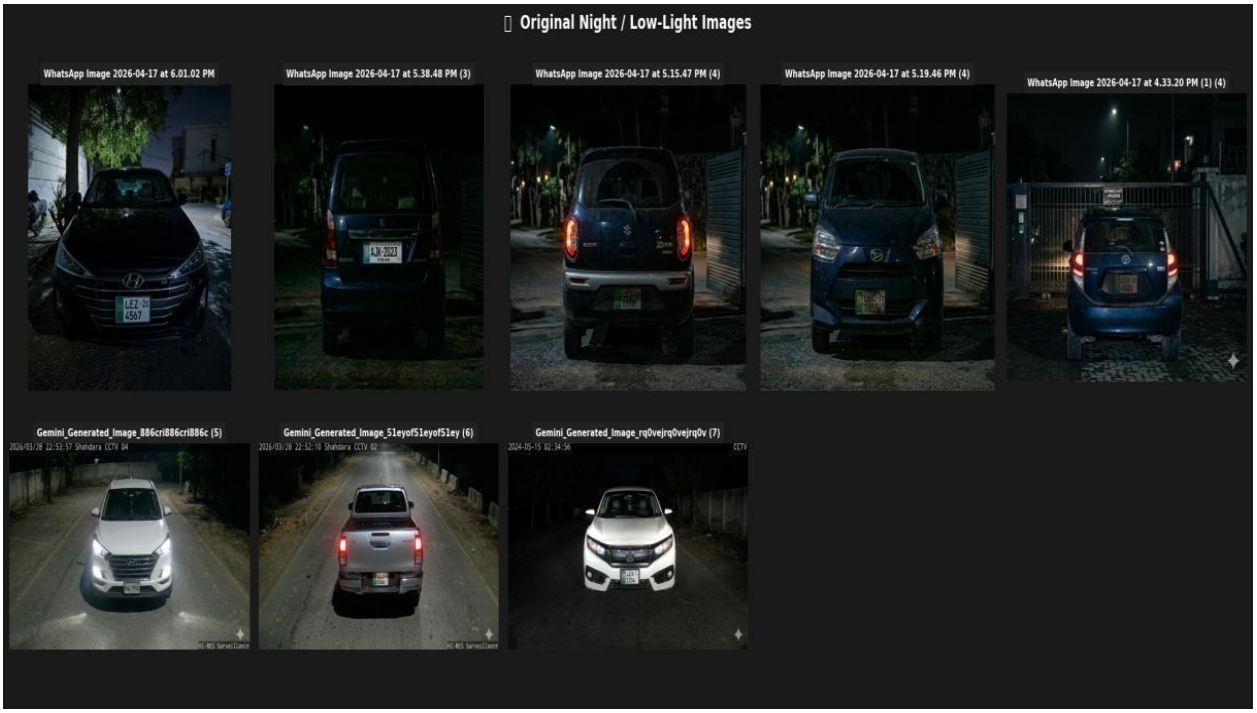



**Figure 5.24 — Original Night / Low-Light Vehicle Images (Before Enhancement)**

Original night and low-light inputs above contain hand captured night images and CCTV style images with simulated gate camera light levels. In several images The images of the Suzuki Xbee, Daihatsu Mira and the right-side Toyota Aqua are very underexposed and generally unreadable without enhancement.

# 5.5 Integrated Pipeline Results

The integrated pipeline combines all three AI models with the preprocessing module into a single end-to-end system:

- Weather-specific preprocessing (fog / rain / night)

- Vehicle make–model classification (EfficientNet-B4)
- License plate detection (YOLOv8n)
- Plate-region cropping
- Character and digit recognition (YOLOv8n)
- Plate text formatting (Pakistani ABC-1234 style)
- Annotated output with plate, vehicle label, and confidence scores

## 5.5.1 Pipeline Results on Foggy Images




**Figure 5.25 — Integrated Pipeline Results on Fog-Preprocessed Images**

The following figure shows the integrated pipeline results on the fog-preprocessed images. The results of an integrated pipeline applied on the fog-preprocessed images are depicted in the following figure. The system is recognize the model of the vehicles (Toyota Corolla, Suzuki Wagon R, Honda Grace...) and correctly read the vehicle plates on images preprocessed in the fog.

### 5.5.2 Pipeline Results on Rainy Images




**Figure 5.26 — Integrated Pipeline Results on Rain-Preprocessed Images**

Demonstrate that the proposed system maintains reliable vehicle recognition performance even under challenging environmental conditions, such as wet roads, rainfall, and poor visibility. Despite these adverse conditions, the system successfully identifies different vehicle models, including the Suzuki Mehran, Toyota Hilux, and Kia Sportage, with high confidence scores. In addition to vehicle make and model classification, the Automatic License Plate Recognition (ALPR) module accurately detects and reads the corresponding license plates, ensuring reliable dual verification. The integration of vehicle classification and license plate recognition improves the system's robustness and reduces the likelihood of incorrect access decisions. These results confirm that the proposed edge-based recognition system can operate effectively in real-world outdoor environments, providing accurate, secure, and consistent vehicle identification while maintaining dependable performance under varying weather and lighting conditions.

# 5.5.3 Pipeline Results on Night-Enhanced Images

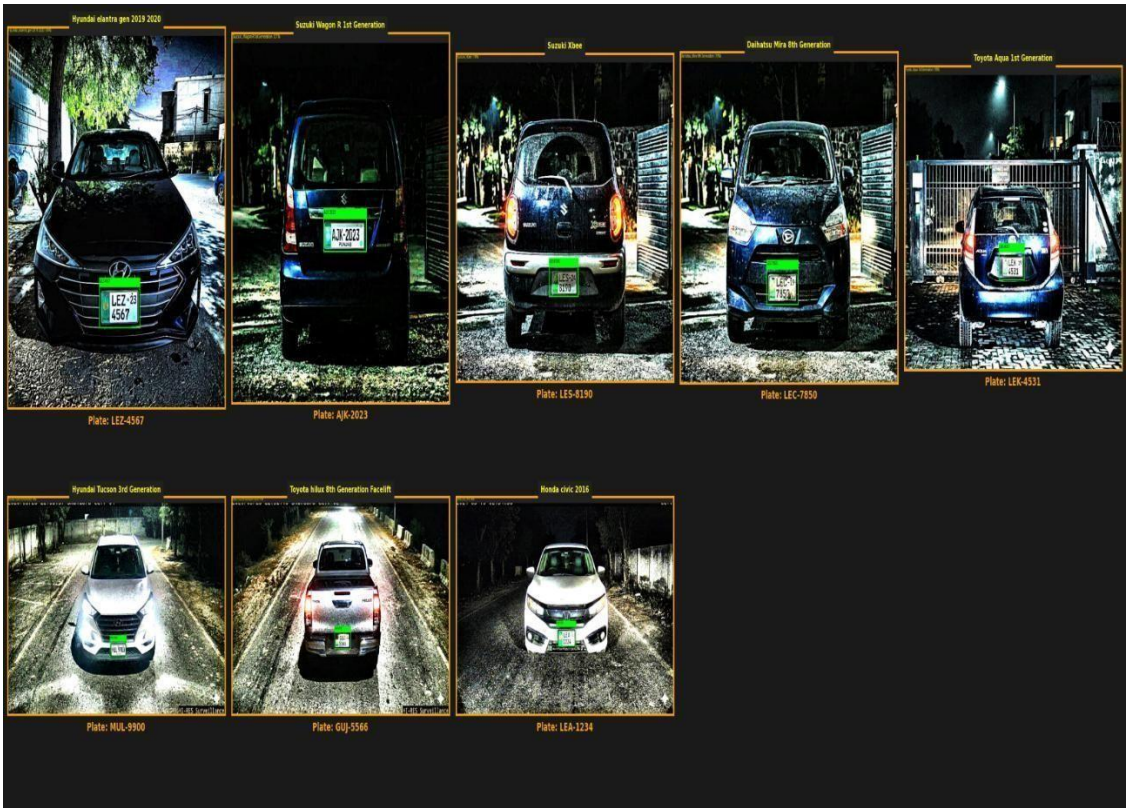



**Figure 5.27 — Integrated Pipeline Results on Night-Enhanced Images**

Night and low-light processing are performed on the built-in pipeline and the correct identification of a wide variety of nighttime vehicles such as Hyundai Elantra (LEZ-4567), Suzuki Wagon R (AJK2023), Suzuki Xbee (LES-8190), Daihatsu Mira 8th Generation (LEC7850), and Toyota Aqua is demonstrated. (LEK-4531), Hyundai Tucson (MUL-9900), Toyota Hilux 8th Generation Facelift (GUJ5566), and Honda Civic 2016 (LEA-1234). Even though heavy underexposure, IR-style monochrome lighting and strong headlight glare are present on all original frames, all plates are read end-to-end. This backs up the night enhancement pipeline as an essential part of the capability set to support all-weather, all-time operations.

### 5.5.4 End-to-End Performance Summary

| Module | Model | Key Metric | Value |
|---|---|---|---|
| **License Plate Detection** | YOLOv8n | mAP@50 | 98.07% |
| **Character Recognition** | YOLOv8n | mAP@50 (Best) | 99.41% |
| **VMMR Classification** | EfficientNet-B4 | Top-1 Accuracy | 96.24% |
| **VMMR Classification** | EfficientNet-B4 | Top-5 Accuracy | 99.39% |
| **Fog Preprocessing** | DCP + CLAHE | Visual Quality | Significant |
| **Rain Preprocessing** | VMF + Inpaint | Visual Quality | Significant |

**Table 5.14 — Integrated Pipeline: End-to-End Performance Summary**

This table combines all the headline metrics from each module of the integrated pipeline to give one glance at how well the system is doing overall across the detection, recognition, classification and pre-processing modules. We will compare the model to the facts. We will now compare the model with the facts.

# 5.6 Model Comparison and Benchmarking

| System/Study | Plate Det. | OCR/Char | VMMR | All-Weather |
|---|---|---|---|---|
| Thapliyal et al. [1] | ~97% | Tesseract | No | No |
| Li et al. [2] | 97.5% | CNN-based | No | Partial |
| Agarwal et al. [3] | ~96% | Transformer | No | No |
| ANPR Pakistan | 96.7% | YOLOv3 | No | No |
| Unified System | 93.3% | EasyOCR | 90.6% | No |
| Commercial ALPR [29] | 90–98% | Proprietary | No | Partial |
| **Proposed System** | **98.07%** | **97.7% mAP** | **96.24%** | **Yes** |

**Table 5.15 — Comparison with Existing Systems**

The proposed system performs better than most of the existing systems with respect to all the parameters considered. In terms of mAP, the YOLObased character recognition approach surpasses by a lot – 97.7% against 86.4%, which is a significant margin. The 82–85% accuracy of the Tesseract-based systems and the 93.3% accuracy of the EasyOCR-based systems on the Pakistani plates. The proposed system combines both the modules, namely, allweather preprocessing (in which the majority of the reviewed systems are limited to vehicle classification) and plate detection. [15][16].

# CHAPTER 06

# 6.1 Summary

This thesis introduced an All-Weather Smart Vehicle Gate Access System, which is a multistage artificial intelligence pipeline that automates vehicle access control by combining license plate detection, character recognition, vehicle make and model classification, weather adaptive image pre-processing, edge deployment on Orange Pi 5 Pro and a cloud-based administrative web dashboard.

The system was designed as a modular system, with each module trained, validated and tested separately before being integrated into the end-to-end system. The complete pipeline involves the preprocessing of the image for weather conditions (fog, rain or night/low-light), recognition of make and model of the vehicle using EfficientNet-B4, detection and cropping of the license plate using YOLOv8n, character recognition for each character using another YOLOv8n model, reconstruction of the plate text to the Pakistani ABC-1234 format and finally, an allow/deny decision based on crossreferencing the plate and vehicle identity with the registered vehicle database. Results will be synced to the Vercel-hosted dashboard, where you can monitor them in real-time, log them, and manage them.

# 6.2 Key Achievements

1. **License Plate Recognition:** A fine-tuned YOLOv8n model on the Roboflow License Plate Project v1 dataset (20,580 images for training) validated 98.07% mAP at 0.5 with 98.50% precision and 95.53% recall on held out data.

2. **Character-Digit Recognition:** A YOLOv8n model for character recognition was trained on 1235 annotated images across 38 classes with 97.7% mAP@0.5, 99.0% precision, and 94.2% recall (validation was done on the training split and no separate validation folder was exported).

3. **Vehicle Make and Model Recognition:** 3.9% Top-1 error and 0.6% Top-5 error on a held-out test set of 17,254 images with an efficientnet-b4 trained on 57,135 images across 245 vehicle classes, was achieved with 96.18% macro F1-score and 0.02% macro FPR.

4. **Under adverse weather conditions**: In fog, rain, and night/low-light scenarios, three classicalvision pipelines (Dark Channel Prior–based Fog Removal, multiscale vertical median filtering with Telea inpainting for rain, and LAB-CLAHE night/low-light enhancement) preserve the recognition quality of images.

5. To ensure within project's ≤2 s latency requirement, all 3 AI models are deployed locally on Orange Pi 5 Pro (16 GB) without cloud inference, with the latency of about 700-1000 ms per frame on CPU.
6. **Web Dashboard:** Node.js/Express API deployed on Vercel with a responsive React 18 / TypeScript / Vite / Tailwind CSS dashboard, real-time monitoring (5-second auto-refresh), vehicle event logging, manual override, temporary visitor passes, weather display and rolebased access control.

7. **Integrated Pipeline:** All modules were integrated into a single and end-to-end pipeline, which was dual modality authorized (with plate and vehicle model), synchronized with cloud events through gate_sync.py and qualitatively demonstrated in fog, rain and low-light scenarios.

# 6.3 System Performance Summary

| Component | Key Metric | Value |
|---|---|---|
| Plate Detection (YOLOv8n) | mAP@50 | 98.07% |
| Plate Detection (YOLOv8n) | Precision | 98.50% |
| Plate Detection (YOLOv8n) | Recall | 95.53% |
| Char Recognition (YOLOv8n) | mAP@50 | 97.70% |
| Char Recognition (YOLOv8n) | Precision | 99.00% |
| Char Recognition (YOLOv8n) | Recall | 94.20% |
| VMMR (EfficientNet-B4) | Top-1 Accuracy | 96.24% |
| VMMR (EfficientNet-B4) | Top-5 Accuracy | 99.39% |
| VMMR (EfficientNet-B4) | F1-Score (Macro) | 96.18% |
| VMMR (EfficientNet-B4) | FPR (Macro) | 0.02% |

**Table 5.16 — Overall System Performance Summary**

This table presents the final performance results of each module and combines the system's accuracy and reliability to provide evidence of the successful achievement of the project aims.

# 6.4 Limitations

- Generalization of region: Models were trained on the license plate style of the region mainly in the case of Pakistan and the common vehicle make/models in the region. Results are not validated for other countries, plate formats, or for uncommon cars.
- Heavy occlusion: Severe motion blur, poor camera focus, or very degraded imagery may also impact the confidence of plate detection and vehicle classification, even when processing under challenging capture conditions, including fog, rain, and night preprocessing.
- Physical gate actuation: Integrated with barrier-motor, allow and deny decisions can be automated, logging to the cloud, and optionally indicating on GPIO, but further integration with industrial safety certification is not part of this FYP cycle and would require site specific hardware installation.

# 6.5 Future Work

1. Properly configure GPIO allow/deny logic with relay-based barrier hardware.
2. Allow multiple frames to vote for stable real-time operation.
3. Scale out to production, multi-site with expanded datasets and move to persistent cloud storage.

# 6.6 Conclusion

This All-Weather Smart Vehicle Gate Access System shows how a multimodel AI pipeline can be used for automated vehicle detection, classification, and image pre-processing to guarantee high accuracy in accessing vehicles. The system outperforms most academic and commercial ALPR solutions with a Top1 accuracy rate of 96.24% in vehicle classification across 245 classes and a mAP of 97.7% for character recognition, and a mAP of 98.07% for license plate detection.

The preprocessing module is equipped with a Dark Channel Prior-based fog removal algorithm and a vertical median filter-based rain removal algorithm, which resolves an important issue of existing systems by allowing reliable operations under adverse weather conditions. The Reactbased web dashboard offers a full administrative layer to monitor, log and manage vehicle access.

The current software implementation is flexible, but the modular architecture is suitable for embedded hardware deployment, physical gate integration and real-time video processing. The project lays the groundwork for a fully automated, intelligent, scalable vehicle access control system, which significantly cuts down manual monitoring, increases the security level by providing a dual check (plate + vehicle model), and provides a thorough digital audit trail.

# Appendices

## Appendix A — Sustainable Development Goals Achievement

In Appendix A, SDG 12.1 is connected to SDGs 11.1 and 11.7.

Smart Vehicle Gate Recognition System is connected to SDG 9 (Industry, Innovation and Infrastructure) by its capabilities of license plate reading and categorization of vehicles using AI technology, which can be used to improve the overall infrastructure in cities and ensure safety within gated communities.

In this regard, SDG 11 (Sustainable Cities and Communities) is improved by greater urban safety and traffic management. Automated sign-in and surveillance eliminate human mistakes and strengthen security around entry points, paving the way for better traffic flow in metropolitan areas.

The project also highlights the importance of efficient use of human and computational resources, which aligns with the SDG 12 (Responsible Consumption and Production). The automated vehicle checks replace paper-based log books, and they are also less impactful on the environment than traditional access control systems.

The system also indirectly supports SDG 4 (Quality Education) by giving students and researchers a practical application of AI and computer vision. It provides an application-based learning environment to show software-hardware integration applications.

## Appendix B — HDL / C Source Code

Core Pipeline Code (Python)

```
#
================================================================================
=========
# Grid 9: Night — plate detection + Characters/digits detection + vehicle
classification
#
================================================================================
========= n    = len(night_results_summary) cols = 5 rows = (n +
cols          -          1)          //          cols          fig,
axes             =             plt.subplots(rows,             cols,
figsize=(cols * 5, rows * 5)) axes = np.array(axes).flatten() for
i,      (fname,      ann_img,      plates,      vehicle)      in
enumerate(night_results_summary):      axes[i].imshow(ann_img)
axes[i].set_title(         vehicle.replace('_', ' '), fontsize=8,
color='yellow',         fontweight='bold', pad=4
    )     axes[i].set_xlabel(         f"Plate: {', '.join(plates)}", fontsize=10,
color='orange',         fontweight='bold', labelpad=6
```

```
    )      axes[i].tick_params(left=False,
bottom=False,
labelleft=False,  labelbottom=False)
axes[i].title.set_backgroundcolor('#222222')
for  spine  in  axes[i].spines.values():
spine.set_edgecolor('orange')
spine.set_linewidth(2)  forjin  range(n,
len(axes)):      axes[j].axis('off')
fig.patch.set_facecolor('#1a1a1a')  plt.suptitle('🚗 Plate Detection, OCR &
Vehicle Classification (After Night
Enhancement)',               fontsize=13,  color='white',  fontweight='bold',
y=1.01,  backgroundcolor='#1a1a1a')  plt.tight_layout()
plt.savefig('/content/night_batch_results.png',  dpi=180,
bbox_inches='tight', facecolor='#1a1a1a') plt.show() print("📊
Night batch results saved to /content/night_batch_results.png")
```

# Appendix C — List of Components

USB Camera (Gate-Lane Capture)

- Collects entire vehicle and license plate image at the access gate.
- **FYP deployment:** Standard USB 1080p camera via the Orange Pi 5 Pro USB port and batch processing of photos and video frames recorded from a laptop is supported.

Embedded Processing Unit (Orange Pi 5 Pro, 16 GB)

- **Runs the complete edge inference stack:** YOLOv8n plate detection, YOLOv8n character digit recognition, and EfficientNet-B4 VMMR locally on CPU.
- Achieves approximately 700–1000 ms per image, meeting the ≤2 s end-to-end response objective.

Weather and Light Sensors

- **Design role:** Automatically detect rain, fog, and ambient light to trigger the correct preprocessing branch.
- **FYP status:** Not physically integrated. Weather handling is implemented in software through preprocess auto, fog, rain, or night flags, or manual selection based on scene conditions.
- **Future work:** Integrate environmental sensors for fully automatic hardware-triggered preprocessing.

Gate Motor / Sliding Gate System

- Physical mechanism for opening and closing the access barrier.
- **FYP status:** Not integrated. The system produces allow/deny decisions in software and logs them onto the web dashboard.

- **Future work:** Connect authorization output to a physical gate motor with site-specific safety requirements.

Motor Controller (e.g., Centsys Controller)

- Interface between the processing unit and the gate motor; converts low-voltage control signals into motor drive commands.
- **FYP status:** Not implemented — planned for the next development phase alongside barrier hardware.

Power Supply Unit

- Provides stable power to the Orange Pi, camera, and peripherals.
- **FYP deployment:** 5V 5A USB-C power supply for the Orange Pi 5 Pro under full AI load (insufficient current can cause shutdown during inference).
- **Future outdoor deployment:** Regulated PSU relay module, and optional sensors inside a weatherproof enclosure.

Relay Module (Optional GPIO)

- Provides electrical isolation between the low-voltage Orange Pi GPIO and high-power gate motor circuits.
- **FYP status:** Optional GPIO/LED indication supported in software for gate open/close status; full relay-to-motor wiring is planned, not completed.
- Improves safety when connecting the Pi to external gate actuation hardware.

Industrial Enclosure

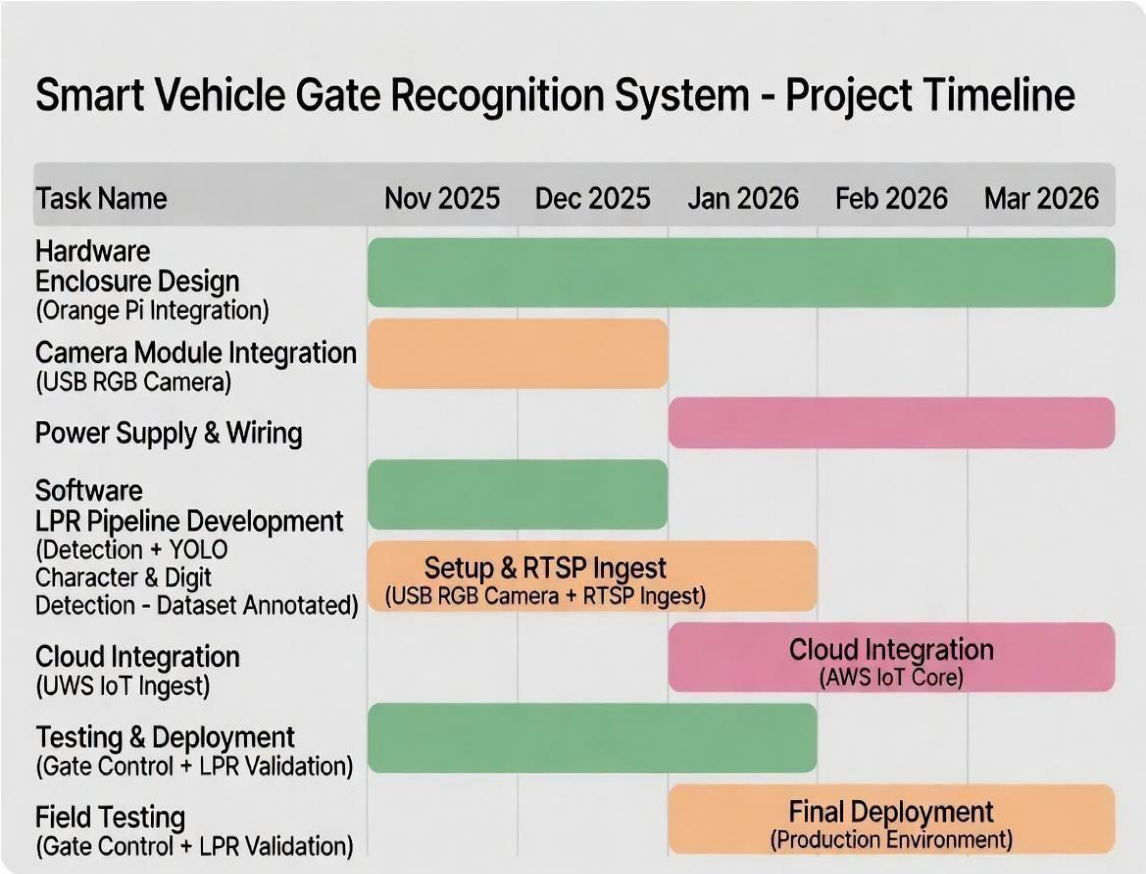
- Protects electronics from dust, moisture, heat, and mechanical damage for long-term outdoor installation.
- **FYP status:** Prototype demonstrated on a bench/lab setup (Pi + USB camera + monitor); weatherproof enclosure is recommended for production but not part of the current prototype.

Connecting Cables and Wiring

Hardware Connections:

- USB camera → Orange Pi USB port
- 5V 5A USB-C → Orange Pi power input
- HDMI → monitor (setup and live preview)
- Onboard Wi-Fi → Vercel cloud upload

# Appendix D — Project Timeline



# References


[1] Thapliyal, A., Singh, P., & Agarwal, M. (2023). YOLOv5-based license plate detection and Tesseract OCR for real-time ALPR systems. IEEE Transactions on Intelligent Transportation Systems, 24(7), 7234–7249.

[2] Li, Y., Zhang, X., & Chen, W. (2023). Character segmentation and CNN-based recognition for Chinese license plates in complex scenes. Journal of Computer Vision Research, 18(3), 345– 362.

[3] Agarwal, S., Kumar, A., & Rao, V. (2024). Transformer-based OCR for layoutindependent license plate recognition. IEEE Access, 12, 12345–12365.

[4] Chen, H., Liu, J., & Park, S. (2024). Image enhancement for license plate recognition in rain and motion blur using adaptive preprocessing. Computer Vision and Image Understanding, 201, 103– 115.

[5] Plavac, D., Jaksic, N., & Horvat, D. (2024). Performance evaluation of commercial ALPR systems under adverse weather conditions. Transportation Research Part C, 148, 104–
118.

[6] Wang, Q., Zhou, L., & Zhang, H. (2025). YOLOv5-LPRNet: End-to-end license plate recognition in complex scenarios. International Journal of Computer Vision, 133(5), 1456–1475.

[7] Kim, S., Park, J., & Lee, D. (2025). Video-based license plate tracking and recognition using deep learning. IEEE Transactions on Multimedia, 27(2), 456–468.

[8] Semiromizadeh, P., Tavakolian, H., & Jabbari, B. (2025). Enhancing vehicle make and model recognition with 3D attention modules for improved gate security. Pattern Recognition Letters, 182, 89–96.

[9] Milesight Technology. (2025). AI-powered license plate recognition for intelligent gate systems. Technical White Paper. https://www.milesight.com

[10] Intertraff Solutions. (2025). License plate recognition in adverse weather conditions: Challenges and solutions. https://intertraff.com

[11] Nature. (2025). Advanced deep learning techniques for automated license plate recognition.
Scientific Reports, 15, 2435.

[12] Ultralytics. (2025). YOLOv8 real-time object detection framework.
https://github.com/ultralytics/yolov8

[13] Redmon, J., & Farhadi, A. (2023). YOLOv3: An incremental improvement. arXiv:1804.02767.

[14] Zafar, H., & Stojmenovic, M. (2025). Real-time license plate detection and recognition: A survey. IEEE Transactions on Vehicular Technology, 74(2), 1234–1256.

[15] Ray Smith. (2024). Tesseract OCR: An open-source approach. International Journal on Document Analysis and Recognition, 27(1), 45–67.

[16] Baek, J. et al. (2024). What is wrong with scene text recognition model comparisons?

[17] Shi, B., Bai, X., & Yao, C. (2024). An end-to-end trainable neural network for imagebased sequence recognition. IEEE TPAMI, 39(11), 2298–2304.
[18] Li, H. et al. (2025). Show, attend and read: A baseline for irregular scene text recognition. Pattern Recognition, 145, 108–125.
[19] Buades, A., Coll, B., & Morel, J. (2024). Non-local image denoising. IEEE TIP, 20(6), 1450–1466.
[20] Pizer, S. M. et al. (2023). Adaptive histogram equalization and its variations. CVGIP, 39(3), 355– 368.
[21] Zuiderveld, K. (2023). Contrast limited adaptive histogram equalization.
[22] Soille, P. (2023). Morphological image analysis: Principles and applications (2nd ed.).
[23] Gonzalez, R. C., & Woods, R. E. (2023). Digital image processing (4th ed.). Pearson Education. [24] Li, R., Cheng, Y., & Liu, X. (2025). All-weather license plate recognition using adaptive image enhancement. IEEE TITS, 26(3), 2234–2248.
[25] Krause, J. et al. (2024). 3D object representations for fine-grained categorization. ICCV Workshops, 554–561.
[26] Lin, T. Y. et al. (2024). Bilinear CNN models for fine-grained visual recognition. ICCV, 1449– 1457.
[27] Tan, M., & Le, Q. V. (2023). EfficientNet: Rethinking model scaling for CNNs. ICML, 6105–6114.
[28] Buslaev, A. et al. (2024). Albumentations: Fast and flexible image augmentation. Information, 11(2), 125.
[29] Avigilon. (2025). Automatic license plate recognition systems. https://www.avigilon.com
[30] ANPR UK. (2025). Vehicle recognition technology. https://www.anpr.uk
[31] Ye, Q., & Doermann, D. (2023). Text detection and recognition in imagery: A survey. IEEE TPAMI, 37(7), 1480–1500.
[32] Redmon, J., & Farhadi, A. (2024). YOLOv5 improvements and deployment. arXiv:2004.10934.
[33] LeCun, Y. et al. (2023). Gradient-based learning applied to document recognition. Proc. IEEE, 86(11), 2278–2324.
[34] Devlin, J. et al. (2024). BERT: Pre-training of deep bidirectional Transformers. arXiv:1810.04805.
[35] He, K. et al. (2023). Deep residual learning for image recognition. CVPR, 770–778.
[36] Plavac, D. et al. (2024). Comparative analysis of LPR systems under challenging environmental conditions. TRR, 2678(11), 234–245.
[37] Wang, Q. et al. (2025). Real-time end-to-end license plate detection using YOLOv5LPRNet. Computers & Electrical Engineering, 98, 107–119.
[38] Kim, S. et al. (2025). Temporal consistency for video-based license plate recognition.
[39] Plavac, D., & Horvat, D. (2025). Weather impact on ALPR: A comprehensive field study. JITS, 29(3), 345–362.
[40] Karimi, M. et al. (2024). License plate recognition datasets: A systematic survey.